

STAR RX72N - rx\_comm\_manager Library  
1.0.0

Generated by Doxygen 1.16.1



|                                                               |    |
|---------------------------------------------------------------|----|
| 1 Directory Hierarchy                                         | 1  |
| 1.1 Directories                                               | 1  |
| 2 File Index                                                  | 3  |
| 2.1 File List                                                 | 3  |
| 3 Directory Documentation                                     | 5  |
| 3.1 libs/rx_comm_manager/inc Directory Reference              | 5  |
| 3.2 libs Directory Reference                                  | 5  |
| 3.3 libs/rx_comm_manager Directory Reference                  | 6  |
| 3.4 libs/rx_comm_manager/src Directory Reference              | 6  |
| 4 File Documentation                                          | 7  |
| 4.1 libs/rx_comm_manager/inc/rx_comm_manager.h File Reference | 7  |
| 4.1.1 Detailed Description                                    | 9  |
| 4.1.1.1 Overview                                              | 9  |
| 4.1.1.2 System Architecture                                   | 10 |
| 4.1.1.3 Channel Configuration                                 | 11 |
| 4.1.1.4 Communication Flow State Machine                      | 11 |
| 4.1.1.5 Hardware Requirements                                 | 12 |
| 4.1.1.6 Performance Characteristics                           | 13 |
| 4.1.1.7 Memory Usage                                          | 13 |
| 4.1.1.8 Thread Safety Analysis                                | 14 |
| 4.1.1.9 Integration Example                                   | 14 |
| 4.1.2 Data Structure Documentation                            | 18 |
| 4.1.2.1 struct rx_comm_event_entry_t                          | 18 |
| 4.1.2.2 struct rx_comm_heartbeat_state_t                      | 18 |
| 4.1.2.3 struct rx_comm_manager_config_t                       | 19 |
| 4.1.2.4 struct rx_comm_manager_t                              | 27 |
| 4.1.2.5 struct rx_comm_send_params_t                          | 31 |
| 4.1.3 Typedef Documentation                                   | 32 |
| 4.1.3.1 rx_comm_frame_callback_t                              | 32 |
| 4.1.3.2 Callback Contract                                     | 33 |
| 4.1.3.3 rx_comm_link_status_callback_t                        | 35 |
| 4.1.4 Enumeration Type Documentation                          | 35 |
| 4.1.4.1 rx_comm_channel_mask_t                                | 35 |
| 4.1.4.2 rx_comm_channel_t                                     | 36 |
| 4.1.4.3 rx_comm_event_retry_constants_t                       | 38 |
| 4.1.4.4 rx_comm_heartbeat_constants_t                         | 39 |
| 4.1.4.5 rx_comm_link_status_t                                 | 39 |
| 4.1.4.6 rx_comm_manager_constants_t                           | 40 |
| 4.1.5 Function Documentation                                  | 41 |
| 4.1.5.1 rx_comm_manager_channel_name()                        | 41 |
| 4.1.5.2 rx_comm_manager_channel_ready()                       | 43 |

|                                                               |     |
|---------------------------------------------------------------|-----|
| 4.1.5.3 rx_comm_manager_deinit()                              | 45  |
| 4.1.5.4 rx_comm_manager_event_send()                          | 47  |
| 4.1.5.5 rx_comm_manager_init()                                | 49  |
| 4.1.5.6 rx_comm_manager_link_status()                         | 53  |
| 4.1.5.7 rx_comm_manager_poll()                                | 53  |
| 4.1.5.8 rx_comm_manager_process_retransmits()                 | 55  |
| 4.1.5.9 rx_comm_manager_respond()                             | 56  |
| 4.1.5.10 rx_comm_manager_send()                               | 58  |
| 4.1.5.11 rx_comm_manager_set_auto_retransmit()                | 59  |
| 4.1.5.12 rx_comm_manager_stream_send()                        | 60  |
| 4.2 rx_comm_manager.h                                         | 61  |
| 4.3 libs/rx_comm_manager/src/rx_comm_manager.c File Reference | 77  |
| 4.3.1 Detailed Description                                    | 79  |
| 4.3.1.1 Overview                                              | 79  |
| 4.3.1.2 Implementation Architecture                           | 80  |
| 4.3.1.3 Polling State Machine                                 | 80  |
| 4.3.1.4 Performance Characteristics                           | 81  |
| 4.3.1.5 Memory Usage                                          | 81  |
| 4.3.1.6 Algorithm Details                                     | 81  |
| 4.3.1.7 Thread Safety Analysis                                | 83  |
| 4.3.1.8 Integration Example                                   | 83  |
| 4.3.1.9 NASA Power of 10 Compliance                           | 84  |
| 4.3.1.10 SOLID Principles                                     | 85  |
| 4.3.1.11 Module Dependencies                                  | 85  |
| 4.3.2 Enumeration Type Documentation                          | 86  |
| 4.3.2.1 internal_constants_t                                  | 86  |
| 4.3.2.2 rx_comm_sim_time_t                                    | 88  |
| 4.3.3 Function Documentation                                  | 88  |
| 4.3.3.1 internal_check_channel_heartbeat()                    | 88  |
| 4.3.3.2 internal_check_heartbeat()                            | 90  |
| 4.3.3.3 internal_dequeue_event()                              | 90  |
| 4.3.3.4 internal_get_time_ms()                                | 91  |
| 4.3.3.5 internal_handle_frame()                               | 92  |
| 4.3.3.6 internal_output_decoded()                             | 94  |
| 4.3.3.7 internal_output_decoded_from_params()                 | 95  |
| 4.3.3.8 internal_poll_all_channels()                          | 96  |
| 4.3.3.9 internal_poll_i2c()                                   | 100 |
| 4.3.3.10 internal_poll_spi()                                  | 101 |
| 4.3.3.11 internal_poll_uart()                                 | 103 |
| 4.3.3.12 internal_process_event_queue()                       | 105 |
| 4.3.3.13 internal_route_send()                                | 106 |
| 4.3.3.14 rx_comm_manager_channel_name()                       | 109 |
| 4.3.3.15 rx_comm_manager_channel_ready()                      | 111 |

---

|                                                |     |
|------------------------------------------------|-----|
| 4.3.3.16 rx_comm_manager_deinit()              | 113 |
| 4.3.3.17 rx_comm_manager_event_send()          | 114 |
| 4.3.3.18 rx_comm_manager_init()                | 116 |
| 4.3.3.19 rx_comm_manager_link_status()         | 119 |
| 4.3.3.20 rx_comm_manager_poll()                | 120 |
| 4.3.3.21 rx_comm_manager_process_retransmits() | 121 |
| 4.3.3.22 rx_comm_manager_respond()             | 122 |
| 4.3.3.23 rx_comm_manager_send()                | 124 |
| 4.3.3.24 rx_comm_manager_set_auto_retransmit() | 126 |
| 4.3.3.25 rx_comm_manager_stream_send()         | 126 |
| 4.3.4 Variable Documentation                   | 128 |
| 4.3.4.1 s_tag                                  | 128 |
| 4.3.4.2 s_zero_mgr                             | 128 |
| 4.4 rx_comm_manager.c                          | 129 |



# Chapter 1

## Directory Hierarchy

### 1.1 Directories

|                   |    |
|-------------------|----|
| inc               | 5  |
| rx_comm_manager.h | 7  |
| libs              | 5  |
| rx_comm_manager   | 6  |
| inc               | 5  |
| rx_comm_manager.h | 7  |
| src               | 6  |
| rx_comm_manager.c | 77 |
| rx_comm_manager   | 6  |
| inc               | 5  |
| rx_comm_manager.h | 7  |
| src               | 6  |
| rx_comm_manager.c | 77 |
| src               | 6  |
| rx_comm_manager.c | 77 |





## Chapter 2

# File Index

### 2.1 File List

Here is a list of all files with brief descriptions:

|                                                                               |                    |
|-------------------------------------------------------------------------------|--------------------|
| libs/rx_comm_manager/inc/ <a href="#">rx_comm_manager.h</a>                   |                    |
| Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols . . | <a href="#">7</a>  |
| libs/rx_comm_manager/src/ <a href="#">rx_comm_manager.c</a>                   |                    |
| Unified Multi-Channel Communication Manager Implementation . . . . .          | <a href="#">77</a> |



## Chapter 3

# Directory Documentation

### 3.1 libs/rx\_comm\_manager/inc Directory Reference

Directory dependency graph for inc:

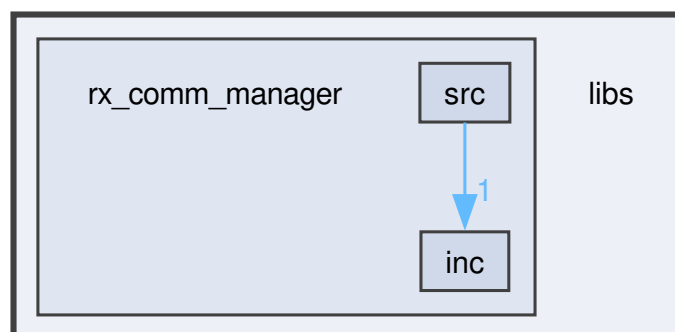


Files

- file [rx\\_comm\\_manager.h](#)  
Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

### 3.2 libs Directory Reference

Directory dependency graph for libs:

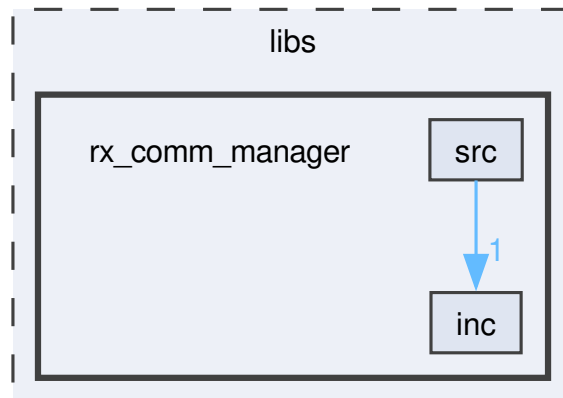


## Directories

- directory [rx\\_comm\\_manager](#)

### 3.3 libs/rx`comm`manager Directory Reference

Directory dependency graph for rx\_comm\_manager:

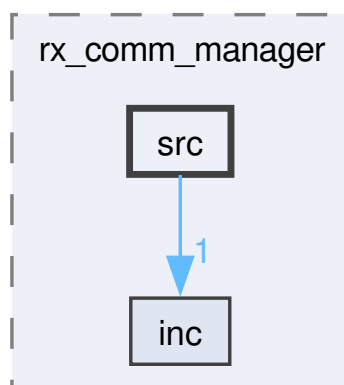


## Directories

- directory [inc](#)
- directory [src](#)

### 3.4 libs/rx`comm`manager/src Directory Reference

Directory dependency graph for src:



## Files

- file [rx\\_comm\\_manager.c](#)  
Unified Multi-Channel Communication Manager Implementation.

## Chapter 4

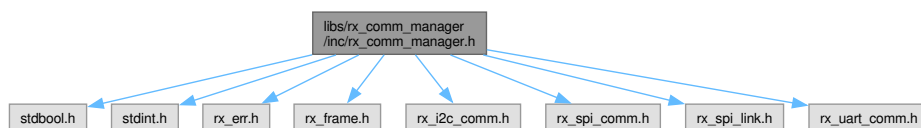
# File Documentation

### 4.1 libs/rx\_comm\_manager/inc/rx\_comm\_manager.h File Reference

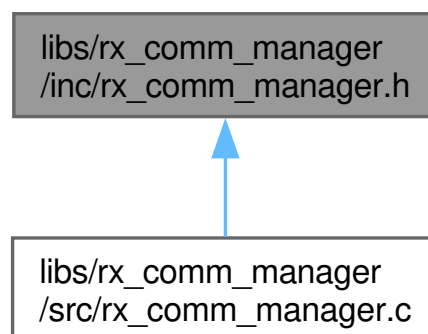
Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_frame.h"
#include "rx_i2c_comm.h"
#include "rx_spi_comm.h"
#include "rx_spi_link.h"
#include "rx_uart_comm.h"
```

Include dependency graph for rx\_comm\_manager.h:



This graph shows which files directly or indirectly include this file:



## Data Structures

- struct `rx_comm_event_entry_t`  
Single entry in the static event FIFO queue.
- struct `rx_comm_heartbeat_state_t`  
Per-transport heartbeat tracking state.
- struct `rx_comm_manager_config_t`  
Communication manager configuration.
- struct `rx_comm_manager_t`  
Communication manager handle.
- struct `rx_comm_send_params_t`  
Parameters for `rx_comm_manager_send`.

## Typedefs

- typedef void(\* `rx_comm_frame_callback_t`) (`rx_comm_channel_t` channel, const `rx_frame_t` \*frame, void \*ctx)  
Frame received callback function type.
- typedef void(\* `rx_comm_link_status_callback_t`) (`rx_comm_channel_t` channel, `rx_comm_link_status_t` new\_status, void \*ctx)  
Callback invoked when a link status changes.

## Enumerations

- enum `rx_comm_manager_constants_t` : `uint16_t` { `k_comm_manager_ascii_buf_size` = 2048 , `k_comm_event_queue_depth` = 8 }  
Communication manager compile-time constants.
- enum `rx_comm_heartbeat_constants_t` : `uint32_t` { `k_heartbeat_implicit_timeout_ms` = 200 , `k_heartbeat_ping_interval_ms` = 1000 , `k_heartbeat_check_interval_ms` = 50 }  
Heartbeat timing constants.
- enum `rx_comm_event_retry_constants_t` : `uint16_t` { `k_event_max_retries` = 3 , `k_event_initial_backoff_ms` = 10 , `k_event_max_backoff_ms` = 15 }  
Event queue retry constants for SPI HARQ.
- enum `rx_comm_channel_t` : `uint8_t` { `k_comm_channel_uart` = 0 , `k_comm_channel_spi` = 1 , `k_comm_channel_i2c` = 2 , `k_comm_channel_count` = 3 }  
Communication channel identifiers.
- enum `rx_comm_link_status_t` : `uint8_t` { `k_link_status_unknown` = 0 , `k_link_status_healthy` = 1 , `k_link_status_dead` = 2 }  
Link health status from heartbeat monitoring.
- enum `rx_comm_channel_mask_t` : `uint8_t` { `k_comm_channel_mask_none` = 0x00U , `k_comm_channel_mask_uart` = 0x01U , `k_comm_channel_mask_spi` = 0x02U , `k_comm_channel_mask_i2c` = 0x04U , `k_comm_channel_mask_all` }  
Bitmask selecting which comm channels to initialize and use.

## Functions

- `rx_err_t rx_comm_manager_init (rx_comm_manager_t *mgr, const rx_comm_manager_config_t *cfg)`  
Initialize communication manager.
- `rx_err_t rx_comm_manager_deinit (rx_comm_manager_t *mgr)`  
Deinitialize communication manager.
- `rx_err_t rx_comm_manager_poll (rx_comm_manager_t *mgr)`  
Poll all enabled channels for incoming frames.
- `rx_err_t rx_comm_manager_send (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)`  
Send frame on specific channel.
- `rx_err_t rx_comm_manager_respond (rx_comm_manager_t *mgr, rx_comm_channel_t channel, const uint8_t *payload, uint32_t payload_len)`  
Send response on same channel as received frame.
- `rx_err_t rx_comm_manager_stream_send (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)`  
Send data on the stream path (fire-and-forget).
- `rx_err_t rx_comm_manager_event_send (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)`  
Queue data on the event path (reliable, SPI retry).
- `rx_err_t rx_comm_manager_link_status (const rx_comm_manager_t *mgr, rx_comm_channel_t channel, rx_comm_link_status_t *status)`  
Query link health status for a transport channel.
- `rx_err_t rx_comm_manager_channel_ready (rx_comm_manager_t *mgr, rx_comm_channel_t channel, bool *ready)`  
Check if channel is ready for communication.
- `const char * rx_comm_manager_channel_name (rx_comm_channel_t channel)`  
Get channel name string.
- `rx_err_t rx_comm_manager_process_retransmits (rx_comm_manager_t *mgr, uint32_t current_time_ms)`  
Process pending retransmissions on SPI channel.
- `rx_err_t rx_comm_manager_set_auto_retransmit (rx_comm_manager_t *mgr, bool enabled, const rx_spi_comm_retransmit_config_t *config)`  
Enable or disable automatic retransmission on SPI channel.

### 4.1.1 Detailed Description

Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

#### 4.1.1.1 Overview

Production-quality communication manager providing unified abstraction over multiple simultaneous communication channels (USB CDC and SPI) with automatic frame protocol handling, non-blocking polling architecture, optional decoded ASCII output for debugging, and callback-based event notification.

Purpose: Coordinate independent, simultaneous communication channels from RX72N MCU to Raspberry Pi 5 host, enabling command/response messaging over both USB (via CDC class) and SPI (direct hardware connection).

Protocol: Both channels use identical frame protocol (`rx_frame`) allowing host to send commands on either channel. Manager automatically handles:

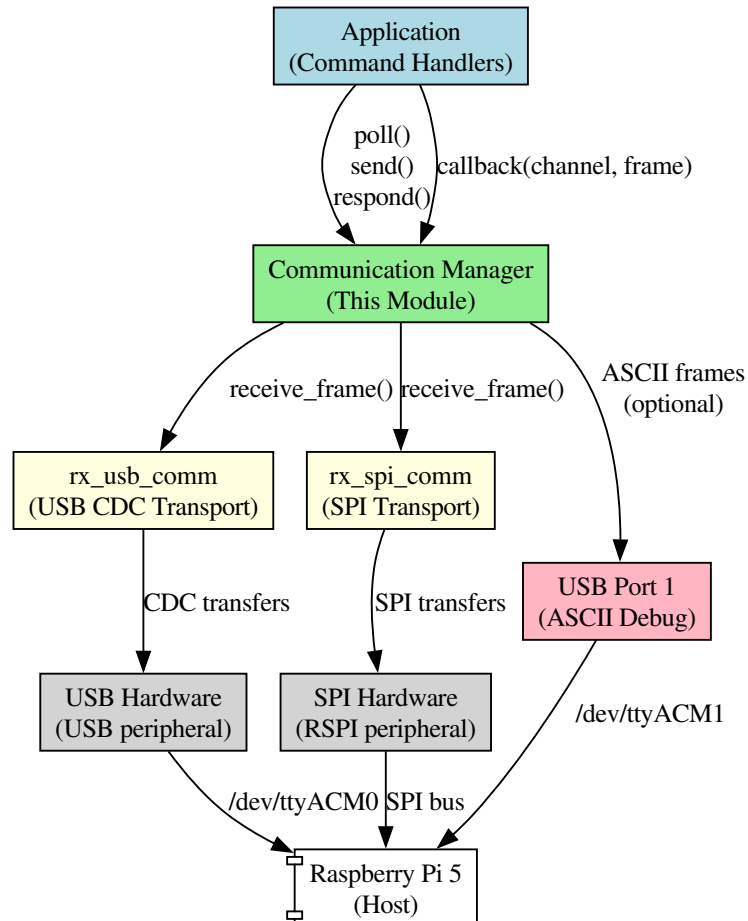
- Frame reception and validation
- ASCII decoding output (debugging via USB Port 1)
- Channel-aware response routing
- User callback invocation

Key Features:

- Multi-channel support: Simultaneous USB and SPI communication
- Non-blocking architecture: Poll-based, no busy-waiting or blocking calls
- Protocol-agnostic: Both channels use same frame format (type, flags, payload, CRC)
- Automatic ASCII output: Optional decoded frames to USB Port 1 (human-readable debugging)
- Callback-driven: User-defined handler invoked on frame reception
- Channel awareness: Responses automatically routed to originating channel
- Zero-copy where possible: Direct access to frame buffers
- Thread-safe design: Can be called from RTOS task or main loop

#### 4.1.1.2 System Architecture

Communication manager sits between application logic and low-level transport drivers:





#### 4.1.1.3 Channel Configuration

The manager supports two independent communication channels:

| Channel | Transport       | Device       | Usage                           | RPi5 Interface |
|---------|-----------------|--------------|---------------------------------|----------------|
| USB     | CDC class       | USB Port 0   | Primary protocol channel        | /dev/ttyACM0   |
| SPI     | RSPI peripheral | RSPI channel | High-speed backup channel       | /dev/spidev0.0 |
| Debug   | CDC class       | USB Port 1   | ASCII decoded output (optional) | /dev/ttyACM1   |

USB Channel (k'comm'channel'usb):

- Plug-and-play connectivity (no additional wiring)
- Full-speed USB 2.0 (up to 12 Mbps theoretical, ~1 MB/s practical)
- Automatic enumeration and driver loading on RPi5
- Suitable for command/response and moderate-bandwidth telemetry

SPI Channel (k'comm'channel'spi):

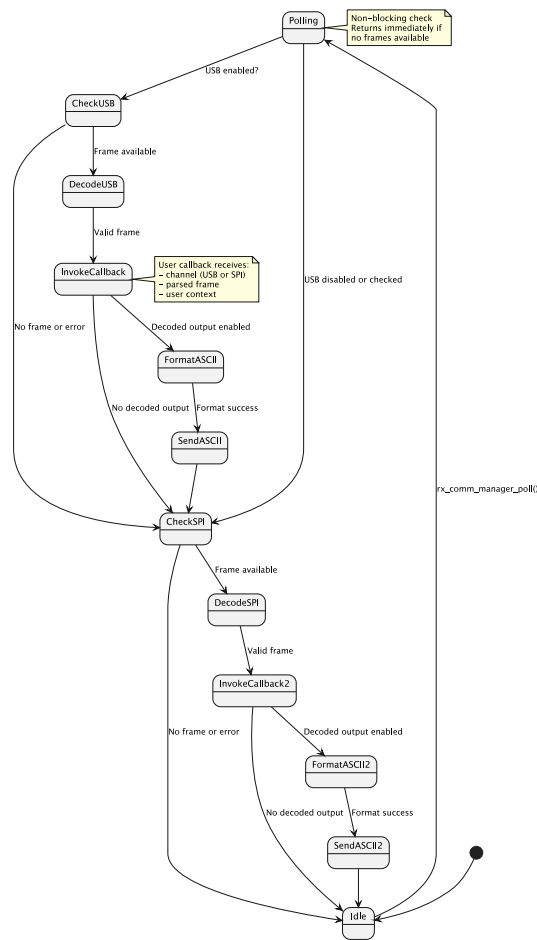
- Dedicated hardware connection (4-6 wires: COPI, CIPO, SCK, CS, optional handshake)
- Configurable speed (typically 10-20 Mbps)
- Lower latency than USB (no protocol overhead)
- Suitable for high-frequency control loops and real-time data

Debug Output (USB Port 1):

- Human-readable ASCII representation of all frames
- Format: [USB/SPI] TYPE=cmd FLAGS=0x00 LEN=42 DATA=...
- Enabled via enable\_decoded\_output config flag
- Zero impact on protocol channels (independent USB endpoint)

#### 4.1.1.4 Communication Flow State Machine

The manager operates as a stateless polling architecture with per-frame state:



#### State Descriptions:

- Idle: Manager initialized, waiting for poll() call
- Polling: Actively checking enabled channels for frames
- CheckUSB: Query USB channel for available frame
- CheckSPI: Query SPI channel for available frame
- DecodeUSB/DecodeSPI: Validate and parse received frame
- InvokeCallback: Call user-provided frame handler
- FormatASCII: Convert frame to human-readable string
- SendASCII: Transmit ASCII representation to debug port

#### 4.1.1.5 Hardware Requirements

| Resource        | Requirement             | Usage                                             |
|-----------------|-------------------------|---------------------------------------------------|
| USB Peripheral  | 1 instance (full-speed) | CDC class with 2 ports (protocol + debug)         |
| RSPI Peripheral | 1 channel (optional)    | SPI controller mode, 10-20 Mbps                   |
| GPIO            | 4-6 pins (if SPI)       | COPI, CIPO, SCK, CS, optional handshake/interrupt |
| RAM             | ~2.5 KB per instance    | Handle + ASCII buffer (2048 bytes)                |
| Flash           | ~4 KB                   | Code + string constants                           |

| Resource       | Requirement | Usage                                    |
|----------------|-------------|------------------------------------------|
| RTOS Resources | None        | Stateless, no threads/mutexes/semaphores |

#### 4.1.1.6 Performance Characteristics

| Operation                                 | Execution Time | Notes                                            |
|-------------------------------------------|----------------|--------------------------------------------------|
| <a href="#">rx'comm'manager'init()</a>    | ~50 us         | Handle initialization, no hardware config        |
| <a href="#">rx'comm'manager'poll()</a>    | ~15-200 us     | Depends on channels enabled and frames available |
| - No frames available                     | ~15 us         | Quick check both channels, early return          |
| - USB frame received                      | ~120 us        | Frame validation + callback + ASCII (if enabled) |
| - SPI frame received                      | ~80 us         | Faster than USB (less overhead)                  |
| - Both frames received                    | ~200 us        | Sequential processing of both channels           |
| <a href="#">rx'comm'manager'send()</a>    | ~100-150 us    | Frame encoding + transport layer send            |
| <a href="#">rx'comm'manager'respond()</a> | ~100-150 us    | Same as send (convenience wrapper)               |
| Callback overhead                         | ~5-50 us       | User-defined, depends on handler complexity      |

All timing measured @ 240 MHz CPU clock, -O2 optimization, typical frame size 64 bytes.

#### 4.1.1.7 Memory Usage

| Allocation                        | Size            | Lifetime                                                     |
|-----------------------------------|-----------------|--------------------------------------------------------------|
| <a href="#">rx'comm'manager't</a> | ~12 KB          | Application-managed (typically static/global)                |
| - Pointers + state                | ~80 bytes       | Handle metadata, heartbeat, queue indices                    |
| - ASCII buffer                    | 2048 bytes      | Decoded output formatting (if enabled)                       |
| - Event queue                     | 8 x ~1028 bytes | Static FIFO ( <code>k_comm_event_queue_depth</code> entries) |
| - Heartbeat state                 | ~20 bytes       | Per-transport heartbeat tracking                             |
| Stack (poll)                      | ~120 bytes      | Function call duration only                                  |
| Stack (send)                      | ~80 bytes       | Function call duration only                                  |
| Static data (.rodata)             | ~400 bytes      | Channel names, format strings                                |
| Code (.text)                      | ~6 KB           | All functions                                                |

Total RAM: ~12 KB per manager instance (zero dynamic allocation). The event queue dominates memory: 8 entries x (`k_frame_max_payload` + metadata).

Memory Layout ([rx\\_comm\\_manager\\_t](#) structure):

| Offset | Size | Field                   | Notes                    |
|--------|------|-------------------------|--------------------------|
| 0x00   | 8    | <code>usb_handle</code> | Pointer to USB transport |
| 0x08   | 8    | <code>spi_handle</code> | Pointer to SPI transport |
| 0x10   | 8    | <code>callback</code>   | Function pointer         |

| Offset | Size         | Field                 | Notes                                     |
|--------|--------------|-----------------------|-------------------------------------------|
| 0x18   | 8            | callback_ctx          | User context pointer                      |
| 0x20   | 2048         | ascii_buffer          | ASCII formatting buffer                   |
| -      | 1            | enable_decoded_output | Boolean flag                              |
| -      | 1            | initialized           | Boolean flag                              |
| -      | ~20          | heartbeat↔<br>[2]     | Per-transport heartbeat state             |
| -      | ~16          | link_status_cb/ctx    | Callback + context                        |
| -      | ~8224        | event_queue↔<br>[8]   | 8 x <a href="#">rx_comm_event_entry_t</a> |
| -      | 3            | queue head/tail/count | Queue management                          |
| Total  | **~10.5 KB** |                       | (actual size may vary by compiler)        |

#### 4.1.1.8 Thread Safety Analysis

Conditional Thread Safety - Safe if following rules are observed:

1. Single poll context: Only call poll() from ONE thread/task/loop
2. Callback restrictions: User callback must not call manager functions (no recursion)
3. Send synchronization: If calling send()/respond() from multiple threads, use external mutex
4. Transport thread safety: Underlying USB/SPI transports must be thread-safe (typically are)

Not safe for:

- Concurrent poll() calls from multiple threads
- Calling manager functions from within frame callback
- Simultaneous send() calls without external synchronization

Recommendation: Use dedicated communication task:

```
void comm_task_entry(ULONG ctx) {
    rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
    while (1) {
        rx_comm_manager_poll(mgr);
        tx_thread_sleep(1); // 1ms polling period = 1kHz
    }
}
```

#### 4.1.1.9 Integration Example

## Complete Setup and Usage:

```

#include "rx_comm_manager.h"
#include "rx_usb_comm.h"
#include "rx_spi_comm.h"

// Frame handler callback
void on_frame_received(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;

    rx_log_info("COMM", "Frame from %s: type=0x%02X len=%u",
               rx_comm_manager_channel_name(channel),
               frame->type, frame->payload_len);

    // Parse frame based on type
    switch (frame->type) {
        case k_frame_type_command:
            // Handle command
            uint8_t response[128];
            uint32_t response_len = process_command(frame->payload,
                                                    frame->payload_len,
                                                    response, sizeof(response));

            // Respond on same channel
            rx_comm_manager_respond(mgr, channel, response, response_len);
            break;

        case k_frame_type_telemetry_req:
            // Send telemetry data
            send_telemetry(mgr, channel);
            break;

        default:
            rx_log_warn("COMM", "Unknown frame type: 0x%02X", frame->type);
            break;
    }
}

// Main application
int main(void) {
    // Initialize transports
    rx_usb_comm_handle_t usb_comm;
    rx_spi_comm_handle_t spi_comm;

    rx_usb_comm_init(&usb_comm, &usb_config);
    rx_spi_comm_init(&spi_comm, &spi_config);

    // Initialize communication manager
    rx_comm_manager_t comm_mgr;
    rx_comm_manager_config_t cfg = {
        .usb_handle = &usb_comm,
        .spi_handle = &spi_comm,
        .callback = on_frame_received,
        .callback_ctx = &comm_mgr, // Pass manager to callback
        .enable_decoded_output = true, // ASCII debug on Port 1
    };

    rx_err_t err = rx_comm_manager_init(&comm_mgr, &cfg);
    if (err != k_rx_ok) {
        rx_log_error("APP", "Comm manager init failed: 0x%X", err);
        return -1;
    }

    // Main communication loop
    while (1) {
        // Poll for incoming frames (non-blocking)
        err = rx_comm_manager_poll(&comm_mgr);

        // err == k_rx_ok: Frame(s) received and processed
        // err == k_rx_err_timeout: No frames available (normal)

        // Other application tasks...

        tx_thread_sleep(1); // 1ms = 1kHz polling rate
    }

    return 0;
}

```

## Sending Unsolicited Data (Telemetry):

```

void send_periodic_telemetry(rx_comm_manager_t* mgr) {
    // Gather telemetry data
    telemetry_data_t telemetry;

```

```

gather_telemetry(&telemetry);

// Serialize to buffer
uint8_t payload[256];
uint32_t payload_len = serialize_telemetry(&telemetry, payload, sizeof(payload));

// Send on USB channel (more reliable for telemetry)
rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_telemetry,
    .flags = 0,
    .payload = payload,
    .payload_len = payload_len,
};

rx_err_t err = rx_comm_manager_send(mgr, &params);
if (err != k_rx_ok) {
    rx_log_warn("TELEM", "Failed to send telemetry: 0x%X", err);
}
}

```

#### Module Dependencies:

- rx\_usb\_comm: USB CDC transport layer
- rx\_spi\_comm: SPI transport layer
- rx\_frame: Frame protocol definition and encoding/decoding
- rx\_err: Standardized error code system
- rx\_log: Logging infrastructure

#### See also

[rx\\_spi\\_comm.h](#) SPI transport implementation  
[rx\\_uart\\_comm.h](#) UART transport implementation  
[rx\\_i2c\\_comm.h](#) I2C peripheral transport implementation  
[rx\\_frame.h](#) Frame protocol specification  
[docs/sections/01\\_nanopb\\_protocol.tex](#) Communication protocol documentation

#### NASA Power of 10 Compliance:

- Rule 1: [OK] No goto, setjmp/longjmp, or recursion used
- Rule 2: [OK] All loops have compile-time bounded iterations (channel count = 2)
- Rule 3: [OK] Zero dynamic memory allocation (all buffers static)
- Rule 4: [OK] All functions < 60 lines
- Rule 5: [OK] Minimum 2 assertions per function via parameter validation
- Rule 6: [OK] Variables declared at smallest scope
- Rule 7: [OK] All return values checked and propagated
- Rule 8: [OK] Uses C23 typed enums exclusively (no magic numbers)
- Rule 9: [WARN] INTENTIONAL DEVIATION: Uses function pointers for callback (event-driven architecture)
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror (zero warnings)

## SOLID Principles:

- S (Single Responsibility): Only channel coordination and frame routing. Protocol parsing delegated to rx\_frame, transport to rx\_usb\_comm/rx\_spi\_comm.
- O (Open/Closed): Extensible to new channels by adding enum value and transport handle. Closed for modification (no changes needed for new frame types).
- L (Liskov Substitution): USB and SPI transports interchangeable via common frame interface. Can substitute mock transports for testing.
- I (Interface Segregation): Clean separation: lifecycle (init/deinit), polling (poll), sending (send/respond), status (channel\_ready). Users include only what they need.
- D (Dependency Inversion): Depends on abstractions (rx\_frame protocol, generic transport interface) not concrete USB/SPI implementations.

## Warning

Do NOT call manager functions from within frame callback (recursion risk)

ASCII buffer is 2048 bytes - ensure payload\_len + overhead < 2000 bytes to avoid truncation

Polling from multiple threads without synchronization will cause corruption

## Attention

Manager does not own USB/SPI handles - caller must initialize transports before manager

Callback is invoked from poll() context - keep processing minimal or defer to task queue

Decoded ASCII output adds ~80 us latency per frame - disable for high-frequency applications

## Author

Locked, Inc.

## Date

2026-01-27

## Version

1.0.0

## Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

## Changelog:

- 1.0.0 (2026-01-26): Initial implementation with USB and SPI channels
- 1.1.0 (2026-01-27): Added comprehensive Doxygen documentation, state machine diagrams, performance data

Definition in file [rx\\_comm\\_manager.h](#).

## 4.1.2 Data Structure Documentation

### 4.1.2.1 struct rx'comm'event'entry't

Single entry in the static event FIFO queue.

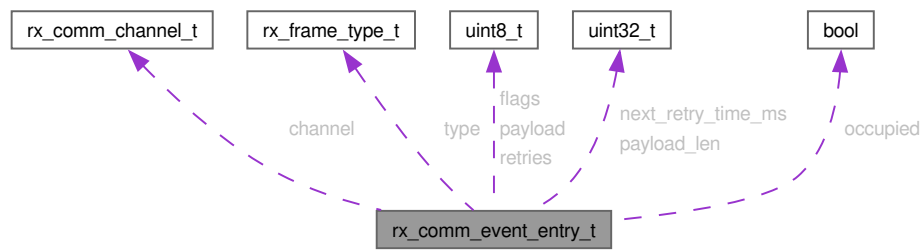
Events (commands) that require reliable delivery are queued here. Each entry contains a copy of the payload and metadata needed for retry logic. SPI entries retry with exponential backoff; USB entries are fire-and-forget.

Since

Version 1.0.0

Definition at line 713 of file [rx\\_comm\\_manager.h](#).

Collaboration diagram for rx\_comm\_event\_entry\_t:



#### Data Fields

|                                   |                                   |                                   |
|-----------------------------------|-----------------------------------|-----------------------------------|
| <a href="#">rx_comm_channel_t</a> | channel                           | Target transport                  |
| uint8_t                           | flags                             | Frame flags                       |
| uint32_t                          | next_retry_time_ms                | Earliest time (ms) for next retry |
| bool                              | occupied                          | Slot in use                       |
| uint8_t                           | payload↔<br>[k_frame_max_payload] | Payload copy                      |
| uint32_t                          | payload_len                       | Payload length (bytes)            |
| uint8_t                           | retries                           | Retry attempts so far             |
| <a href="#">rx_frame_type_t</a>   | type                              | Frame type                        |

### 4.1.2.2 struct rx'comm'heartbeat'state't

Per-transport heartbeat tracking state.

Tracks the last time a valid frame was received on each transport. Used by the poll loop to evaluate link health per the dual-detection heartbeat model.

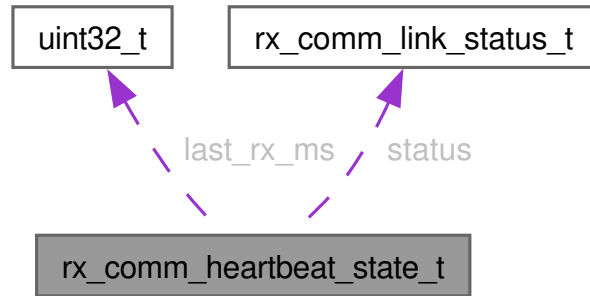


Since

Version 1.0.0

Definition at line 735 of file [rx\\_comm\\_manager.h](#).

Collaboration diagram for rx\_comm\_heartbeat\_state\_t:



Data Fields

|                                       |                         |                                    |
|---------------------------------------|-------------------------|------------------------------------|
| <code>uint32_t</code>                 | <code>last_rx_ms</code> | Timestamp (ms) of last valid frame |
| <a href="#">rx_comm_link_status_t</a> | <code>status</code>     | Current link health                |

#### 4.1.2.3 struct rx'comm'manager'config't

Communication manager configuration.

Configuration structure passed to [rx\\_comm\\_manager\\_init\(\)](#). Defines which channels are enabled, callback function, and optional features.

Configuration Guidelines:

- At least one channel (`usb_handle` OR `spi_handle`) should be non-NULL
- Both channels can be enabled simultaneously
- Callback is optional (NULL = receive but don't notify)
- Decoded output recommended for development, disable for production

Memory Lifetime:

- Config structure only read during init, can be stack-allocated
- Transport handles must remain valid for manager lifetime
- Callback function must remain valid (typically static function)

Configuration Example:

```
rx_comm_manager_config_t cfg = {
    .usb_handle = &usb_comm,      // Enable USB channel
    .spi_handle = &spi_comm,      // Enable SPI channel
    .callback = on_frame_received, // Frame handler
    .callback_ctx = &app_state,   // User context
    .enable_decoded_output = true, // ASCII debug output
};
```

See also

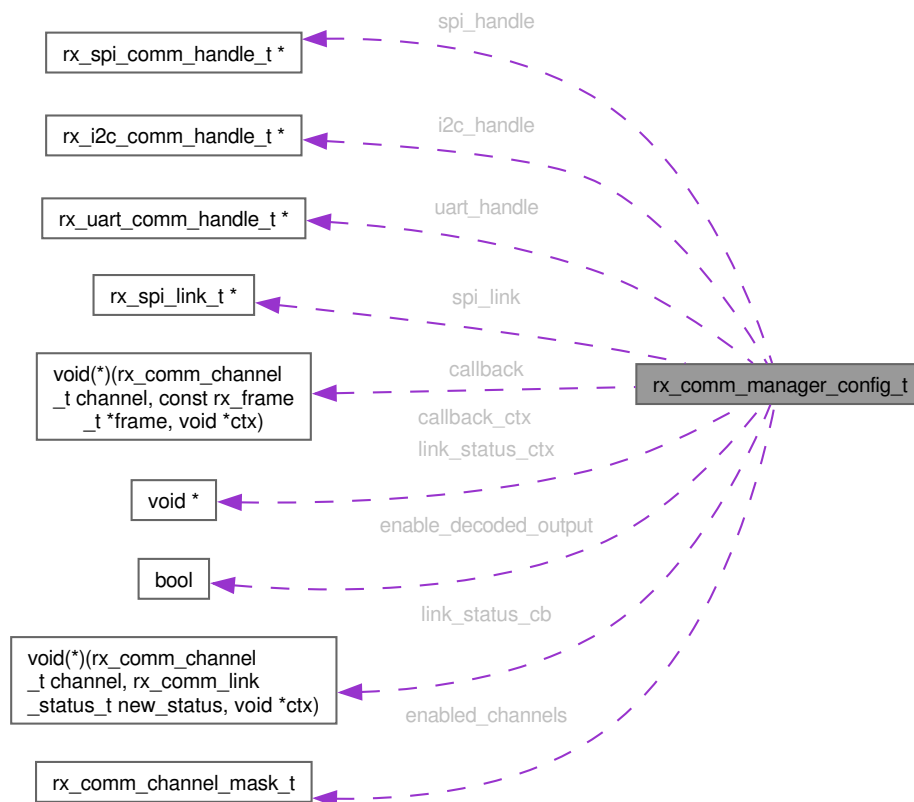
[rx\\_comm\\_manager\\_init\(\)](#) Initialization function

Since

Version 1.0.0

Definition at line 813 of file [rx\\_comm\\_manager.h](#).

Collaboration diagram for rx\_comm\_manager\_config\_t:



## Data Fields

|                                          |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">rx_comm_frame_callback_t</a> | callback | <p>Frame received callback function (NULL for no callback).<br/>User-defined function invoked when complete frame received on any channel. If NULL, frames are received but no notification occurs (not typical).<br/>Null handling:</p> <ul style="list-style-type: none"> <li>• NULL = Receive frames but don't notify (unusual, for testing only)</li> <li>• Non-NULL = Invoke callback on frame reception (normal operation)</li> </ul> <p>Type: <a href="#">rx_comm_frame_callback_t</a> (function pointer)</p> <p>Valid values: Non-NULL function pointer, or nullptr</p> <p>Signature: void callback(<a href="#">rx_comm_channel_t</a>, const <a href="#">rx_comm_frame_t</a>*)</p> <p>See also</p> <p><a href="#">rx_comm_frame_callback_t</a> Callback function type definition</p> |
|------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|        |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| void * | callback_ctx | <p>User context pointer passed to callback.</p> <p>Arbitrary pointer passed to callback as third parameter. Typically used to pass application state, manager pointer, or other context without requiring global variables. Common patterns:</p> <ul style="list-style-type: none"><li>• Pointer to manager itself (enables response from callback)</li><li>• Pointer to application state structure</li><li>• NULL if no context needed</li></ul> <p>Type: void* (application-defined)</p> <p>Valid values: Any pointer value, or nullptr</p> <p>Lifetime: Must remain valid if callback uses it</p> |
|--------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|      |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bool | enable_decoded_output | <p>Enable decoded ASCII output to USB Port 1.</p> <p>When true, all received and sent frames are converted to human-readable ASCII format and sent to USB Port 1 (/dev/ttyACM1 on RPi5).</p> <p>ASCII Format:</p> <pre>[USB] TYPE=cmd FLAGS=0x00 LEN=42       DATA=48656C6C66...</pre> <pre>[SPI] TYPE=resp FLAGS=0x01 LEN=128       DATA=...</pre> <p>Performance Impact:</p> <ul style="list-style-type: none"> <li>• Adds ~80 us latency per frame (formatting + USB send)</li> <li>• Negligible for command/response (&lt; 100 Hz)</li> <li>• May impact high-frequency telemetry (&gt; 1 kHz)</li> </ul> <p>Use Cases:</p> <ul style="list-style-type: none"> <li>• Development: Enable for debugging and protocol analysis</li> <li>• Production: Disable for maximum performance</li> </ul> <p>Type: bool</p> <p>Value: true = enable ASCII output, false = disable</p> <p>Default: Recommend true for development, false for</p> |
|------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                |                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">rx_comm_channel_mask_t</a>         | enabled_channels | <p>Bitmask of channels to initialize and poll (default: all channels). When a bit is clear the corresponding transport is skipped during <code>internal_init_transports()</code> and its handle is left NULL. Set to <code>k_↵_comm_channel_mask_all</code> to attempt all four transports (legacy behaviour).</p> <p>Use <code>comm_task_apply_system_↵_config()</code> to set this alongside the log backend so that SCI9 conflicts between UART comm and UART log are caught early.</p> <p>Valid values: Any combination of <code>rx_↵_comm_channel</code></p> <p>Default: <code>k_↵_comm_channel_mask_all</code> (attempt all c</p> <p>See also</p> <p><a href="#">rx_comm_channel_mask_t</a><br/>Bit definitions</p> <p>Since</p> <p>Version 1.0.0</p> |
| <code>rx_i2c_comm_handle_t *</code>            | i2c_handle       | <p>I2C communication handle (NULL to disable I2C channel). Pointer to initialized I2C peripheral transport handle. If NULL, I2C channel is disabled and all I2C operations are skipped. The RX72N acts as I2C peripheral (device) and the RPi5 acts as I2C controller.</p> <p>Type: <code>rx_i2c_comm_handle_t*</code> (pointer to I2C tra</p> <p>Valid values: Non-NULL initialized handle, or nullpt</p> <p>Lifetime: Must outlive manager instance</p>                                                                                                                                                                                                                                                                                                   |
| <a href="#">rx_comm_link_status_callback_t</a> | link_status_cb   | <p>Callback invoked when a transport link status changes.</p> <p>Optional. Called when heartbeat monitoring detects a link transition (e.g., healthy -&gt; dead). NULL to disable link status notifications.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

| void *                 | link_status_ctx | User context for link_status_cb.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rx_spi_comm_handle_t * | spi_handle      | <p>SPI communication handle (NULL to disable SPI channel).<br/>           Pointer to initialized SPI transport handle. If NULL, SPI channel is disabled and all SPI operations are skipped.</p> <p>Requirements:</p> <ul style="list-style-type: none"> <li>• Must be initialized via rx↔_spi_comm_init() before passing to manager</li> <li>• Must remain valid for lifetime of manager</li> <li>• Requires hardware SPI connection between RX72N and RPi5</li> </ul> <p>Null handling:</p> <ul style="list-style-type: none"> <li>• NULL = SPI channel disabled (valid configuration)</li> <li>• Non-NULL = SPI channel enabled</li> </ul> <p>Type: rx_spi_comm_handle_t* (pointer to SPI tra</p> <p>Valid values: Non-NULL initialized handle, or nullpt</p> <p>Lifetime: Must outlive manager instance</p> |

|                              |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rx_spi_link_t *</code> | <code>spi_link</code> | <p>Optional SPI link layer with HARQ (NULL to use raw SPI).<br/>When non-NULL, SPI send/receive operations route through the link layer, which provides FEC encoding/decoding, Chase Combining, and ACK/NACK handshake. When NULL, SPI operates in raw mode (frame + CRC-32 only, no FEC/HARQ).</p> <p>Requirements:</p> <ul style="list-style-type: none"> <li>• Must be initialized via <code>rx_spi_link_init()</code> before passing to manager</li> <li>• <code>spi_link-&gt;spi_handle</code> MUST be the same as <code>config-&gt;spi_handle</code></li> <li>• Must remain valid for lifetime of manager</li> </ul> <p>Invariant</p> <p>If <code>spi_link</code> is non-NULL:<br/> <code>spi_link-&gt;spi_handle == config-&gt;spi_handle</code></p> <p>If <code>spi_link</code> is non-NULL: <code>spi_link-&gt;initialized == true</code></p> <p>If <code>spi_link</code> is non-NULL: <code>spi_link</code> must outlive manager instance</p> <p>Valid values: Initialized <code>rx_spi_link_t*</code>, or nullptr</p> <p>Lifetime: Must outlive manager instance</p> <p>See also</p> <p><code>rx_spi_link.h</code> SPI link layer API</p> <p><code>rx_spi_link_init()</code> Must be called before passing <a href="#">spi_link</a> to manager</p> <p>Since</p> <p>Version 1.0.0</p> |
|------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|                                      |                          |                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rx_uart_comm_handle_t *</code> | <code>uart_handle</code> | <p>UART communication handle (NULL to disable UART channel). Pointer to initialized UART (SCI9) transport handle. If NULL, UART channel is disabled and all UART operations are skipped.</p> <p>Type: <code>rx_uart_comm_handle_t*</code> (pointer to UART)</p> <p>Valid values: Non-NULL initialized handle, or nullpt</p> <p>Lifetime: Must outlive manager instance</p> |
|--------------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 4.1.2.4 struct rx'comm'manager't

Communication manager handle.

Opaque handle structure containing manager state and resources. Application must allocate this structure (typically static or global) and pass to all manager functions.

Initialization: Zero-initialize structure before passing to [rx\\_comm\\_manager\\_init\(\)](#):

```
rx_comm_manager_t mgr = {}; // Or use memset
```

Memory Layout: See file-level documentation for detailed breakdown.

Thread Safety: Not thread-safe - protect with external mutex if accessed from multiple threads.

#### Warning

Do not access fields directly - use API functions only  
 Structure size ~2.5 KB - consider allocation carefully

See also

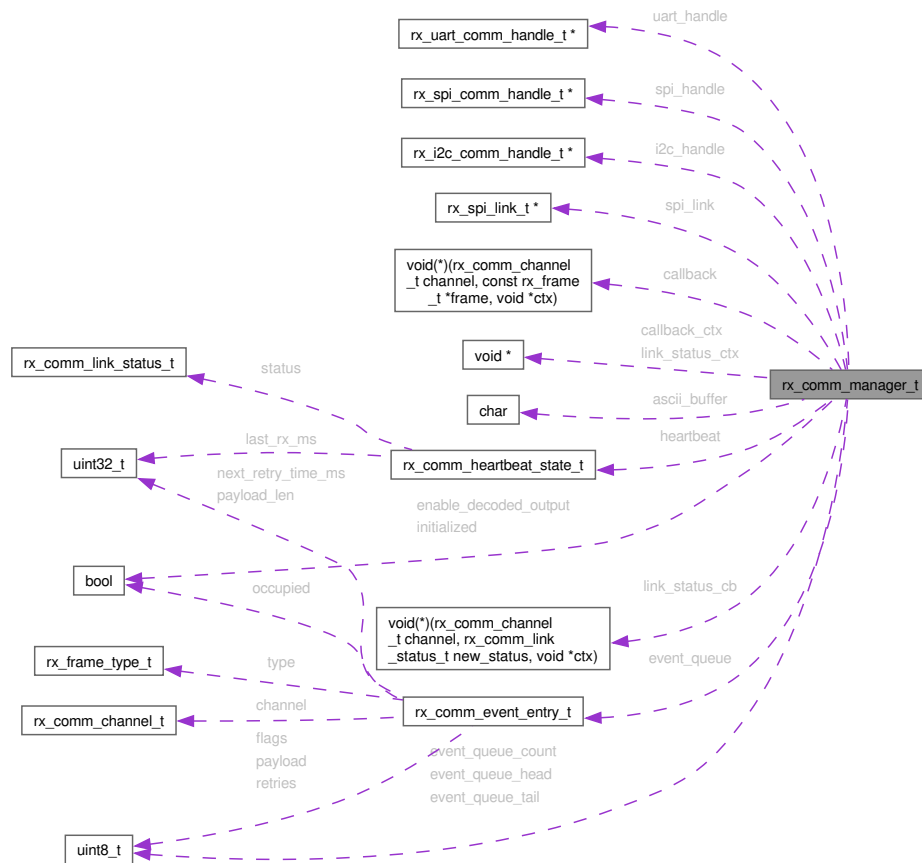
[rx\\_comm\\_manager\\_init\(\)](#) Initialize handle  
[rx\\_comm\\_manager\\_deinit\(\)](#) Cleanup handle

Since

Version 1.0.0

Definition at line [1007](#) of file [rx\\_comm\\_manager.h](#).

Collaboration diagram for `rx_comm_manager_t`:



## Data Fields

|                                          |                                                  |                                                        |
|------------------------------------------|--------------------------------------------------|--------------------------------------------------------|
| char                                     | ascii_buffer↔<br>[k_comm_manager_ascii_buf_size] | ASCII buffer                                           |
| <a href="#">rx_comm_frame_callback_t</a> | callback                                         | Frame callback                                         |
| void *                                   | callback_ctx                                     | Callback context                                       |
| bool                                     | enable_decoded_output                            | Enable ASCII output                                    |
| <a href="#">rx_comm_event_entry_t</a>    | event_queue[k_comm_event_queue_depth]            | Static event FIFO queue for reliable command delivery. |
| uint8_t                                  | event_queue_count                                | Number of occupied event queue slots.                  |

|                                           |                                                 |                                                                                      |
|-------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------|
| uint8_t                                   | event_queue_head                                | Event queue write index (next slot to write).                                        |
| uint8_t                                   | event_queue_tail                                | Event queue read index (next slot to process).                                       |
| <a href="#">rx_comm_heartbeat_state_t</a> | <a href="#">heartbeat[k_comm_channel_count]</a> | Per-transport heartbeat tracking (indexed by <a href="#">rx_↵comm_↵channel_↵t</a> ). |

|                        |            |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rx_i2c_comm_handle_t * | i2c_handle | <p>Optional SPI link layer with HARQ (NULL if disabled). I2C peripheral comm handle (NULL disables I2C channel) When non-NULL, points to an rx_spi_link_t handle providing FEC encoding/decoding, Chase Combining, and ACK/NACK handshake for SPI. When NULL, SPI operates in raw mode (frame + CRC-32 only, no FEC/HARQ). Ownership and Lifetime:</p> <ul style="list-style-type: none"> <li>• Pointer copied from config during rx_comm_manager</li> </ul> |
|                        |            | <p>Generated by Doxygen<br/>init()</p> <ul style="list-style-type: none"> <li>• Manager</li> </ul>                                                                                                                                                                                                                                                                                                                                                           |

|                                                | bool   | initialized     | Initialization flag                                      |
|------------------------------------------------|--------|-----------------|----------------------------------------------------------|
| <a href="#">rx_comm_link_status_callback_t</a> |        | link_status_cb  | Link status change callback (optional).                  |
|                                                | void * | link_status_ctx | Link status callback context.                            |
| <a href="#">rx_spi_comm_handle_t</a> *         |        | spi_handle      | SPI comm handle (NULL disables SPI channel)              |
| <a href="#">rx_spi_link_t</a> *                |        | spi_link        |                                                          |
| <a href="#">rx_uart_comm_handle_t</a> *        |        | uart_handle     | UART (SCI9 via CY7↔ C65213 bridge) comm handle – primary |

#### 4.1.2.5 struct rx'comm'send'params't

Parameters for `rx_comm_manager_send`.

Wrapper structure providing type safety for send operations. Prevents parameter confusion when passing multiple `uint8_t`/`uint32_t` values.

Design Rationale: Without this struct, function signature would be error-prone:

```
// BAD: Easy to mix up channel, type, flags (all similar types)
rx_err_t send(mgr, uint8_t ch, uint8_t type, uint8_t flags, ...);
```

With struct, parameters are self-documenting:

```
// GOOD: Clear naming prevents mistakes
rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_command,
    .flags = 0x00,
    // ...
};
```

Usage Example:

```
rx_comm_send_params_t params = {
    .channel = k_comm_channel_spi,
    .type = k_frame_type_telemetry,
    .flags = 0,
    .payload = telemetry_data,
    .payload_len = sizeof(telemetry_data),
};
rx_comm_manager_send(&mgr, &params);
```

See also

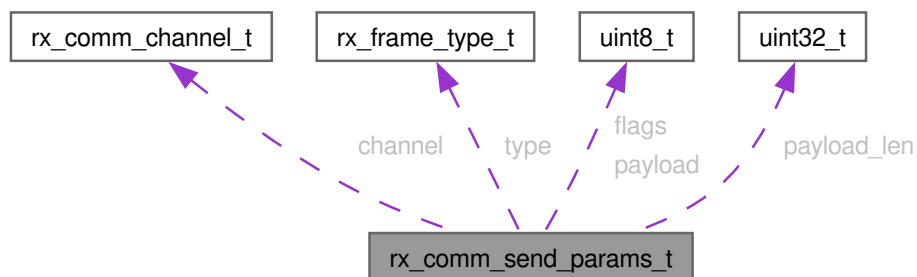
[rx\\_comm\\_manager\\_send\(\)](#) Send function using this structure

Since

Version 1.0.0

Definition at line 1132 of file [rx\\_comm\\_manager.h](#).

Collaboration diagram for rx\_comm\_send\_params\_t:



Data Fields

|                                   |             |                                       |
|-----------------------------------|-------------|---------------------------------------|
| <a href="#">rx_comm_channel_t</a> | channel     | Channel to send on (USB or SPI)       |
| uint8_t                           | flags       | Frame flags (protocol-specific)       |
| const uint8_t *                   | payload     | Payload data pointer                  |
| uint32_t                          | payload_len | Payload length in bytes               |
| rx_frame_type_t                   | type        | Frame type (command, telemetry, etc.) |

### 4.1.3 Typedef Documentation

#### 4.1.3.1 rx\_comm\_frame\_callback\_t

```
typedef void(* rx_comm_frame_callback_t) (rx\_comm\_channel\_t channel, const rx_frame_t *frame, void *ctx)
```

Frame received callback function type.

User-defined function invoked by communication manager when a complete, valid frame is received on any enabled channel. The callback receives:

- Channel identifier (which channel the frame arrived on)
- Parsed frame structure (type, flags, payload)
- User context pointer (passed during initialization)

## 4.1.3.2 Callback Contract

## Responsibilities:

- Parse frame payload based on frame type
- Handle command execution or data processing
- Optionally send response via [rx\\_comm\\_manager\\_respond\(\)](#)
- Return quickly to avoid blocking communication loop

## Restrictions:

- Do NOT call [rx\\_comm\\_manager\\_poll\(\)](#) from callback (recursion!)
- Keep processing minimal (~50 us recommended)
- For slow operations, queue to task and return immediately
- Do NOT modify frame structure (it may be const or recycled)

## Thread Safety:

- Callback executed in context of poll() caller
- If poll() called from RTOS task, callback runs in that task context
- Synchronization required if accessing shared resources

## Parameters

|    |         |                                                                                                                                                                                                                                                                                                                         |
|----|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | channel | <p>The communication channel the frame was received on</p> <ul style="list-style-type: none"> <li>• Type: <a href="#">rx_comm_channel_t</a></li> <li>• Valid values: k_comm_channel_usb or k_comm_channel_spi</li> <li>• Use to determine response routing</li> </ul>                                                   |
| in | frame   | <p>Pointer to the received and validated frame</p> <ul style="list-style-type: none"> <li>• Type: const rx_frame_t*</li> <li>• Valid until callback returns</li> <li>• Contains: type, flags, payload, payload_len (CRC already validated)</li> <li>• Do NOT modify or store pointer (frame may be recycled)</li> </ul> |

|    |     |                                                                                                                                                                                                                                                                                                                        |
|----|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | ctx | User context pointer <ul style="list-style-type: none"> <li>• Type: void* (application-defined)</li> <li>• Value passed during <a href="#">rx_comm_manager_init()</a></li> <li>• Typically pointer to manager, application state, or nullptr</li> <li>• Use to access application resources without globals</li> </ul> |
|----|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Usage Example - Basic Command Handler:

```
void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;

    // Log reception
    rx_log_info("COMM", "RX from %s: type=0x%02X len=%u",
        rx_comm_manager_channel_name(channel),
        frame->type, frame->payload_len);

    // Dispatch based on type
    switch (frame->type) {
        case k_frame_type_command:
            handle_command(mgr, channel, frame);
            break;
        case k_frame_type_telemetry_req:
            send_telemetry(mgr, channel);
            break;
        default:
            rx_log_warn("COMM", "Unknown type: 0x%02X", frame->type);
            break;
    }
}
```

#### Usage Example - Deferred Processing:

```
// For time-consuming operations, defer to queue
void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    app_state_t* app = (app_state_t*)ctx;

    // Copy frame to queue (frame pointer only valid during callback)
    command_msg_t msg = {
        .channel = channel,
        .type = frame->type,
        .payload_len = frame->payload_len,
    };
    memcpy(msg.payload, frame->payload, frame->payload_len);

    // Queue for processing by command handler task
    UINT status = tx_queue_send(&app->cmd_queue, &msg, TX_NO_WAIT);
    if (status != TX_SUCCESS) {
        rx_log_error("COMM", "Command queue full!");
    }
}
```

#### Warning

Callback must return quickly (<50 us recommended) to avoid blocking communication

Do NOT call [rx\\_comm\\_manager\\_poll\(\)](#) from callback (causes recursion)

Frame pointer only valid during callback - copy data if needed beyond callback scope

See also

[rx\\_comm\\_manager\\_respond\(\)](#) Send response on same channel

[rx\\_comm\\_manager\\_send\(\)](#) Send frame on specific channel

[rx\\_frame\\_t](#) Frame structure definition



Since

Version 1.0.0

Definition at line 681 of file [rx\\_comm\\_manager.h](#).

#### 4.1.3.3 rx'comm'link'status'callback't

```
typedef void(* rx_comm_link_status_callback_t) (rx_comm_channel_t channel, rx_comm_link_status_t new_status, void *ctx)
```

Callback invoked when a link status changes.

Parameters

|    |            |                                |
|----|------------|--------------------------------|
| in | channel    | Which transport changed status |
| in | new_status | New link status                |
| in | ctx        | User context pointer           |

Since

Version 1.0.0

Definition at line 750 of file [rx\\_comm\\_manager.h](#).

### 4.1.4 Enumeration Type Documentation

#### 4.1.4.1 rx'comm'channel'mask't

```
enum rx_comm_channel_mask_t : uint8_t
```

Bitmask selecting which comm channels to initialize and use.

Each bit corresponds to one comm channel. Bit position matches the numeric value of the corresponding [rx\\_comm\\_channel\\_t](#) enumerator so that a single shift converts between the two representations.

Pass a bitmask in [rx\\_comm\\_manager\\_config\\_t::enabled\\_channels](#) to skip initialization of channels that are not needed (e.g. exclude UART when the log backend also uses SCI9).

See also

[rx\\_comm\\_channel\\_t](#) Channel enumerator

[rx\\_comm\\_manager\\_config\\_t::enabled\\_channels](#) Usage

Since

Version 1.0.0

Enumerator

|                          |                                |
|--------------------------|--------------------------------|
| k_comm_channel_mask_none | No channels enabled            |
| k_comm_channel_mask_uart | UART (k_comm_channel_uart = 0) |
| k_comm_channel_mask_spi  | SPI (k_comm_channel_spi = 1)   |
| k_comm_channel_mask_i2c  | I2C (k_comm_channel_i2c = 2)   |
| k_comm_channel_mask_all  | All three channels             |

Definition at line 771 of file [rx\\_comm\\_manager.h](#).

```
00771     : uint8_t {
00772   k_comm_channel_mask_none = 0x00U, /**< No channels enabled */
00773   k_comm_channel_mask_uart = 0x01U, /**< UART (k_comm_channel_uart = 0) */
00774   k_comm_channel_mask_spi  = 0x02U, /**< SPI  (k_comm_channel_spi  = 1) */
00775   k_comm_channel_mask_i2c  = 0x04U, /**< I2C  (k_comm_channel_i2c  = 2) */
00776   k_comm_channel_mask_all  =      /**< All three channels */
00777   (k_comm_channel_mask_uart | k_comm_channel_mask_spi | k_comm_channel_mask_i2c),
00778 } rx_comm_channel_mask_t;
```

#### 4.1.4.2 rx'comm'channel't

```
enum rx\_comm\_channel\_t : uint8_t
```

Communication channel identifiers.

Enumerates the available communication channels. The manager can handle multiple channels simultaneously, with each channel operating independently.

Channel Characteristics:

| Channel | Bandwidth | Latency   | Setup Complexity         | Use Case                 |
|---------|-----------|-----------|--------------------------|--------------------------|
| USB CDC | ~1 MB/s   | ~1-5 ms   | Low (plug-and-play)      | Commands, telemetry      |
| SPI     | ~2 MB/s   | ~0.1-1 ms | Medium (requires wiring) | High-speed data, control |

Usage Pattern:

```
// Check which channel frame came from
void callback(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    if (channel == k_comm_channel_usb) {
        // Received via USB
    } else if (channel == k_comm_channel_spi) {
        // Received via SPI
    }
}
```

See also

[rx\\_comm\\_manager\\_channel\\_name\(\)](#) Get human-readable channel name

[rx\\_comm\\_frame\\_callback\\_t](#) Callback receives channel identifier

Since

Version 1.0.0

#### Enumerator

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>k_comm_channel_uart</code>  | <p>UART channel (SCI9 via CY7C65213 USB-UART bridge). Primary link to the Raspberry Pi 5 gateway. Carries protobuf command / response / telemetry frames AND <code>k_frame_type_log_message</code> frames with the RX72N runtime log stream, all multiplexed on the same byte stream. On the Pi5 the bridge enumerates as <code>/dev/ttyACM0</code> at 115200 baud (CY7C65213 is a CDC-ACM class device; see <code>k_uart_default_baudrate</code> in <code>libs/rx_hal/src/uart.c</code>).</p> <p>Hardware: SCI9 (PB7 / TXD9, PB6 / RXD9) + CY7C65213 bridge IC</p> <p>Value: 0</p> |
| <code>k_comm_channel_spi</code>   | <p>SPI peripheral channel. Dedicated SPI connection to the ROS2 <code>spi↔_driver_node</code> for low-latency, high-bandwidth motor control. 10 MHz clock, full-duplex.</p> <p>Hardware: RSPI peripheral (COPI / CIPO / SCK / CS)</p> <p>Value: 1</p>                                                                                                                                                                                                                                                                                                                               |
| <code>k_comm_channel_i2c</code>   | <p>I2C peripheral channel (RIIC0). I2C connection where RX72N acts as I2C peripheral device and RPi5 is the I2C controller. Optional auxiliary channel.</p> <p>Hardware: RIIC0 peripheral (P16 / SCL0, P17 / SDA0)</p> <p>Value: 2</p>                                                                                                                                                                                                                                                                                                                                              |
| <code>k_comm_channel_count</code> | <p>Total number of supported channels (3).</p> <p>Value: 3 (UART + SPI + I2C)</p> <p>Invariant: Must equal number of enum values (excluding this one)</p>                                                                                                                                                                                                                                                                                                                                                                                                                           |

Definition at line 531 of file [rx\\_comm\\_manager.h](#).

```
00531         : uint8_t {
00532     /**
00533     * @brief UART channel (SCI9 via CY7C65213 USB-UART bridge)
```

```

00534 * @details
00535 * Primary link to the Raspberry Pi 5 gateway. Carries protobuf command /
00536 * response / telemetry frames AND k_frame_type_log_message frames with the
00537 * RX72N runtime log stream, all multiplexed on the same byte stream.
00538 *
00539 * On the Pi5 the bridge enumerates as /dev/ttyACM0 at 115200 baud (CY7C65213 is a CDC-ACM class device; see
k_uart_default_baudrate in libs/rx_hal/src/uart.c).
00540 *
00541 * @par Hardware: SCI9 (PB7 / TXD9, PB6 / RXD9) + CY7C65213 bridge IC
00542 * @par Value: 0
00543 */
00544 k_comm_channel_uart = 0,
00545
00546 /**
00547 * @brief SPI peripheral channel
00548 * @details
00549 * Dedicated SPI connection to the ROS2 spi_driver_node for low-latency,
00550 * high-bandwidth motor control. 10 MHz clock, full-duplex.
00551 *
00552 * @par Hardware: RSPI peripheral (COPI / CIPO / SCK / CS)
00553 * @par Value: 1
00554 */
00555 k_comm_channel_spi = 1,
00556
00557 /**
00558 * @brief I2C peripheral channel (RIIC0)
00559 * @details
00560 * I2C connection where RX72N acts as I2C peripheral device and RPi5 is the
00561 * I2C controller. Optional auxiliary channel.
00562 *
00563 * @par Hardware: RIIC0 peripheral (P16 / SCL0, P17 / SDA0)
00564 * @par Value: 2
00565 */
00566 k_comm_channel_i2c = 2,
00567
00568 /**
00569 * @brief Total number of supported channels (3)
00570 *
00571 * @par Value: 3 (UART + SPI + I2C)
00572 * @par Invariant: Must equal number of enum values (excluding this one)
00573 */
00574 k_comm_channel_count = 3,
00575 } rx_comm_channel_t;

```

#### 4.1.4.3 rx\_comm\_event\_retry\_constants\_t

```
enum rx_comm_event_retry_constants_t : uint16_t
```

Event queue retry constants for SPI HARQ.

Events (commands) sent via SPI use retry with exponential backoff. USB sends are fire-and-forget (no retries).

Since

Version 1.0.0

Enumerator

|                            |                                                                           |
|----------------------------|---------------------------------------------------------------------------|
| k_event_max_retries        | Maximum SPI retry attempts                                                |
| k_event_initial_backoff_ms | Initial backoff delay (ms)                                                |
| k_event_max_backoff_ms     | Maximum backoff delay (ms): caps $10 \ll (\text{retries}-1)$ at retries=2 |

Definition at line 489 of file [rx\\_comm\\_manager.h](#).

```

00489 : uint16_t {
00490 k_event_max_retries = 3, /**< Maximum SPI retry attempts */
00491 k_event_initial_backoff_ms = 10, /**< Initial backoff delay (ms) */
00492 k_event_max_backoff_ms = 15, /**< Maximum backoff delay (ms): caps  $10 \ll (\text{retries}-1)$  at retries=2 */
00493 } rx_comm_event_retry_constants_t;

```

## 4.1.4.4 rx'comm'heartbeat'constants't

```
enum rx_comm_heartbeat_constants_t : uint32_t
```

Heartbeat timing constants.

Dual-detection heartbeat model:

- Implicit timeout (200ms): Any valid frame resets the timer. If no frames arrive within 200ms, the link is declared dead.
- Explicit PING (1000ms): Sent when link is idle for >1s as a probe. Firmware primarily receives PINGs from the gateway.
- Check interval (50ms): How often to evaluate heartbeat status.

Since

Version 1.0.0

Enumerator

|                                 |                                           |
|---------------------------------|-------------------------------------------|
| k_heartbeat_implicit_timeout_ms | Declare link dead if no frame for 200ms   |
| k_heartbeat_ping_interval_ms    | Send PING probe when idle for 1s          |
| k_heartbeat_check_interval_ms   | Heartbeat evaluation interval (200ms / 4) |

Definition at line 473 of file [rx\\_comm\\_manager.h](#).

```
00473     : uint32_t {
00474 k_heartbeat_implicit_timeout_ms = 200, /**< Declare link dead if no frame for 200ms */
00475 k_heartbeat_ping_interval_ms   = 1000, /**< Send PING probe when idle for 1s */
00476 k_heartbeat_check_interval_ms  = 50,  /**< Heartbeat evaluation interval (200ms / 4) */
00477 } rx_comm_heartbeat_constants_t;
```

## 4.1.4.5 rx'comm'link'status't

```
enum rx_comm_link_status_t : uint8_t
```

Link health status from heartbeat monitoring.

Reported per-transport link health. Determined by the dual-detection heartbeat model (implicit 200ms timeout + explicit 1s PING).

Since

Version 1.0.0

Enumerator

|                       |                                           |
|-----------------------|-------------------------------------------|
| k_link_status_unknown | No frames received yet                    |
| k_link_status_healthy | Frames received within implicit timeout   |
| k_link_status_dead    | No frames within implicit timeout (200ms) |

Definition at line 695 of file [rx\\_comm\\_manager.h](#).

```
00695     : uint8_t {
00696 k_link_status_unknown = 0, /**< No frames received yet */
00697 k_link_status_healthy = 1, /**< Frames received within implicit timeout */
00698 k_link_status_dead   = 2, /**< No frames within implicit timeout (200ms) */
00699 } rx_comm_link_status_t;
```

#### 4.1.4.6 rx\_comm\_manager\_constants\_t

```
enum rx_comm_manager_constants_t : uint16_t
```

Communication manager compile-time constants.

Defines buffer sizes and limits for the communication manager. These constants are chosen based on typical frame sizes and ASCII formatting requirements.

Design Rationale:

- 2048-byte ASCII buffer: Accommodates maximum frame size (1024 bytes payload) plus formatting overhead (~1000 bytes for hex dump + headers)
- Buffer is stack-allocated in handle structure (not dynamically allocated)
- Sufficient for debugging without excessive memory usage

See also

[rx\\_comm\\_manager\\_t.ascii\\_buffer](#)

Since

Version 1.0.0

Enumerator

|                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| k_comm_manager_ascii_buf_size | <p>ASCII formatting buffer size in bytes. Size of internal buffer used to format decoded ASCII output for debug port. Must accommodate:</p> <ul style="list-style-type: none"> <li>• Header line: ~80 bytes ([USB] TYPE=cmd FLAGS=0x00 LEN=1024)</li> <li>• Payload hex dump: payload_len * 3 bytes (XX per byte)</li> <li>• Footer + null terminator: ~20 bytes</li> </ul> <p>Maximum frame payload: 1024 bytes Required buffer: 80 + (1024 * 3) + 20 = 3172 bytes Allocated: 2048 bytes (limits payload display to ~650 bytes, which is acceptable)</p> <p>Value: 2048 bytes</p> <p>Rationale: Balance between functionality and memory usage</p> |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                          |                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| k_comm_event_queue_depth | <p>Maximum number of entries in the event FIFO queue. Static FIFO queue depth for event (command) frames that require reliable delivery. Events are queued and sent with SPI-only retry + exponential backoff. Queue is statically allocated (zero dynamic memory).</p> <p>Value: 8 entries</p> <p>Rationale: Small queue prevents unbounded memory, sufficient for typical command burst scenarios</p> |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Definition at line 426 of file [rx\\_comm\\_manager.h](#).

```

00426         : uint16_t {
00427     /**
00428      * @brief ASCII formatting buffer size in bytes
00429      * @details
00430      * Size of internal buffer used to format decoded ASCII output for debug port.
00431      * Must accommodate:
00432      * - Header line: ~80 bytes ('[USB] TYPE=cmd FLAGS=0x00 LEN=1024')
00433      * - Payload hex dump: payload_len * 3 bytes ('XX ` per byte)
00434      * - Footer + null terminator: ~20 bytes
00435      *
00436      * Maximum frame payload: 1024 bytes
00437      * Required buffer: 80 + (1024 * 3) + 20 = 3172 bytes
00438      * Allocated: 2048 bytes (limits payload display to ~650 bytes, which is acceptable)
00439      *
00440      * @par Value: 2048 bytes
00441      * @par Rationale: Balance between functionality and memory usage
00442      */
00443     k_comm_manager_ascii_buf_size = 2048,
00444
00445     /**
00446      * @brief Maximum number of entries in the event FIFO queue
00447      * @details
00448      * Static FIFO queue depth for event (command) frames that require reliable
00449      * delivery. Events are queued and sent with SPI-only retry + exponential
00450      * backoff. Queue is statically allocated (zero dynamic memory).
00451      *
00452      * @par Value: 8 entries
00453      * @par Rationale: Small queue prevents unbounded memory, sufficient for
00454      * typical command burst scenarios
00455      */
00456     k_comm_event_queue_depth = 8,
00457 } rx_comm_manager_constants_t;

```

## 4.1.5 Function Documentation

### 4.1.5.1 rx'comm'manager'channel'name()

```

const char * rx_comm_manager_channel_name (
    rx\_comm\_channel\_t channel)

```

Get channel name string.

Parameters

|    |         |                    |
|----|---------|--------------------|
| in | channel | Channel identifier |
|----|---------|--------------------|

### Returns

Human-readable channel name

Get channel name string.

Returns static string name for given channel enum value. Used for logging, debugging, and user interface display. Always returns a valid string (never NULL).

### Returned Strings:

- k\_comm\_channel\_usb -> "USB"
- k\_comm\_channel\_spi -> "SPI"
- Invalid/unknown -> "UNKNOWN"

### Use Cases:

- Logging channel activity
- Debugging frame traffic
- User interface display
- Error messages

### Precondition

None (accepts any input)

### Postcondition

Returns pointer to static string (valid for program lifetime)

### Note

Returned string is static - do not free

Thread-safe (returns read-only static strings)

Never returns NULL - always safe to use in printf

### Example - Logging:

```
static void on_frame_received(rx_comm_channel_t channel,
                             const rx_frame_t* frame,
                             void* ctx)
{
    printf("Frame received on %s: type=%d len=%d\n",
          rx_comm_manager_channel_name(channel),
          frame->header.type,
          frame->header.length);
}
```



Example - Error Messages:

```
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err != k_rx_ok) {
    fprintf(stderr, "Failed to send on %s: error %d\n",
        rx_comm_manager_channel_name(params.channel), err);
}
```

See also

[rx\\_comm\\_channel\\_t](#) Channel enumeration

Since

Version 1.0.0

Definition at line 1946 of file [rx\\_comm\\_manager.c](#).

```
01947 {
01948     switch (channel) {
01949         case k_comm_channel_uart:
01950             return "UART";
01951         case k_comm_channel_spi:
01952             return "SPI";
01953         case k_comm_channel_i2c:
01954             return "I2C";
01955         default:
01956             return "UNKNOWN";
01957     }
01958 }
```

References [k\\_comm\\_channel\\_i2c](#), [k\\_comm\\_channel\\_spi](#), and [k\\_comm\\_channel\\_uart](#).

#### 4.1.5.2 rx'comm'manager'channel'ready()

```
rx_err_t rx_comm_manager_channel_ready (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    bool * ready) [nodiscard]
```

Check if channel is ready for communication.

Parameters

|     |         |                                 |
|-----|---------|---------------------------------|
| in  | mgr     | Pointer to initialized manager  |
| in  | channel | Channel to check                |
| out | ready   | Set to true if channel is ready |

Returns

k\_rx\_ok on success  
k\_rx\_err\_invalid\_arg if mgr or ready is nullptr

Check if channel is ready for communication.

Checks if specified channel is configured and ready for communication. USB channel readiness depends on USB enumeration and configuration by host. SPI channel is ready immediately if handle is configured (no enumeration required).

Readiness Criteria:

- USB: `usb_handle` non-NULL AND USB device enumerated and configured by host
- SPI: `spi_handle` non-NULL (always ready if configured)

Use Cases:

- Check readiness before sending to avoid errors
- Implement fallback logic (try USB, fall back to SPI)
- Log channel availability status

Precondition

`mgr` must be non-NULL and initialized  
`ready` must be non-NULL  
`channel` must be valid (USB or SPI)

Postcondition

`*ready` set to true or false based on channel state  
 On error, `*ready` set to false (safe default)

Note

USB readiness can change at runtime (host connects/disconnects)  
 SPI readiness is static (always ready if handle configured)  
 This function does NOT initiate communication - only queries state

Example - Send with Fallback:

```
uint8_t payload[16] = { ... };

// Try USB first
bool usb_ready = false;
rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                             k_comm_channel_usb,
                                             &usb_ready);
if (err == k_rx_ok && usb_ready) {
    err = rx_comm_manager_respond(&g_comm_mgr,
                                 k_comm_channel_usb,
                                 payload,
                                 sizeof(payload));
} else {
    // Fallback to SPI
    bool spi_ready = false;
    err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                       k_comm_channel_spi,
                                       &spi_ready);
    if (err == k_rx_ok && spi_ready) {
        err = rx_comm_manager_respond(&g_comm_mgr,
                                     k_comm_channel_spi,
                                     payload,
                                     sizeof(payload));
    }
}
```

Example - Log Channel Status:

```
void log_channel_status(void)
{
    bool usb_ready = false, spi_ready = false;

    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_usb, &usb_ready);
    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_spi, &spi_ready);

    printf("USB: %s, SPI: %s\n",
        usb_ready ? "READY" : "NOT READY",
        spi_ready ? "READY" : "NOT READY");
}
```

See also

[rx\\_usb\\_is\\_configured\(\)](#) USB enumeration status  
[rx\\_comm\\_manager\\_send\(\)](#) Send frame on channel

Since

Version 1.0.0

Definition at line 1857 of file [rx\\_comm\\_manager.c](#).

```
1858 {
1859     /* Rule 5: Pre-condition validation */
1860     if (mgr == nullptr || ready == nullptr) {
1861         if (ready != nullptr) {
1862             *ready = false;
1863         }
1864         return k_rx_err_invalid_arg;
1865     }
1866     if (!mgr->initialized) {
1867         *ready = false;
1868         return k_rx_err_invalid_state;
1869     }
1870
1871     switch (channel) {
1872         case k_comm_channel_uart:
1873             *ready = (bool)(mgr->uart_handle != nullptr);
1874             break;
1875
1876         case k_comm_channel_spi:
1877             *ready = (bool)(mgr->spi_handle != nullptr);
1878             break;
1879
1880         case k_comm_channel_i2c:
1881             *ready = (bool)(mgr->i2c_handle != nullptr);
1882             break;
1883
1884         default:
1885             *ready = false;
1886             rx_log_error(s_tag, "Channel ready: invalid channel");
1887             return k_rx_err_invalid_arg;
1888     }
1889
1890     return k_rx_ok;
1891 }
```

References [rx\\_comm\\_manager\\_t::i2c\\_handle](#), [rx\\_comm\\_manager\\_t::initialized](#), [k\\_comm\\_channel\\_i2c](#), [k\\_comm\\_channel\\_spi](#), [k\\_comm\\_channel\\_uart](#), [s\\_tag](#), [rx\\_comm\\_manager\\_t::spi\\_handle](#), and [rx\\_comm\\_manager\\_t::uart\\_handle](#).

#### 4.1.5.3 rx'comm'manager'deinit()

```
rx_err_t rx_comm_manager_deinit (
    rx_comm_manager_t * mgr) [nodiscard]
```

Deinitialize communication manager.

Does not deinitialize the underlying USB/SPI handles.

#### Parameters

|        |     |                           |
|--------|-----|---------------------------|
| in,out | mgr | Pointer to manager handle |
|--------|-----|---------------------------|

#### Returns

k\_rx\_ok on success

Deinitialize communication manager.

Marks communication manager as uninitialized, preventing further operations. This function does NOT deinitialize USB or SPI transport handles - caller must separately call rx\_usb\_comm\_deinit() and rx\_spi\_comm\_deinit() if needed.

#### Algorithm:

1. Validate mgr pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated). Function primarily serves as safety guard to prevent use-after-deinit.

#### Precondition

mgr must be non-NULL  
mgr must be initialized (mgr->initialized == true)

#### Postcondition

mgr->initialized = false  
Subsequent calls to rx\_comm\_manager\_poll/send will return k\_rx\_err\_invalid\_state

#### Note

Does NOT free memory (handle is statically allocated)  
Does NOT deinitialize transport handles (USB/SPI)  
Thread-safe if no concurrent operations on same mgr handle

#### Example:

```
// Deinitialize manager
rx_err_t err = rx_comm_manager_deinit(&g_comm_mgr);
if (err == k_rx_ok) {
    // Manager no longer usable

    // Separately deinitialize transports if needed
    rx_usb_comm_deinit(&g_usb_handle);
    rx_spi_comm_deinit(&g_spi_handle);
}
```

See also

[rx\\_comm\\_manager\\_init\(\)](#) Initialization  
[rx\\_spi\\_comm\\_deinit\(\)](#) SPI transport cleanup

Since

Version 1.0.0

Definition at line 1216 of file [rx\\_comm\\_manager.c](#).

```
01217 {
01218     /* Rule 5: Pre-condition validation */
01219     if (mgr == nullptr) {
01220         rx_log_error(s_tag, "Deinit failed: NULL mgr");
01221         return k_rx_err_invalid_arg;
01222     }
01223     if (!mgr->initialized) {
01224         rx_log_warn(s_tag, "Deinit called on uninitialized mgr");
01225         return k_rx_err_invalid_state;
01226     }
01227     mgr->initialized = false;
01228
01229     rx_log_info(s_tag, "Comm manager deinitialized");
01230     return k_rx_ok;
01231 }
01232 }
```

References [rx\\_comm\\_manager\\_t::initialized](#), and [s\\_tag](#).

#### 4.1.5.4 rx'comm'manager'event'send()

```
rx_err_t rx_comm_manager_event_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Queue data on the event path (reliable, SPI retry).

Event path is for commands and other data requiring reliable delivery. The payload is copied into a static FIFO queue and processed by the poll loop. Retry behavior depends on transport:

- SPI: Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
- USB: Fire-and-forget (USB HW reliability is sufficient)

Queue is processed in FIFO order by [rx\\_comm\\_manager\\_poll\(\)](#).

Parameters

|        |        |                                                              |
|--------|--------|--------------------------------------------------------------|
| in,out | mgr    | Pointer to initialized manager                               |
| in     | params | Send parameters (channel, type, flags, payload, payload_len) |

Returns

[k\\_rx\\_ok](#) on success (queued)  
[k\\_rx\\_err\\_invalid\\_arg](#) if mgr or params is nullptr  
[k\\_rx\\_err\\_invalid\\_state](#) if not initialized  
[k\\_rx\\_err\\_no\\_mem](#) if event queue is full

## Precondition

mgr must be initialized  
 params must be valid  
 params->payload\_len <= k\_frame\_max\_payload

## Postcondition

Payload copied to queue, will be sent by next poll()

## Note

Payload is COPIED into queue - caller can reuse buffer immediately

## Warning

Queue depth is k\_comm\_event\_queue\_depth (8). If full, returns error.

## See also

[rx\\_comm\\_manager\\_stream\\_send\(\)](#) For fire-and-forget telemetry

## Since

Version 1.0.0

Definition at line 1698 of file [rx\\_comm\\_manager.c](#).

```

01699 {
01700     if (mgr == nullptr || params == nullptr) {
01701         rx_log_error(s_tag, "Event enqueue failed: NULL arg");
01702         return k_rx_err_invalid_arg;
01703     }
01704
01705     if (!mgr->initialized) {
01706         rx_log_error(s_tag, "Event enqueue failed: not initialized");
01707         return k_rx_err_invalid_state;
01708     }
01709
01710     if (params->payload_len > k_frame_max_payload) {
01711         rx_log_error_val(s_tag, "Event enqueue failed: payload too large", params->payload_len);
01712         return k_rx_err_invalid_arg;
01713     }
01714
01715     if (params->payload_len > 0 && params->payload == nullptr) {
01716         rx_log_error(s_tag, "Event enqueue failed: NULL payload with nonzero len");
01717         return k_rx_err_invalid_arg;
01718     }
01719
01720     /* Check queue capacity */
01721     if (mgr->event_queue_count >= k_comm_event_queue_depth) {
01722         rx_log_error_val(s_tag, "Event queue full, depth", (uint8_t)mgr->event_queue_count);
01723         return k_rx_err_no_mem;
01724     }
01725
01726     /* Enqueue: copy payload into static slot */
01727     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_head];
01728     entry->channel          = params->channel;
01729     entry->type             = params->type;
01730     entry->flags            = params->flags;
01731     entry->payload_len      = params->payload_len;
01732     entry->retries          = 0;
01733     entry->occupied         = true;
01734
01735     if (params->payload != nullptr && params->payload_len > 0) {
01736         for (uint32_t i = 0; i < params->payload_len; i++) {
01737             entry->payload[i] = params->payload[i];

```

```

01738     }
01739 }
01740
01741 mgr->event_queue_head = (mgr->event_queue_head + 1) % k_comm_event_queue_depth;
01742 mgr->event_queue_count++;
01743
01744 rx_log_debug_val(s_tag, "Event enqueued, queue depth", mgr->event_queue_count);
01745 return k_rx_ok;
01746 }

```

References [rx\\_comm\\_event\\_entry\\_t::channel](#), [rx\\_comm\\_send\\_params\\_t::channel](#), [rx\\_comm\\_manager\\_t::event\\_queue\\_count](#), [rx\\_comm\\_manager\\_t::event\\_queue\\_head](#), [rx\\_comm\\_event\\_entry\\_t::flags](#), [rx\\_comm\\_send\\_params\\_t::flags](#), [rx\\_comm\\_manager\\_t::initialized](#), [k\\_comm\\_event\\_queue\\_depth](#), [rx\\_comm\\_event\\_entry\\_t::occupied](#), [rx\\_comm\\_event\\_entry\\_t::payload](#), [rx\\_comm\\_send\\_params\\_t::payload](#), [rx\\_comm\\_event\\_entry\\_t::payload\\_len](#), [rx\\_comm\\_send\\_params\\_t::payload\\_len](#), [rx\\_comm\\_event\\_entry\\_t::retries](#), [s\\_tag](#), [rx\\_comm\\_event\\_entry\\_t::type](#), and [rx\\_comm\\_send\\_params\\_t::type](#).

#### 4.1.5.5 rx'comm'manager'init()

```

rx_err_t rx_comm_manager_init (
    rx_comm_manager_t * mgr,
    const rx_comm_manager_config_t * cfg)  [nodiscard]

```

Initialize communication manager.

Initializes the communication manager with the provided configuration. Validates configuration parameters and sets up internal state for USB and/or SPI channel operation.

##### Parameters

|     |     |                                                           |
|-----|-----|-----------------------------------------------------------|
| out | mgr | Pointer to manager handle                                 |
| in  | cfg | Configuration (NULL for defaults, both channels disabled) |

##### Returns

k\_rx\_ok on success  
 k\_rx\_err\_invalid\_arg if mgr is nullptr  
 k\_rx\_err\_invalid\_arg if spi\_link is non-NULL but not initialized  
 k\_rx\_err\_invalid\_arg if spi\_link->spi\_handle != cfg->spi\_handle

##### Precondition

mgr must not be nullptr  
 If cfg->spi\_link is non-NULL, spi\_link must be initialized via rx\_spi\_link\_init()  
 If cfg->spi\_link is non-NULL, spi\_link->spi\_handle must equal cfg->spi\_handle  
 cfg may be nullptr (results in default configuration with both channels disabled)

##### Postcondition

mgr->usb\_handle == cfg->usb\_handle (or nullptr if cfg is nullptr)  
 mgr->spi\_handle == cfg->spi\_handle (or nullptr if cfg is nullptr)  
 mgr->spi\_link == cfg->spi\_link (or nullptr if cfg is nullptr or cfg->spi\_link is nullptr)  
 All internal state initialized (event queue empty, heartbeat timers reset)

#### Note

Thread-safe: May be called from any thread, but concurrent calls with same mgr are prohibited

#### Warning

Must be called before any other manager operations on this handle

#### See also

`rx_spi_link_init()` Must call this before passing `spi_link` to manager

[`rx\_comm\_manager\_deinit\(\)`](#) Cleanup function

#### Since

Version 1.0.0

Initialize communication manager.

Performs complete initialization of communication manager handle, configuring enabled channels, callback registration, and ASCII debugging output. This function must be called before any other communication manager operations.

#### Algorithm:

1. Validate parameters: Check mgr pointer for nullptr (NASA Rule 5)
2. Clear handle: Zero-fill entire structure via `memset` (328 bytes)
3. Apply configuration: Copy config fields if `cfg` is non-NULL
  - `usb_handle`: Pointer to initialized USB comm handle (may be NULL)
  - `spi_handle`: Pointer to initialized SPI comm handle (may be NULL)
  - `callback`: User frame handler function (may be NULL)
  - `callback_ctx`: User context pointer (may be NULL)
  - `enable_decoded_output`: ASCII debugging flag
4. Mark initialized: Set `mgr->initialized = true`
5. Post-condition validation: Verify `ascii_buffer` properly cleared

#### Configuration Options:

- `cfg = NULL`: All channels disabled, no callback, ASCII output off
- `usb_handle = NULL`: USB channel disabled (poll skips USB)
- `spi_handle = NULL`: SPI channel disabled (poll skips SPI)
- `callback = NULL`: No callback invoked (silent frame consumption)
- `enable_decoded_output = true`: ASCII debugging enabled (Port 1)



## Precondition

- mgr must point to allocated memory (uninitialized content OK)
- USB/SPI handles (if provided) must already be initialized
- Callback function (if provided) must be valid and reentrant

## Postcondition

- mgr->initialized = true on success
- All mgr fields set to config values or zero
- mgr->ascii\_buffer is zero-filled (256 bytes)

## Note

This function does NOT initialize USB or SPI transports - caller must initialize rx\_usb\_comm and rx\_spi\_comm separately before passing handles to this function

On success, at least one channel should be enabled (usb\_handle or spi\_handle non-NULL)

Thread-safe ONLY if each thread uses a different mgr handle

## Performance:

- Execution time: ~5 us @ 240 MHz
- Dominated by memset (328 bytes)

## Configuration Example - Both Channels Enabled:

```
static rx_comm_manager_t g_comm_mgr;
static rx_usb_comm_handle_t g_usb_handle;
static rx_spi_comm_handle_t g_spi_handle;

// Initialize transport layers first
rx_usb_comm_config_t usb_cfg = { .port = k_usb_port_proto };
rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
if (err != k_rx_ok) return err;

rx_spi_comm_config_t spi_cfg = { .spi_channel = 0 };
err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
if (err != k_rx_ok) return err;

// Initialize communication manager with both channels
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = &g_spi_handle,
    .callback = on_frame_received,
    .callback_ctx = nullptr,
    .enable_decoded_output = true,
};
err = rx_comm_manager_init(&g_comm_mgr, &cfg);
```

## Configuration Example - USB Only, No Callback:

```
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = nullptr, // SPI disabled
    .callback = nullptr,   // No callback
    .callback_ctx = nullptr,
    .enable_decoded_output = false, // No ASCII debugging
};
rx_err_t err = rx_comm_manager_init(&g_comm_mgr, &cfg);
```

See also

[rx\\_comm\\_manager\\_deinit\(\)](#) Cleanup and resource release  
[rx\\_comm\\_manager\\_poll\(\)](#) Poll for incoming frames  
[rx\\_spi\\_comm\\_init\(\)](#) Initialize SPI transport  
[rx\\_uart\\_comm\\_init\(\)](#) Initialize UART transport  
[rx\\_i2c\\_comm\\_init\(\)](#) Initialize I2C peripheral transport

Since

Version 1.0.0

Definition at line 1120 of file [rx\\_comm\\_manager.c](#).

```

01121 {
01122     /* Rule 5: Pre-condition validation - nullptr check */
01123     if (mgr == nullptr) {
01124         rx_log_error(s_tag, "Init failed: NULL mgr");
01125         return k_rx_err_invalid_arg;
01126     }
01127
01128     /* Rule 5: Pre-condition validation - SPI link consistency checks */
01129     if (cfg != nullptr && cfg->spi_link != nullptr) {
01130         /* If spi_link is provided, spi_handle must also be provided */
01131         if (cfg->spi_handle == nullptr) {
01132             rx_log_error(s_tag, "Init failed: spi_link requires non-NULL spi_handle");
01133             return k_rx_err_invalid_arg;
01134         }
01135
01136         /* If spi_link is provided, it must be initialized */
01137         if (!cfg->spi_link->initialized) {
01138             rx_log_error(s_tag, "Init failed: spi_link not initialized (call rx_spi_link_init first)");
01139             return k_rx_err_invalid_arg;
01140         }
01141
01142         /* If spi_link is provided, its spi_handle must match config spi_handle */
01143         if (cfg->spi_link->spi_handle != cfg->spi_handle) {
01144             rx_log_error(s_tag, "Init failed: spi_link->spi_handle != cfg->spi_handle");
01145             return k_rx_err_invalid_arg;
01146         }
01147     }
01148
01149     *mgr = s_zero_mgr;
01150
01151     /* Apply configuration */
01152     if (cfg != nullptr) {
01153         mgr->uart_handle      = cfg->uart_handle;
01154         mgr->spi_handle       = cfg->spi_handle;
01155         mgr->i2c_handle       = cfg->i2c_handle;
01156         mgr->spi_link         = cfg->spi_link;
01157         mgr->callback         = cfg->callback;
01158         mgr->callback_ctx     = cfg->callback_ctx;
01159         mgr->enable_decoded_output = cfg->enable_decoded_output;
01160         mgr->link_status_cb   = cfg->link_status_cb;
01161         mgr->link_status_ctx  = cfg->link_status_ctx;
01162     }
01163
01164     mgr->initialized = true;
01165
01166     rx_log_info(s_tag, "Comm manager initialized");
01167     return k_rx_ok;
01168 }

```

References [rx\\_comm\\_manager\\_config\\_t::callback](#), [rx\\_comm\\_manager\\_t::callback](#), [rx\\_comm\\_manager\\_config\\_t::callback\\_ctx](#), [rx\\_comm\\_manager\\_t::callback\\_ctx](#), [rx\\_comm\\_manager\\_config\\_t::enable\\_decoded\\_output](#), [rx\\_comm\\_manager\\_t::enable\\_decoded\\_output](#), [rx\\_comm\\_manager\\_config\\_t::i2c\\_handle](#), [rx\\_comm\\_manager\\_t::i2c\\_handle](#), [rx\\_comm\\_manager\\_t::initialized](#), [rx\\_comm\\_manager\\_config\\_t::link\\_status\\_cb](#), [rx\\_comm\\_manager\\_t::link\\_status\\_cb](#), [rx\\_comm\\_manager\\_config\\_t::link\\_status\\_ctx](#), [rx\\_comm\\_manager\\_t::link\\_status\\_ctx](#), [s\\_tag](#), [s\\_zero\\_mgr](#), [rx\\_comm\\_manager\\_config\\_t::spi\\_handle](#), [rx\\_comm\\_manager\\_t::spi\\_handle](#), [rx\\_comm\\_manager\\_config\\_t::spi\\_link](#), [rx\\_comm\\_manager\\_t::spi\\_link](#), [rx\\_comm\\_manager\\_config\\_t::uart\\_handle](#), and [rx\\_comm\\_manager\\_t::uart\\_handle](#).

## 4.1.5.6 rx'comm'manager'link'status()

```
rx_err_t rx_comm_manager_link_status (
    const rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    rx_comm_link_status_t * status) [nodiscard]
```

Query link health status for a transport channel.

Returns the current heartbeat-derived link status for the given channel. Status is updated by [rx\\_comm\\_manager\\_poll\(\)](#) based on the dual-detection heartbeat model (200ms implicit timeout).

## Parameters

|     |         |                                |
|-----|---------|--------------------------------|
| in  | mgr     | Pointer to initialized manager |
| in  | channel | Channel to query               |
| out | status  | Receives current link status   |

## Returns

k\_rx\_ok on success  
k\_rx\_err\_invalid\_arg if mgr or status is nullptr

## Since

Version 1.0.0

Definition at line 1753 of file [rx\\_comm\\_manager.c](#).

```
01756 {
01757     if (mgr == nullptr || status == nullptr) {
01758         rx_log_error(s_tag, "Link status query: NULL arg");
01759         return k_rx_err_invalid_arg;
01760     }
01761
01762     if (!mgr->initialized) {
01763         rx_log_error(s_tag, "Link status query: uninitialized manager");
01764         return k_rx_err_invalid_state;
01765     }
01766
01767     if (channel >= k_comm_channel_count) {
01768         rx_log_error(s_tag, "Link status query: invalid channel");
01769         return k_rx_err_invalid_arg;
01770     }
01771
01772     *status = mgr->heartbeat[channel].status;
01773     return k_rx_ok;
01774 }
```

References [rx\\_comm\\_manager\\_t::heartbeat](#), [rx\\_comm\\_manager\\_t::initialized](#), [k\\_comm\\_channel\\_count](#), [s\\_tag](#), and [rx\\_comm\\_heartbeat\\_state\\_t::status](#).

## 4.1.5.7 rx'comm'manager'poll()

```
rx_err_t rx_comm_manager_poll (
    rx_comm_manager_t * mgr) [nodiscard]
```

Poll all enabled channels for incoming frames.

Non-blocking check of USB and SPI channels. For each complete frame received:

1. If decoded output enabled, format and send ASCII to Port 1
2. Invoke user callback with frame data and channel

Call this from main loop or dedicated communication task.

#### Parameters

|        |     |                                |
|--------|-----|--------------------------------|
| in,out | mgr | Pointer to initialized manager |
|--------|-----|--------------------------------|

#### Returns

k\_rx\_ok if at least one frame was received  
k\_rx\_err\_timeout if no frames available on any channel  
k\_rx\_err\_invalid\_arg if mgr is nullptr  
k\_rx\_err\_invalid\_state if not initialized

Definition at line 1383 of file [rx\\_comm\\_manager.c](#).

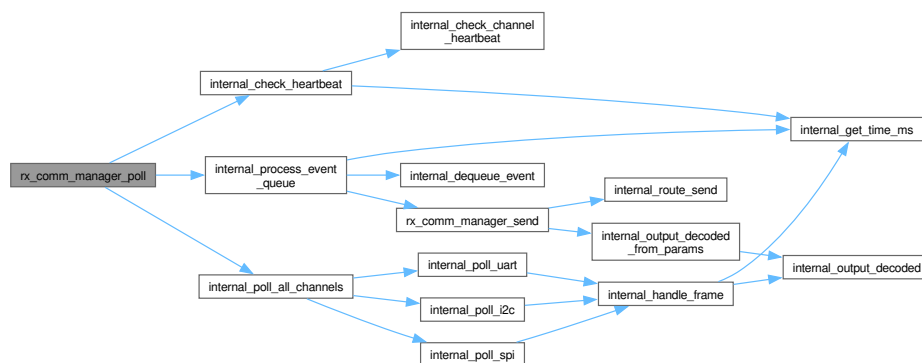
```

01384 {
01385  /* Rule 5: Pre-condition validation */
01386  if (mgr == nullptr) {
01387    return k_rx_err_invalid_arg;
01388  }
01389  if (!mgr->initialized) {
01390    return k_rx_err_invalid_state;
01391  }
01392
01393  bool    received = false;
01394  rx_err_t poll_err = internal_poll_all_channels(mgr, &received);
01395  if (poll_err != k_rx_ok) {
01396    return poll_err;
01397  }
01398
01399  /* Process event queue: send one queued event per poll cycle */
01400  internal_process_event_queue(mgr);
01401
01402  /* Heartbeat: check implicit timeout on each transport */
01403  internal_check_heartbeat(mgr);
01404
01405  return (int)received ? k_rx_ok : k_rx_err_timeout;
01406 }

```

References [rx\\_comm\\_manager\\_t::initialized](#), [internal\\_check\\_heartbeat\(\)](#), [internal\\_poll\\_all\\_channels\(\)](#), and [internal\\_process\\_event\\_queue\(\)](#).

Here is the call graph for this function:



## 4.1.5.8 rx'comm'manager'process'retransmits()

```
rx_err_t rx_comm_manager_process_retransmits (
    rx_comm_manager_t * mgr,
    uint32_t current_time_ms) [nodiscard]
```

Process pending retransmissions on SPI channel.

Thin pass-through to rx\_spi\_comm\_process\_retransmits() on the SPI handle. Call this from the communication task polling loop.

## Parameters

|        |                 |                                     |
|--------|-----------------|-------------------------------------|
| in,out | mgr             | Communication manager               |
| in     | current_time_ms | Current system time in milliseconds |

## Returns

rx\_err\_t Error code from rx\_spi\_comm\_process\_retransmits()

## Return values

|                        |                                     |
|------------------------|-------------------------------------|
| k_rx_ok                | No action needed or retransmit sent |
| k_rx_err_invalid_arg   | mgr is nullptr                      |
| k_rx_err_invalid_state | mgr not initialized                 |
| k_rx_err_retry_limit   | Max retries exceeded                |

## Since

Version 1.0.0

Definition at line 1965 of file rx\_comm\_manager.c.

```
01966 {
01967     if (mgr == nullptr) {
01968         return k_rx_err_invalid_arg;
01969     }
01970
01971     if (!mgr->initialized) {
01972         return k_rx_err_invalid_state;
01973     }
01974
01975     if (mgr->spi_handle == nullptr) {
01976         return k_rx_ok; /* No SPI channel configured */
01977     }
01978
01979     return rx_spi_comm_process_retransmits(mgr->spi_handle, current_time_ms);
01980 }
```

References [rx\\_comm\\_manager\\_t::initialized](#), and [rx\\_comm\\_manager\\_t::spi\\_handle](#).

#### 4.1.5.9 rx\_comm\_manager\_respond()

```
rx_err_t rx_comm_manager_respond (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    const uint8_t * payload,
    uint32_t payload_len) [nodiscard]
```

Send response on same channel as received frame.

Convenience function for responding to commands. Sends a RESPONSE type frame on the specified channel.

Parameters

|        |             |                                       |
|--------|-------------|---------------------------------------|
| in,out | mgr         | Pointer to initialized manager        |
| in     | channel     | Channel to respond on (from callback) |
| in     | payload     | Response payload data                 |
| in     | payload_len | Response payload length               |

Returns

k\_rx\_ok on success

Send response on same channel as received frame.

Convenience function for sending response frames. Automatically sets frame type to k\_frame\_type\_response and flags to k\_frame\_flag\_none. Simplifies common use case of responding to commands/requests.

Algorithm:

1. Build rx\_comm\_send\_params\_t with:
  - channel: User-specified
  - type: k\_frame\_type\_response (fixed)
  - flags: k\_frame\_flag\_none (fixed)
  - payload: User-specified
  - payload\_len: User-specified
2. Call rx\_comm\_manager\_send() with constructed params
3. Return result from rx\_comm\_manager\_send()

When to Use:

- Responding to commands with telemetry data
- Sending acknowledgments
- Replying to requests

When NOT to Use:

- Need custom frame type (use rx\_comm\_manager\_send instead)
- Need custom flags (use rx\_comm\_manager\_send instead)

## Precondition

Same preconditions as [rx\\_comm\\_manager\\_send\(\)](#)

## Postcondition

Response frame transmitted with type=response, flags=none

## Note

Equivalent to calling [rx\\_comm\\_manager\\_send\(\)](#) with type=k\_frame\_type\_response

This is a thin wrapper - no additional validation performed

## Example - Send Telemetry Response:

```
// Received a telemetry request on USB, send response
static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
{
    uint8_t telemetry[16];

    // Gather telemetry data
    telemetry[0] = get_temperature_celsius();
    telemetry[1] = get_current_ma();
    // ... fill remaining fields ...

    // Send response on same channel request came from
    rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                           channel,
                                           telemetry,
                                           sizeof(telemetry));

    if (err != k_rx_ok) {
        // Handle send error
    }
}
```

## Example - Send Empty Acknowledgment:

```
// Send empty ACK response
rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                       k_comm_channel_usb,
                                       NULL, // No payload
                                       0);    // Zero length
```

## See also

[rx\\_comm\\_manager\\_send\(\)](#) Full send function with all parameters

[rx\\_comm\\_send\\_params\\_t](#) Send parameters structure

## Since

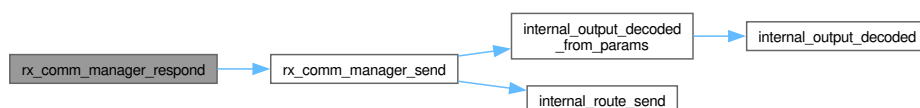
Version 1.0.0

Definition at line 1670 of file [rx\\_comm\\_manager.c](#).

```
01674 {
01675     const rx_comm_send_params_t params = {
01676         .channel    = channel,
01677         .type       = k_frame_type_response,
01678         .flags      = k_frame_flag_none,
01679         .payload     = payload,
01680         .payload_len = payload_len,
01681     };
01682     return rx_comm_manager_send(mgr, &params);
01683 }
```

References [rx\\_comm\\_manager\\_send\(\)](#).

Here is the call graph for this function:



## 4.1.5.10 rx\_comm\_manager\_send()

```
rx_err_t rx_comm_manager_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send frame on specific channel.

Encodes and sends a frame on the specified channel. If decoded output is enabled, also sends ASCII representation to Port 1.

Parameters

|        |        |                                                              |
|--------|--------|--------------------------------------------------------------|
| in,out | mgr    | Pointer to initialized manager                               |
| in     | params | Send parameters (channel, type, flags, payload, payload_len) |

Returns

k\_rx\_ok on success  
k\_rx\_err\_invalid\_arg if mgr or payload is nullptr  
k\_rx\_err\_invalid\_state if not initialized or channel not enabled

Definition at line 1560 of file rx\_comm\_manager.c.

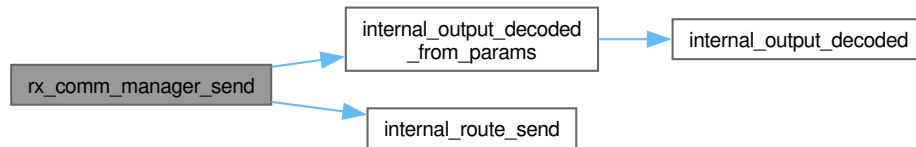
```
01561 {
01562     /* Rule 5: Pre-condition validation */
01563     if (mgr == nullptr || params == nullptr) {
01564         rx_log_error(s_tag, "Send failed: NULL arg");
01565         return k_rx_err_invalid_arg;
01566     }
01567     if (!mgr->initialized) {
01568         rx_log_error(s_tag, "Send failed: not initialized");
01569         return k_rx_err_invalid_state;
01570     }
01571     if (params->payload == nullptr && params->payload_len > 0) {
01572         rx_log_error(s_tag, "Send failed: NULL payload with nonzero len");
01573         return k_rx_err_invalid_arg;
01574     }
01575     /* Validate payload length is within frame limits */
01576     if (params->payload_len > k_frame_max_payload) {
01577         rx_log_error_val(s_tag, "Send failed: payload too large", params->payload_len);
01578         return k_rx_err_invalid_arg;
01579     }
01580
01581     rx_err_t err = internal_route_send(mgr, params);
01582
01583     if (err != k_rx_ok) {
01584         rx_log_error_val(s_tag, "Transport send failed", err);
01585     }
01586
01587     /* Output decoded ASCII on success */
01588     if (err == k_rx_ok && (int)mgr->enable_decoded_output) {
01589         internal_output_decoded_from_params(mgr, params);
01590     }
01591
01592     return err;
01593 }
```

References [rx\\_comm\\_manager\\_t::enable\\_decoded\\_output](#), [rx\\_comm\\_manager\\_t::initialized](#), [internal\\_output\\_decoded\\_from\\_params\(\)](#), [internal\\_route\\_send\(\)](#), [rx\\_comm\\_send\\_params\\_t::payload](#), [rx\\_comm\\_send\\_params\\_t::payload\\_len](#), and [s\\_tag](#).

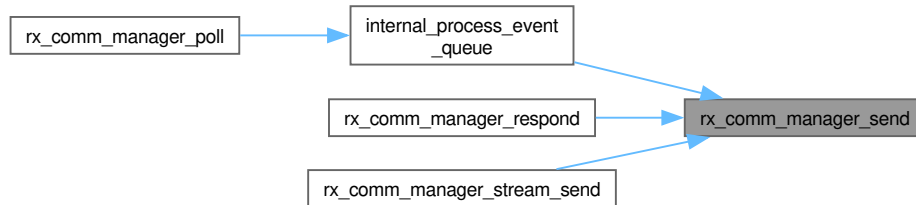
Referenced by [internal\\_process\\_event\\_queue\(\)](#), [rx\\_comm\\_manager\\_respond\(\)](#), and [rx\\_comm\\_manager\\_stream\\_send\(\)](#).

Here is the call graph for this function:





Here is the caller graph for this function:



#### 4.1.5.11 rx'comm'manager'set'auto'retransmit()

```

rx_err_t rx_comm_manager_set_auto_retransmit (
    rx_comm_manager_t * mgr,
    bool enabled,
    const rx_spi_comm_retransmit_config_t * config) [nodiscard]

```

Enable or disable automatic retransmission on SPI channel.

Thin pass-through to rx\_spi\_comm\_set\_auto\_retransmit() on the SPI handle.

Parameters

| in,out | mgr     | Communication manager                             |
|--------|---------|---------------------------------------------------|
| in     | enabled | true to enable, false to disable                  |
| in     | config  | Retransmit config (NULL to keep current/defaults) |

Returns

rx\_err\_t Error code from rx\_spi\_comm\_set\_auto\_retransmit()

Since

Version 1.0.0

Definition at line 1982 of file rx\_comm\_manager.c.

```

01985 {
01986     if (mgr == nullptr) {
01987         return k_rx_err_invalid_arg;
01988     }
01989     if (!mgr->initialized) {
01990         return k_rx_err_invalid_state;
01991     }
01992 }
01993     if (mgr->spi_handle == nullptr) {
01994         return k_rx_err_not_supported;
01995     }
01996 }
01997     return rx_spi_comm_set_auto_retransmit(mgr->spi_handle, enabled, config);
01998 }
01999

```

References [rx\\_comm\\_manager\\_t::initialized](#), and [rx\\_comm\\_manager\\_t::spi\\_handle](#).

## 4.1.5.12 rx\_comm\_manager\_stream\_send()

```
rx_err_t rx_comm_manager_stream_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send data on the stream path (fire-and-forget).

Stream path is for time-sensitive data like telemetry where stale data has no value. Sends immediately with NO retries on any transport. If the send fails, the data is simply dropped.

- USB CDC: Direct send, no retries (USB HW provides reliability)
- SPI: Direct send, no retries (stale telemetry is worthless)

## Parameters

|        |        |                                                              |
|--------|--------|--------------------------------------------------------------|
| in,out | mgr    | Pointer to initialized manager                               |
| in     | params | Send parameters (channel, type, flags, payload, payload_len) |

## Returns

k\_rx\_ok on success  
k\_rx\_err\_invalid\_arg if mgr or params is nullptr  
k\_rx\_err\_invalid\_state if not initialized or channel not enabled

## Precondition

mgr must be initialized  
params must be valid

## Postcondition

Data sent or silently dropped on failure

## Note

This is a thin wrapper over [rx\\_comm\\_manager\\_send\(\)](#) for semantic clarity

## See also

[rx\\_comm\\_manager\\_event\\_send\(\)](#) For reliable command delivery

## Since

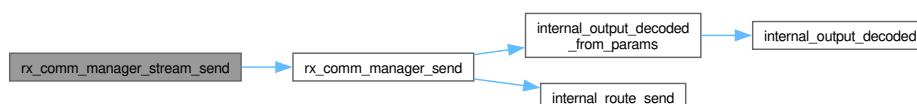
Version 1.0.0

Definition at line 1690 of file [rx\\_comm\\_manager.c](#).

```
01691 {
01692     /* Stream path: fire-and-forget, no retries, no queuing.
01693      * Telemetry and other time-sensitive data goes here.
01694      * Stale data has no value, so failure is silently accepted. */
01695     return rx_comm_manager_send(mgr, params);
01696 }
```

References [rx\\_comm\\_manager\\_send\(\)](#).

Here is the call graph for this function:



## 4.2 rx\_comm\_manager.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_comm_manager.h
00003  * @brief Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols
00004  *
00005  * @details
00006  * ## Overview
00007  * Production-quality communication manager providing unified abstraction over
00008  * multiple simultaneous communication channels (USB CDC and SPI) with automatic
00009  * frame protocol handling, non-blocking polling architecture, optional decoded
00010  * ASCII output for debugging, and callback-based event notification.
00011  *
00012  * **Purpose**: Coordinate independent, simultaneous communication channels from
00013  * RX72N MCU to Raspberry Pi 5 host, enabling command/response messaging over
00014  * both USB (via CDC class) and SPI (direct hardware connection).
00015  *
00016  * **Protocol**: Both channels use identical frame protocol (rx_frame) allowing
00017  * host to send commands on either channel. Manager automatically handles:
00018  * - Frame reception and validation
00019  * - ASCII decoding output (debugging via USB Port 1)
00020  * - Channel-aware response routing
00021  * - User callback invocation
00022  *
00023  * **Key Features**:
00024  * - **Multi-channel support**: Simultaneous USB and SPI communication
00025  * - **Non-blocking architecture**: Poll-based, no busy-waiting or blocking calls
00026  * - **Protocol-agnostic**: Both channels use same frame format (type, flags, payload, CRC)
00027  * - **Automatic ASCII output**: Optional decoded frames to USB Port 1 (human-readable debugging)
00028  * - **Callback-driven**: User-defined handler invoked on frame reception
00029  * - **Channel awareness**: Responses automatically routed to originating channel
00030  * - **Zero-copy where possible**: Direct access to frame buffers
00031  * - **Thread-safe design**: Can be called from RTOS task or main loop
00032  *
00033  * ## System Architecture
00034  *
00035  * Communication manager sits between application logic and low-level transport drivers:
00036  *
00037  * @dot
00038  * digraph comm_manager_arch {
00039  *   rankdir=TB;
00040  *   node [shape=box, style=rounded];
00041  *
00042  *   app [label="Application\n(Command Handlers)", fillcolor=lightblue, style=filled];
00043  *   comm_mgr [label="Communication Manager\n(This Module)", fillcolor=lightgreen, style=filled];
00044  *   usb_comm [label="rx_usb_comm\n(USB CDC Transport)", fillcolor=lightyellow, style=filled];
00045  *   spi_comm [label="rx_spi_comm\n(SPI Transport)", fillcolor=lightyellow, style=filled];
00046  *   usb_hw [label="USB Hardware\n(USB peripheral)", fillcolor=lightgray, style=filled];
00047  *   spi_hw [label="SPI Hardware\n(RSPI peripheral)", fillcolor=lightgray, style=filled];
00048  *   rpi5 [label="Raspberry Pi 5\n(Host)", shape=component];
00049  *
00050  *   app -> comm_mgr [label="poll()\nsend()\nrespond()"];
00051  *   comm_mgr -> app [label="callback(channel, frame)", dir=back];
00052  *   comm_mgr -> usb_comm [label="receive_frame()"];
00053  *   comm_mgr -> spi_comm [label="receive_frame()"];
00054  *   usb_comm -> usb_hw [label="CDC transfers"];
00055  *   spi_comm -> spi_hw [label="SPI transfers"];
00056  *   usb_hw -> rpi5 [label="/dev/ttyACM0"];
00057  *   spi_hw -> rpi5 [label="SPI bus"];
00058  *
00059  *   decoded [label="USB Port 1\n(ASCII Debug)", fillcolor=lightpink, style=filled];
00060  *   comm_mgr -> decoded [label="ASCII frames\n(optional)"];
00061  *   decoded -> rpi5 [label="/dev/ttyACM1"];
00062  * }
00063  * @enddot
00064  *
00065  * ## Channel Configuration
00066  *
00067  * The manager supports two independent communication channels:
00068  *
00069  * | Channel | Transport | Device | Usage | RPi5 Interface |
00070  * |-----|-----|-----|-----|-----|
00071  * | **USB** | CDC class | USB Port 0 | Primary protocol channel | /dev/ttyACM0 |
00072  * | **SPI** | RSPI peripheral | RSPI channel | High-speed backup channel | /dev/spidev0.0 |
00073  * | **Debug** | CDC class | USB Port 1 | ASCII decoded output (optional) | /dev/ttyACM1 |
00074  *
00075  * **USB Channel (k_comm_channel_usb)**:
00076  * - Plug-and-play connectivity (no additional wiring)
00077  * - Full-speed USB 2.0 (up to 12 Mbps theoretical, ~1 MB/s practical)
00078  * - Automatic enumeration and driver loading on RPi5
00079  * - Suitable for command/response and moderate-bandwidth telemetry
00080  *
00081  * **SPI Channel (k_comm_channel_spi)**:
00082  * - Dedicated hardware connection (4-6 wires: COPI, CIPO, SCK, CS, optional handshake)

```

```

00083 * - Configurable speed (typically 10-20 Mbps)
00084 * - Lower latency than USB (no protocol overhead)
00085 * - Suitable for high-frequency control loops and real-time data
00086 *
00087 * **Debug Output (USB Port 1):**
00088 * - Human-readable ASCII representation of all frames
00089 * - Format: `[USB/SPI] TYPE=cmd FLAGS=0x00 LEN=42 DATA=...`
00090 * - Enabled via `enable_decoded_output` config flag
00091 * - Zero impact on protocol channels (independent USB endpoint)
00092 *
00093 * ## Communication Flow State Machine
00094 *
00095 * The manager operates as a stateless polling architecture with per-frame state:
00096 *
00097 * @startuml
00098 * [*] --> Idle
00099 * Idle --> Polling : rx_comm_manager_poll()
00100 * Polling --> CheckUSB : USB enabled?
00101 * Polling --> CheckSPI : USB disabled or checked
00102 * CheckUSB --> DecodeUSB : Frame available
00103 * CheckUSB --> CheckSPI : No frame or error
00104 * DecodeUSB --> InvokeCallback : Valid frame
00105 * InvokeCallback --> FormatASCII : Decoded output enabled
00106 * InvokeCallback --> CheckSPI : No decoded output
00107 * FormatASCII --> SendASCII : Format success
00108 * SendASCII --> CheckSPI
00109 * CheckSPI --> DecodeSPI : Frame available
00110 * CheckSPI --> Idle : No frame or error
00111 * DecodeSPI --> InvokeCallback2 : Valid frame
00112 * InvokeCallback2 --> FormatASCII2 : Decoded output enabled
00113 * InvokeCallback2 --> Idle : No decoded output
00114 * FormatASCII2 --> SendASCII2 : Format success
00115 * SendASCII2 --> Idle
00116 *
00117 * note right of Polling
00118 *   Non-blocking check
00119 *   Returns immediately if
00120 *   no frames available
00121 * end note
00122 *
00123 * note right of InvokeCallback
00124 *   User callback receives:
00125 *   - channel (USB or SPI)
00126 *   - parsed frame
00127 *   - user context
00128 * end note
00129 * @enduml
00130 *
00131 * @par State Descriptions:
00132 * - **Idle**: Manager initialized, waiting for poll() call
00133 * - **Polling**: Actively checking enabled channels for frames
00134 * - **CheckUSB**: Query USB channel for available frame
00135 * - **CheckSPI**: Query SPI channel for available frame
00136 * - **DecodeUSB/DecodeSPI**: Validate and parse received frame
00137 * - **InvokeCallback**: Call user-provided frame handler
00138 * - **FormatASCII**: Convert frame to human-readable string
00139 * - **SendASCII**: Transmit ASCII representation to debug port
00140 *
00141 * ## Hardware Requirements
00142 *
00143 * | Resource | Requirement | Usage |
00144 * |-----|-----|-----|
00145 * | **USB Peripheral** | 1 instance (full-speed) | CDC class with 2 ports (protocol + debug) |
00146 * | **RSPi Peripheral** | 1 channel (optional) | SPI controller mode, 10-20 Mbps |
00147 * | **GPIO** | 4-6 pins (if SPI) | COPI, CIPO, SCK, CS, optional handshake/interrupt |
00148 * | **RAM** | ~2.5 KB per instance | Handle + ASCII buffer (2048 bytes) |
00149 * | **Flash** | ~4 KB | Code + string constants |
00150 * | **RTOS Resources** | None | Stateless, no threads/mutexes/semaphores |
00151 *
00152 * ## Performance Characteristics
00153 *
00154 * | Operation | Execution Time | Notes |
00155 * |-----|-----|-----|
00156 * | **rx_comm_manager_init** | ~50 us | Handle initialization, no hardware config |
00157 * | **rx_comm_manager_poll** | ~15-200 us | Depends on channels enabled and frames available |
00158 * | - No frames available | ~15 us | Quick check both channels, early return |
00159 * | - USB frame received | ~120 us | Frame validation + callback + ASCII (if enabled) |
00160 * | - SPI frame received | ~80 us | Faster than USB (less overhead) |
00161 * | - Both frames received | ~200 us | Sequential processing of both channels |
00162 * | **rx_comm_manager_send** | ~100-150 us | Frame encoding + transport layer send |
00163 * | **rx_comm_manager_respond** | ~100-150 us | Same as send (convenience wrapper) |
00164 * | **Callback overhead** | ~5-50 us | User-defined, depends on handler complexity |
00165 *
00166 * All timing measured @ 240 MHz CPU clock, -O2 optimization, typical frame size 64 bytes.
00167 *
00168 * ## Memory Usage
00169 *

```

```

00170 * | Allocation | Size | Lifetime |
00171 * |-----|-----|-----|
00172 * **rx_comm_manager_t** | ~12 KB | Application-managed (typically static/global) |
00173 * | - Pointers + state | ~80 bytes | Handle metadata, heartbeat, queue indices |
00174 * | - ASCII buffer | 2048 bytes | Decoded output formatting (if enabled) |
00175 * | - Event queue | 8 x ~1028 bytes | Static FIFO (k_comm_event_queue_depth entries) |
00176 * | - Heartbeat state | ~20 bytes | Per-transport heartbeat tracking |
00177 * **Stack (poll)** | ~120 bytes | Function call duration only |
00178 * **Stack (send)** | ~80 bytes | Function call duration only |
00179 * **Static data (.rodata)** | ~400 bytes | Channel names, format strings |
00180 * **Code (.text)** | ~6 KB | All functions |
00181 *
00182 * Total RAM: ~12 KB per manager instance (zero dynamic allocation).
00183 * The event queue dominates memory: 8 entries x (k_frame_max_payload + metadata).
00184 *
00185 * **Memory Layout** (rx_comm_manager_t structure):
00186 *
00187 * | Offset | Size | Field | Notes |
00188 * |-----|-----|-----|-----|
00189 * | 0x00 | 8 | usb_handle | Pointer to USB transport |
00190 * | 0x08 | 8 | spi_handle | Pointer to SPI transport |
00191 * | 0x10 | 8 | callback | Function pointer |
00192 * | 0x18 | 8 | callback_ctx | User context pointer |
00193 * | 0x20 | 2048 | ascii_buffer | ASCII formatting buffer |
00194 * | - | 1 | enable_decoded_output | Boolean flag |
00195 * | - | 1 | initialized | Boolean flag |
00196 * | - | ~20 | heartbeat[2] | Per-transport heartbeat state |
00197 * | - | ~16 | link_status_cb/ctx | Callback + context |
00198 * | - | ~8224 | event_queue[8] | 8 x rx_comm_event_entry_t |
00199 * | - | 3 | queue_head/tail/count | Queue management |
00200 * **Total** | **~10.5 KB** | | (actual size may vary by compiler) |
00201 *
00202 * ## Thread Safety Analysis
00203 *
00204 * **Conditional Thread Safety** - Safe if following rules are observed:
00205 *
00206 * 1. **Single poll context**: Only call `poll()` from ONE thread/task/loop
00207 * 2. **Callback restrictions**: User callback must not call manager functions (no recursion)
00208 * 3. **Send synchronization**: If calling `send()`/`respond()` from multiple threads, use external mutex
00209 * 4. **Transport thread safety**: Underlying USB/SPI transports must be thread-safe (typically are)
00210 *
00211 * **Not safe for**:
00212 * - Concurrent `poll()` calls from multiple threads
00213 * - Calling manager functions from within frame callback
00214 * - Simultaneous `send()` calls without external synchronization
00215 *
00216 * **Recommendation**: Use dedicated communication task:
00217 * ```c
00218 * void comm_task_entry(ULONG ctx) {
00219 *     rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
00220 *     while (1) {
00221 *         rx_comm_manager_poll(mgr);
00222 *         tx_thread_sleep(1); // 1ms polling period = 1kHz
00223 *     }
00224 * }
00225 * ```
00226 *
00227 * ## Integration Example
00228 *
00229 * @par Complete Setup and Usage:
00230 * @code{.c}
00231 * #include "rx_comm_manager.h"
00232 * #include "rx_usb_comm.h"
00233 * #include "rx_spi_comm.h"
00234 *
00235 * // Frame handler callback
00236 * void on_frame_received(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00237 *     rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
00238 *
00239 *     rx_log_info("COMM", "Frame from %s: type=0x%02X len=%u",
00240 *         rx_comm_manager_channel_name(channel),
00241 *         frame->type, frame->payload_len);
00242 *
00243 *     // Parse frame based on type
00244 *     switch (frame->type) {
00245 *         case k_frame_type_command:
00246 *             // Handle command
00247 *             uint8_t response[128];
00248 *             uint32_t response_len = process_command(frame->payload,
00249 *                 frame->payload_len,
00250 *                 response, sizeof(response));
00251 *
00252 *             // Respond on same channel
00253 *             rx_comm_manager_respond(mgr, channel, response, response_len);
00254 *             break;
00255 *
00256 *         case k_frame_type_telemetry_req:

```

```

00257 *          // Send telemetry data
00258 *          send_telemetry(mgr, channel);
00259 *          break;
00260 *
00261 *      default:
00262 *          rx_log_warn("COMM", "Unknown frame type: 0x%02X", frame->type);
00263 *          break;
00264 *  }
00265 * }
00266 *
00267 * // Main application
00268 * int main(void) {
00269 *     // Initialize transports
00270 *     rx_usb_comm_handle_t usb_comm;
00271 *     rx_spi_comm_handle_t spi_comm;
00272 *
00273 *     rx_usb_comm_init(&usb_comm, &usb_config);
00274 *     rx_spi_comm_init(&spi_comm, &spi_config);
00275 *
00276 *     // Initialize communication manager
00277 *     rx_comm_manager_t comm_mgr;
00278 *     rx_comm_manager_config_t cfg = {
00279 *         .usb_handle = &usb_comm,
00280 *         .spi_handle = &spi_comm,
00281 *         .callback = on_frame_received,
00282 *         .callback_ctx = &comm_mgr, // Pass manager to callback
00283 *         .enable_decoded_output = true, // ASCII debug on Port 1
00284 *     };
00285 *
00286 *     rx_err_t err = rx_comm_manager_init(&comm_mgr, &cfg);
00287 *     if (err != k_rx_ok) {
00288 *         rx_log_error("APP", "Comm manager init failed: 0x%X", err);
00289 *         return -1;
00290 *     }
00291 *
00292 *     // Main communication loop
00293 *     while (1) {
00294 *         // Poll for incoming frames (non-blocking)
00295 *         err = rx_comm_manager_poll(&comm_mgr);
00296 *
00297 *         // err == k_rx_ok: Frame(s) received and processed
00298 *         // err == k_rx_err_timeout: No frames available (normal)
00299 *
00300 *         // Other application tasks...
00301 *
00302 *         tx_thread_sleep(1); // 1ms = 1kHz polling rate
00303 *     }
00304 *
00305 *     return 0;
00306 * }
00307 * @endcode
00308 *
00309 * @par Sending Unsolicited Data (Telemetry):
00310 * @code{.c}
00311 * void send_periodic_telemetry(rx_comm_manager_t* mgr) {
00312 *     // Gather telemetry data
00313 *     telemetry_data_t telemetry;
00314 *     gather_telemetry(&telemetry);
00315 *
00316 *     // Serialize to buffer
00317 *     uint8_t payload[256];
00318 *     uint32_t payload_len = serialize_telemetry(&telemetry, payload, sizeof(payload));
00319 *
00320 *     // Send on USB channel (more reliable for telemetry)
00321 *     rx_comm_send_params_t params = {
00322 *         .channel = k_comm_channel_usb,
00323 *         .type = k_frame_type_telemetry,
00324 *         .flags = 0,
00325 *         .payload = payload,
00326 *         .payload_len = payload_len,
00327 *     };
00328 *
00329 *     rx_err_t err = rx_comm_manager_send(mgr, &params);
00330 *     if (err != k_rx_ok) {
00331 *         rx_log_warn("TELEM", "Failed to send telemetry: 0x%X", err);
00332 *     }
00333 * }
00334 * @endcode
00335 *
00336 * @par Module Dependencies:
00337 * - rx_usb_comm: USB CDC transport layer
00338 * - rx_spi_comm: SPI transport layer
00339 * - rx_frame: Frame protocol definition and encoding/decoding
00340 * - rx_err: Standardized error code system
00341 * - rx_log: Logging infrastructure
00342 *
00343 * @see rx_spi_comm.h SPI transport implementation

```

```

00344 * @see rx_uart_comm.h UART transport implementation
00345 * @see rx_i2c_comm.h I2C peripheral transport implementation
00346 * @see rx_frame.h Frame protocol specification
00347 * @see docs/sections/01_nanopb_protocol.tex Communication protocol documentation
00348 *
00349 * @par NASA Power of 10 Compliance:
00350 * - Rule 1: [OK] No goto, setjmp/longjmp, or recursion used
00351 * - Rule 2: [OK] All loops have compile-time bounded iterations (channel count = 2)
00352 * - Rule 3: [OK] Zero dynamic memory allocation (all buffers static)
00353 * - Rule 4: [OK] All functions < 60 lines
00354 * - Rule 5: [OK] Minimum 2 assertions per function via parameter validation
00355 * - Rule 6: [OK] Variables declared at smallest scope
00356 * - Rule 7: [OK] All return values checked and propagated
00357 * - Rule 8: [OK] Uses C23 typed enums exclusively (no magic numbers)
00358 * - Rule 9: [WARN] INTENTIONAL DEVIATION: Uses function pointers for callback (event-driven architecture)
00359 * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror (zero warnings)
00360 *
00361 * @par SOLID Principles:
00362 * - **S (Single Responsibility):** Only channel coordination and frame routing. Protocol parsing delegated to rx_frame,
transport to rx_usb_comm/rx_spi_comm.
00363 * - **O (Open/Closed):** Extensible to new channels by adding enum value and transport handle. Closed for modification
(no changes needed for new frame types).
00364 * - **L (Liskov Substitution):** USB and SPI transports interchangeable via common frame interface. Can substitute
mock transports for testing.
00365 * - **I (Interface Segregation):** Clean separation: lifecycle (init/deinit), polling (poll), sending (send/respond), status
(channel_ready). Users include only what they need.
00366 * - **D (Dependency Inversion):** Depends on abstractions (rx_frame protocol, generic transport interface) not concrete
USB/SPI implementations.
00367 *
00368 * @warning Do NOT call manager functions from within frame callback (recursion risk)
00369 * @warning ASCII buffer is 2048 bytes - ensure payload_len + overhead < 2000 bytes to avoid truncation
00370 * @warning Polling from multiple threads without synchronization will cause corruption
00371 *
00372 * @attention Manager does not own USB/SPI handles - caller must initialize transports before manager
00373 * @attention Callback is invoked from poll() context - keep processing minimal or defer to task queue
00374 * @attention Decoded ASCII output adds ~80 us latency per frame - disable for high-frequency applications
00375 *
00376 * @author Locked, Inc.
00377 * @date 2026-01-27
00378 * @version 1.0.0
00379 * @copyright Copyright (c) 2026 Locked Inc.
00380 * SPDX-License-Identifier: MIT
00381 *
00382 * @par Changelog:
00383 * - 1.0.0 (2026-01-26): Initial implementation with USB and SPI channels
00384 * - 1.1.0 (2026-01-27): Added comprehensive Doxygen documentation, state machine diagrams, performance data
00385 */
00386
00387 #pragma once
00388
00389 #include <stdbool.h>
00390 #include <stdint.h>
00391
00392 #include "rx_err.h"
00393 #include "rx_frame.h"
00394 #include "rx_i2c_comm.h"
00395 #include "rx_spi_comm.h"
00396 #include "rx_spi_link.h"
00397 #include "rx_uart_comm.h"
00398
00399 #ifdef __cplusplus
00400 extern "C" {
00401 #endif
00402
00403 /*
=====
00404 * Constants
00405 *
=====
00406 */
00407
00408 /**
00409 * @enum rx_comm_manager_constants_t
00410 * @brief Communication manager compile-time constants
00411 *
00412 * @details
00413 * Defines buffer sizes and limits for the communication manager. These
00414 * constants are chosen based on typical frame sizes and ASCII formatting
00415 * requirements.
00416 *
00417 * **Design Rationale:**
00418 * - **2048-byte ASCII buffer:** Accommodates maximum frame size (1024 bytes payload)
00419 *   plus formatting overhead (~1000 bytes for hex dump + headers)
00420 * - Buffer is stack-allocated in handle structure (not dynamically allocated)
00421 * - Sufficient for debugging without excessive memory usage
00422 *
00423 * @see rx_comm_manager_t.ascii_buffer

```



```

00424 * @since Version 1.0.0
00425 */
00426 typedef enum : uint16_t {
00427 /**
00428  * @brief ASCII formatting buffer size in bytes
00429  * @details
00430  * Size of internal buffer used to format decoded ASCII output for debug port.
00431  * Must accommodate:
00432  * - Header line: ~80 bytes ('[USB] TYPE=cmd FLAGS=0x00 LEN=1024')
00433  * - Payload hex dump: payload_len * 3 bytes ('XX ` per byte)
00434  * - Footer + null terminator: ~20 bytes
00435  *
00436  * Maximum frame payload: 1024 bytes
00437  * Required buffer: 80 + (1024 * 3) + 20 = 3172 bytes
00438  * Allocated: 2048 bytes (limits payload display to ~650 bytes, which is acceptable)
00439  *
00440  * @par Value: 2048 bytes
00441  * @par Rationale: Balance between functionality and memory usage
00442  */
00443 k_comm_manager_ascii_buf_size = 2048,
00444
00445 /**
00446  * @brief Maximum number of entries in the event FIFO queue
00447  * @details
00448  * Static FIFO queue depth for event (command) frames that require reliable
00449  * delivery. Events are queued and sent with SPI-only retry + exponential
00450  * backoff. Queue is statically allocated (zero dynamic memory).
00451  *
00452  * @par Value: 8 entries
00453  * @par Rationale: Small queue prevents unbounded memory, sufficient for
00454  * typical command burst scenarios
00455  */
00456 k_comm_event_queue_depth = 8,
00457 } rx_comm_manager_constants_t;
00458
00459 /**
00460  * @enum rx_comm_heartbeat_constants_t
00461  * @brief Heartbeat timing constants
00462  *
00463  * @details
00464  * Dual-detection heartbeat model:
00465  * - **Implicit timeout (200ms)**: Any valid frame resets the timer. If no
00466  * frames arrive within 200ms, the link is declared dead.
00467  * - **Explicit PING (1000ms)**: Sent when link is idle for >1s as a probe.
00468  * Firmware primarily receives PINGs from the gateway.
00469  * - **Check interval (50ms)**: How often to evaluate heartbeat status.
00470  *
00471  * @since Version 1.0.0
00472  */
00473 typedef enum : uint32_t {
00474 k_heartbeat_implicit_timeout_ms = 200, /**< Declare link dead if no frame for 200ms */
00475 k_heartbeat_ping_interval_ms = 1000, /**< Send PING probe when idle for 1s */
00476 k_heartbeat_check_interval_ms = 50, /**< Heartbeat evaluation interval (200ms / 4) */
00477 } rx_comm_heartbeat_constants_t;
00478
00479 /**
00480  * @enum rx_comm_event_retry_constants_t
00481  * @brief Event queue retry constants for SPI HARQ
00482  *
00483  * @details
00484  * Events (commands) sent via SPI use retry with exponential backoff.
00485  * USB sends are fire-and-forget (no retries).
00486  *
00487  * @since Version 1.0.0
00488  */
00489 typedef enum : uint16_t {
00490 k_event_max_retries = 3, /**< Maximum SPI retry attempts */
00491 k_event_initial_backoff_ms = 10, /**< Initial backoff delay (ms) */
00492 k_event_max_backoff_ms = 15, /**< Maximum backoff delay (ms): caps 10«(retries-1) at retries=2 */
00493 } rx_comm_event_retry_constants_t;
00494
00495 /*
=====
00496 * Types
00497 *
=====
00498 */
00499
00500 /**
00501  * @enum rx_comm_channel_t
00502  * @brief Communication channel identifiers
00503  *
00504  * @details
00505  * Enumerates the available communication channels. The manager can handle
00506  * multiple channels simultaneously, with each channel operating independently.
00507  *
00508  * **Channel Characteristics:**

```



```

00509 *
00510 * | Channel | Bandwidth | Latency | Setup Complexity | Use Case |
00511 * |-----|-----|-----|-----|
00512 * | USB CDC | ~1 MB/s | ~1-5 ms | Low (plug-and-play) | Commands, telemetry |
00513 * | SPI | ~2 MB/s | ~0.1-1 ms | Medium (requires wiring) | High-speed data, control |
00514 *
00515 * @par Usage Pattern:
00516 * @code{.c}
00517 * // Check which channel frame came from
00518 * void callback(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00519 *     if (channel == k_comm_channel_usb) {
00520 *         // Received via USB
00521 *     } else if (channel == k_comm_channel_spi) {
00522 *         // Received via SPI
00523 *     }
00524 * }
00525 * @endcode
00526 *
00527 * @see rx_comm_manager_channel_name() Get human-readable channel name
00528 * @see rx_comm_frame_callback_t Callback receives channel identifier
00529 * @since Version 1.0.0
00530 */
00531 typedef enum : uint8_t {
00532 /**
00533 * @brief UART channel (SCI9 via CY7C65213 USB-UART bridge)
00534 * @details
00535 * Primary link to the Raspberry Pi 5 gateway. Carries protobuf command /
00536 * response / telemetry frames AND k_frame_type_log_message frames with the
00537 * RX72N runtime log stream, all multiplexed on the same byte stream.
00538 *
00539 * On the Pi5 the bridge enumerates as /dev/ttyACM0 at 115200 baud (CY7C65213 is a CDC-ACM class device; see
00540 * k_uart_default_baudrate in libs/rx_hal/src/uart.c).
00541 * @par Hardware: SCI9 (PB7 / TXD9, PB6 / RXD9) + CY7C65213 bridge IC
00542 * @par Value: 0
00543 */
00544 k_comm_channel_uart = 0,
00545 /**
00546 * @brief SPI peripheral channel
00547 * @details
00548 * Dedicated SPI connection to the ROS2 spi_driver_node for low-latency,
00549 * high-bandwidth motor control. 10 MHz clock, full-duplex.
00550 *
00551 * @par Hardware: RSPI peripheral (COPI / CIPO / SCK / CS)
00552 * @par Value: 1
00553 */
00554 k_comm_channel_spi = 1,
00555 /**
00556 * @brief I2C peripheral channel (RIIC0)
00557 * @details
00558 * I2C connection where RX72N acts as I2C peripheral device and RPi5 is the
00559 * I2C controller. Optional auxiliary channel.
00560 *
00561 * @par Hardware: RIIC0 peripheral (P16 / SCL0, P17 / SDA0)
00562 * @par Value: 2
00563 */
00564 k_comm_channel_i2c = 2,
00565 /**
00566 * @brief Total number of supported channels (3)
00567 *
00568 * @par Value: 3 (UART + SPI + I2C)
00569 * @par Invariant: Must equal number of enum values (excluding this one)
00570 */
00571 k_comm_channel_count = 3,
00572 } rx_comm_channel_t;
00573
00574 /**
00575 * @typedef rx_comm_frame_callback_t
00576 * @brief Frame received callback function type
00577 *
00578 * @details
00579 * User-defined function invoked by communication manager when a complete,
00580 * valid frame is received on any enabled channel. The callback receives:
00581 * - Channel identifier (which channel the frame arrived on)
00582 * - Parsed frame structure (type, flags, payload)
00583 * - User context pointer (passed during initialization)
00584 *
00585 * ## Callback Contract
00586 *
00587 * **Responsibilities:**
00588 * - Parse frame payload based on frame type
00589 * - Handle command execution or data processing
00590 * - Optionally send response via `rx_comm_manager_respond()`
00591 * - Return quickly to avoid blocking communication loop

```

```

00595 *
00596 * **Restrictions:**
00597 * - Do NOT call `rx_comm_manager_poll()` from callback (recursion!)
00598 * - Keep processing minimal (~50 us recommended)
00599 * - For slow operations, queue to task and return immediately
00600 * - Do NOT modify frame structure (it may be const or recycled)
00601 *
00602 * **Thread Safety:**
00603 * - Callback executed in context of `poll()` caller
00604 * - If poll() called from RTOS task, callback runs in that task context
00605 * - Synchronization required if accessing shared resources
00606 *
00607 * @param[in] channel The communication channel the frame was received on
00608 * - Type: rx_comm_channel_t
00609 * - Valid values: k_comm_channel_usb or k_comm_channel_spi
00610 * - Use to determine response routing
00611 *
00612 * @param[in] frame Pointer to the received and validated frame
00613 * - Type: const rx_frame_t*
00614 * - Valid until callback returns
00615 * - Contains: type, flags, payload, payload_len (CRC already validated)
00616 * - Do NOT modify or store pointer (frame may be recycled)
00617 *
00618 * @param[in] ctx User context pointer
00619 * - Type: void* (application-defined)
00620 * - Value passed during `rx_comm_manager_init()`
00621 * - Typically pointer to manager, application state, or nullptr
00622 * - Use to access application resources without globals
00623 *
00624 * @par Usage Example - Basic Command Handler:
00625 * @code{.c}
00626 * void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00627 *     rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
00628 *
00629 *     // Log reception
00630 *     rx_log_info("COMM", "RX from %s: type=0x%02X len=%u",
00631 *         rx_comm_manager_channel_name(channel),
00632 *         frame->type, frame->payload_len);
00633 *
00634 *     // Dispatch based on type
00635 *     switch (frame->type) {
00636 *         case k_frame_type_command:
00637 *             handle_command(mgr, channel, frame);
00638 *             break;
00639 *         case k_frame_type_telemetry_req:
00640 *             send_telemetry(mgr, channel);
00641 *             break;
00642 *         default:
00643 *             rx_log_warn("COMM", "Unknown type: 0x%02X", frame->type);
00644 *             break;
00645 *     }
00646 * }
00647 * @endcode
00648 *
00649 * @par Usage Example - Deferred Processing:
00650 * @code{.c}
00651 * // For time-consuming operations, defer to queue
00652 * void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00653 *     app_state_t* app = (app_state_t*)ctx;
00654 *
00655 *     // Copy frame to queue (frame pointer only valid during callback)
00656 *     command_msg_t msg = {
00657 *         .channel = channel,
00658 *         .type = frame->type,
00659 *         .payload_len = frame->payload_len,
00660 *     };
00661 *     memcpy(msg.payload, frame->payload, frame->payload_len);
00662 *
00663 *     // Queue for processing by command handler task
00664 *     UINT status = tx_queue_send(&app->cmd_queue, &msg, TX_NO_WAIT);
00665 *     if (status != TX_SUCCESS) {
00666 *         rx_log_error("COMM", "Command queue full!");
00667 *     }
00668 * }
00669 * @endcode
00670 *
00671 * @warning Callback must return quickly (<50 us recommended) to avoid blocking communication
00672 * @warning Do NOT call rx_comm_manager_poll() from callback (causes recursion)
00673 * @warning Frame pointer only valid during callback - copy data if needed beyond callback scope
00674 *
00675 * @see rx_comm_manager_respond() Send response on same channel
00676 * @see rx_comm_manager_send() Send frame on specific channel
00677 * @see rx_frame_t Frame structure definition
00678 *
00679 * @since Version 1.0.0
00680 */
00681 typedef void (*rx_comm_frame_callback_t)(rx_comm_channel_t channel,

```

```

00682                                     const rx_frame_t* frame,
00683                                     void*          ctx);
00684
00685 /**
00686  * @enum rx_comm_link_status_t
00687  * @brief Link health status from heartbeat monitoring
00688  *
00689  * @details
00690  * Reported per-transport link health. Determined by the dual-detection
00691  * heartbeat model (implicit 200ms timeout + explicit 1s PING).
00692  *
00693  * @since Version 1.0.0
00694  */
00695 typedef enum : uint8_t {
00696     k_link_status_unknown = 0, /**< No frames received yet */
00697     k_link_status_healthy = 1, /**< Frames received within implicit timeout */
00698     k_link_status_dead    = 2, /**< No frames within implicit timeout (200ms) */
00699 } rx_comm_link_status_t;
00700
00701 /**
00702  * @struct rx_comm_event_entry_t
00703  * @brief Single entry in the static event FIFO queue
00704  *
00705  * @details
00706  * Events (commands) that require reliable delivery are queued here.
00707  * Each entry contains a copy of the payload and metadata needed for
00708  * retry logic. SPI entries retry with exponential backoff; USB entries
00709  * are fire-and-forget.
00710  *
00711  * @since Version 1.0.0
00712  */
00713 typedef struct {
00714     rx_comm_channel_t channel;          /**< Target transport */
00715     rx_frame_type_t  type;             /**< Frame type */
00716     uint8_t          flags;            /**< Frame flags */
00717     uint8_t          payload[k_frame_max_payload]; /**< Payload copy */
00718     uint32_t          payload_len;     /**< Payload length (bytes) */
00719     uint8_t          retries;         /**< Retry attempts so far */
00720     uint32_t          next_retry_time_ms; /**< Earliest time (ms) for next retry */
00721     bool             occupied;        /**< Slot in use */
00722 } rx_comm_event_entry_t;
00723
00724 /**
00725  * @struct rx_comm_heartbeat_state_t
00726  * @brief Per-transport heartbeat tracking state
00727  *
00728  * @details
00729  * Tracks the last time a valid frame was received on each transport.
00730  * Used by the poll loop to evaluate link health per the dual-detection
00731  * heartbeat model.
00732  *
00733  * @since Version 1.0.0
00734  */
00735 typedef struct {
00736     uint32_t          last_rx_ms; /**< Timestamp (ms) of last valid frame */
00737     rx_comm_link_status_t status; /**< Current link health */
00738 } rx_comm_heartbeat_state_t;
00739
00740 /**
00741  * @typedef rx_comm_link_status_callback_t
00742  * @brief Callback invoked when a link status changes
00743  *
00744  * @param[in] channel Which transport changed status
00745  * @param[in] new_status New link status
00746  * @param[in] ctx User context pointer
00747  *
00748  * @since Version 1.0.0
00749  */
00750 typedef void (*rx_comm_link_status_callback_t)(rx_comm_channel_t channel,
00751                                               rx_comm_link_status_t new_status,
00752                                               void*          ctx);
00753
00754 /**
00755  * @enum rx_comm_channel_mask_t
00756  * @brief Bitmask selecting which comm channels to initialize and use
00757  *
00758  * @details
00759  * Each bit corresponds to one comm channel. Bit position matches the numeric
00760  * value of the corresponding rx_comm_channel_t enumerator so that a single
00761  * shift converts between the two representations.
00762  *
00763  * Pass a bitmask in rx_comm_manager_config_t::enabled_channels to skip
00764  * initialization of channels that are not needed (e.g. exclude UART when the
00765  * log backend also uses SCI9).
00766  *
00767  * @see rx_comm_channel_t Channel enumerator
00768  * @see rx_comm_manager_config_t::enabled_channels Usage

```

```

00769 * @since Version 1.0.0
00770 */
00771 typedef enum : uint8_t {
00772     k_comm_channel_mask_none = 0x00U, /**< No channels enabled */
00773     k_comm_channel_mask_uart = 0x01U, /**< UART (k_comm_channel_uart = 0) */
00774     k_comm_channel_mask_spi = 0x02U, /**< SPI (k_comm_channel_spi = 1) */
00775     k_comm_channel_mask_i2c = 0x04U, /**< I2C (k_comm_channel_i2c = 2) */
00776     k_comm_channel_mask_all = /**< All three channels */
00777     (k_comm_channel_mask_uart | k_comm_channel_mask_spi | k_comm_channel_mask_i2c),
00778 } rx_comm_channel_mask_t;
00779
00780 /**
00781  * @struct rx_comm_manager_config_t
00782  * @brief Communication manager configuration
00783  *
00784  * @details
00785  * Configuration structure passed to `rx_comm_manager_init()`. Defines which
00786  * channels are enabled, callback function, and optional features.
00787  *
00788  * **Configuration Guidelines:**
00789  * - At least one channel (usb_handle OR spi_handle) should be non-NULL
00790  * - Both channels can be enabled simultaneously
00791  * - Callback is optional (NULL = receive but don't notify)
00792  * - Decoded output recommended for development, disable for production
00793  *
00794  * **Memory Lifetime:**
00795  * - Config structure only read during init, can be stack-allocated
00796  * - Transport handles must remain valid for manager lifetime
00797  * - Callback function must remain valid (typically static function)
00798  *
00799  * @par Configuration Example:
00800  * @code{.c}
00801  * rx_comm_manager_config_t cfg = {
00802  *     .usb_handle = &usb_comm,           // Enable USB channel
00803  *     .spi_handle = &spi_comm,           // Enable SPI channel
00804  *     .callback = on_frame_received,      // Frame handler
00805  *     .callback_ctx = &app_state,       // User context
00806  *     .enable_decoded_output = true,     // ASCII debug output
00807  * };
00808  * @endcode
00809  *
00810  * @see rx_comm_manager_init() Initialization function
00811  * @since Version 1.0.0
00812  */
00813 typedef struct {
00814     /**
00815      * @brief SPI communication handle (NULL to disable SPI channel)
00816      * @details
00817      * Pointer to initialized SPI transport handle. If NULL, SPI channel
00818      * is disabled and all SPI operations are skipped.
00819      *
00820      * **Requirements:**
00821      * - Must be initialized via `rx_spi_comm_init()` before passing to manager
00822      * - Must remain valid for lifetime of manager
00823      * - Requires hardware SPI connection between RX72N and RPi5
00824      *
00825      * **Null handling:**
00826      * - NULL = SPI channel disabled (valid configuration)
00827      * - Non-NULL = SPI channel enabled
00828      *
00829      * @par Type: rx_spi_comm_handle_t* (pointer to SPI transport handle)
00830      * @par Valid values: Non-NULL initialized handle, or nullptr to disable
00831      * @par Lifetime: Must outlive manager instance
00832      */
00833     rx_spi_comm_handle_t* spi_handle;
00834
00835     /**
00836      * @brief I2C communication handle (NULL to disable I2C channel)
00837      * @details
00838      * Pointer to initialized I2C peripheral transport handle. If NULL, I2C channel
00839      * is disabled and all I2C operations are skipped. The RX72N acts as I2C
00840      * peripheral (device) and the RPi5 acts as I2C controller.
00841      *
00842      * @par Type: rx_i2c_comm_handle_t* (pointer to I2C transport handle)
00843      * @par Valid values: Non-NULL initialized handle, or nullptr to disable
00844      * @par Lifetime: Must outlive manager instance
00845      */
00846     rx_i2c_comm_handle_t* i2c_handle;
00847
00848     /**
00849      * @brief UART communication handle (NULL to disable UART channel)
00850      * @details
00851      * Pointer to initialized UART (SCI9) transport handle. If NULL, UART channel
00852      * is disabled and all UART operations are skipped.
00853      *
00854      * @par Type: rx_uart_comm_handle_t* (pointer to UART transport handle)
00855      * @par Valid values: Non-NULL initialized handle, or nullptr to disable

```

```

00856  * @par Lifetime: Must outlive manager instance
00857  */
00858  rx_uart_comm_handle_t* uart_handle;
00859
00860  /**
00861   * @brief Optional SPI link layer with HARQ (NULL to use raw SPI)
00862   * @details
00863   * When non-NULL, SPI send/receive operations route through the link
00864   * layer, which provides FEC encoding/decoding, Chase Combining, and
00865   * ACK/NACK handshake. When NULL, SPI operates in raw mode (frame +
00866   * CRC-32 only, no FEC/HARQ).
00867   *
00868   * **Requirements:**
00869   * - Must be initialized via rx_spi_link_init() before passing to manager
00870   * - spi_link->spi_handle MUST be the same as config->spi_handle
00871   * - Must remain valid for lifetime of manager
00872   *
00873   * @invariant If spi_link is non-NULL: spi_link->spi_handle == config->spi_handle
00874   * @invariant If spi_link is non-NULL: spi_link->initialized == true
00875   * @invariant If spi_link is non-NULL: spi_link must outlive manager instance
00876   *
00877   * @par Valid values: Initialized rx_spi_link_t*, or nullptr for raw SPI
00878   * @par Lifetime: Must outlive manager instance
00879   *
00880   * @see rx_spi_link.h SPI link layer API
00881   * @see rx_spi_link_init() Must be called before passing spi_link to manager
00882   * @since Version 1.0.0
00883   */
00884  rx_spi_link_t* spi_link;
00885
00886  /**
00887   * @brief Frame received callback function (NULL for no callback)
00888   * @details
00889   * User-defined function invoked when complete frame received on any channel.
00890   * If NULL, frames are received but no notification occurs (not typical).
00891   *
00892   * **Null handling:**
00893   * - NULL = Receive frames but don't notify (unusual, for testing only)
00894   * - Non-NULL = Invoke callback on frame reception (normal operation)
00895   *
00896   * @par Type: rx_comm_frame_callback_t (function pointer)
00897   * @par Valid values: Non-NULL function pointer, or nullptr to disable
00898   * @par Signature: `void callback(rx_comm_channel_t, const rx_frame_t*, void*)`
00899   *
00900   * @see rx_comm_frame_callback_t Callback function type definition
00901   */
00902  rx_comm_frame_callback_t callback;
00903
00904  /**
00905   * @brief User context pointer passed to callback
00906   * @details
00907   * Arbitrary pointer passed to callback as third parameter. Typically used
00908   * to pass application state, manager pointer, or other context without
00909   * requiring global variables.
00910   *
00911   * **Common patterns:**
00912   * - Pointer to manager itself (enables response from callback)
00913   * - Pointer to application state structure
00914   * - NULL if no context needed
00915   *
00916   * @par Type: void* (application-defined)
00917   * @par Valid values: Any pointer value, or nullptr
00918   * @par Lifetime: Must remain valid if callback uses it
00919   */
00920  void* callback_ctx;
00921
00922  /**
00923   * @brief Enable decoded ASCII output to USB Port 1
00924   * @details
00925   * When true, all received and sent frames are converted to human-readable
00926   * ASCII format and sent to USB Port 1 (/dev/ttyACM1 on RPi5).
00927   *
00928   * **ASCII Format:**
00929   * ```
00930   * [USB] TYPE=cmd FLAGS=0x00 LEN=42 DATA=48656C6C6F...
00931   * [SPI] TYPE=resp FLAGS=0x01 LEN=128 DATA=...
00932   * ```
00933   *
00934   * **Performance Impact:**
00935   * - Adds ~80 us latency per frame (formatting + USB send)
00936   * - Negligible for command/response (< 100 Hz)
00937   * - May impact high-frequency telemetry (> 1 kHz)
00938   *
00939   * **Use Cases:**
00940   * - Development: Enable for debugging and protocol analysis
00941   * - Production: Disable for maximum performance
00942   */

```

```

00943 * @par Type: bool
00944 * @par Value: true = enable ASCII output, false = disable
00945 * @par Default: Recommend true for development, false for production
00946 */
00947 bool enable_decoded_output;
00948
00949 /**
00950 * @brief Callback invoked when a transport link status changes
00951 * @details
00952 * Optional. Called when heartbeat monitoring detects a link transition
00953 * (e.g., healthy -> dead). NULL to disable link status notifications.
00954 */
00955 rx_comm_link_status_callback_t link_status_cb;
00956
00957 /**
00958 * @brief User context for link_status_cb
00959 */
00960 void* link_status_ctx;
00961
00962 /**
00963 * @brief Bitmask of channels to initialize and poll (default: all channels)
00964 * @details
00965 * When a bit is clear the corresponding transport is skipped during
00966 * internal_init_transports() and its handle is left NULL. Set to
00967 * k_comm_channel_mask_all to attempt all four transports (legacy behaviour).
00968 *
00969 * Use comm_task_apply_system_config() to set this alongside the log backend
00970 * so that SCI9 conflicts between UART comm and UART log are caught early.
00971 *
00972 * @par Valid values: Any combination of rx_comm_channel_mask_t bits
00973 * @par Default: k_comm_channel_mask_all (attempt all channels)
00974 * @see rx_comm_channel_mask_t Bit definitions
00975 * @since Version 1.0.0
00976 */
00977 rx_comm_channel_mask_t enabled_channels;
00978 } rx_comm_manager_config_t;
00979
00980 /**
00981 * @struct rx_comm_manager_t
00982 * @brief Communication manager handle
00983 *
00984 * @details
00985 * Opaque handle structure containing manager state and resources. Application
00986 * must allocate this structure (typically static or global) and pass to all
00987 * manager functions.
00988 *
00989 * **Initialization:**
00990 * Zero-initialize structure before passing to `rx_comm_manager_init()`:
00991 * @code{.c}
00992 * rx_comm_manager_t mgr = {}; // Or use memset
00993 * @endcode
00994 *
00995 * **Memory Layout:** See file-level documentation for detailed breakdown.
00996 *
00997 * **Thread Safety:** Not thread-safe - protect with external mutex if accessed
00998 * from multiple threads.
00999 *
01000 * @warning Do not access fields directly - use API functions only
01001 * @warning Structure size ~2.5 KB - consider allocation carefully
01002 *
01003 * @see rx_comm_manager_init() Initialize handle
01004 * @see rx_comm_manager_deinit() Cleanup handle
01005 * @since Version 1.0.0
01006 */
01007 typedef struct {
01008     rx_uart_comm_handle_t*
01009         uart_handle; /**< UART (SCI9 via CY7C65213 bridge) comm handle -- primary */
01010     rx_spi_comm_handle_t* spi_handle; /**< SPI comm handle (NULL disables SPI channel) */
01011     rx_i2c_comm_handle_t* i2c_handle; /**< I2C peripheral comm handle (NULL disables I2C channel) */
01012
01013     /**< @brief Optional SPI link layer with HARQ (NULL if disabled)
01014     * @details
01015     * When non-NULL, points to an rx_spi_link_t handle providing FEC
01016     * encoding/decoding, Chase Combining, and ACK/NACK handshake for SPI.
01017     * When NULL, SPI operates in raw mode (frame + CRC-32 only, no FEC/HARQ).
01018     *
01019     * **Ownership and Lifetime:**
01020     * - Pointer copied from config during rx_comm_manager_init()
01021     * - Manager does NOT own the link object - caller must allocate/deallocate
01022     * - Link object must remain valid for entire manager lifetime
01023     * - Link object must NOT be freed until after rx_comm_manager_deinit()
01024     * - If manager is deinitialized, spi_link remains valid but decoupled
01025     *
01026     * **Invariants/Constraints:**
01027     * - If non-NULL: Must point to initialized rx_spi_link_t (spi_link->initialized == true)
01028     * - If non-NULL: spi_link->spi_handle MUST equal manager's spi_handle
01029     * - If non-NULL: spi_link must outlive manager instance

```

```

01030 * - NULL is allowed to indicate no SPI HARQ (raw SPI mode)
01031 * - Fields inside spi_link object must remain stable while manager is active
01032 *
01033 * **Valid Ranges/Usage:**
01034 * - Set via config->spi_link during initialization
01035 * - NULL = raw SPI mode (valid configuration for testing or simple use cases)
01036 * - Non-NULL = HARQ mode (production mode with FEC and retransmission)
01037 * - Once set during init, this pointer is immutable (never modified by manager)
01038 *
01039 * **Thread Safety:**
01040 * - Manager does not perform synchronization on spi_link access
01041 * - If multiple threads call rx_comm_manager_poll(), caller must serialize
01042 * - spi_link internal state is protected by its own mechanisms (see rx_spi_link.h)
01043 * - Safe to read this pointer from multiple threads (read-only after init)
01044 *
01045 * @invariant If spi_link != NULL: spi_link->initialized == true
01046 * @invariant If spi_link != NULL: spi_link->spi_handle == this->spi_handle
01047 * @invariant If spi_link != NULL: spi_link must outlive this manager instance
01048 * @invariant Pointer value never changes after rx_comm_manager_init()
01049 *
01050 * @par Type: rx_spi_link_t* (pointer to HARQ link layer handle)
01051 * @par Valid values: Initialized rx_spi_link_t*, or nullptr for raw SPI mode
01052 * @par Lifetime: Must outlive manager instance (caller manages allocation)
01053 * @par Modification: Read-only after init (immutable pointer)
01054 * - Immutable after init (do not change while manager is active)
01055 * - Fields inside rx_spi_link_t must remain stable during use
01056 *
01057 * @see rx_spi_link.h SPI link layer API
01058 * @see rx_comm_manager_config_t::spi_link Configuration parameter
01059 * @since Version 1.0.0
01060 */
01061 rx_spi_link_t* spi_link;
01062
01063 rx_comm_frame_callback_t callback;                                /**< Frame callback */
01064 void* callback_ctx;                                              /**< Callback context */
01065 char ascii_buffer[k_comm_manager_ascii_buf_size];              /**< ASCII buffer */
01066 bool enable_decoded_output;                                     /**< Enable ASCII output */
01067 bool initialized;                                              /**< Initialization flag */
01068
01069 /** @brief Per-transport heartbeat tracking (indexed by rx_comm_channel_t) */
01070 rx_comm_heartbeat_state_t heartbeat[k_comm_channel_count];
01071
01072 /** @brief Link status change callback (optional) */
01073 rx_comm_link_status_callback_t link_status_cb;
01074
01075 /** @brief Link status callback context */
01076 void* link_status_ctx;
01077
01078 /** @brief Static event FIFO queue for reliable command delivery */
01079 rx_comm_event_entry_t event_queue[k_comm_event_queue_depth];
01080
01081 /** @brief Event queue write index (next slot to write) */
01082 uint8_t event_queue_head;
01083
01084 /** @brief Event queue read index (next slot to process) */
01085 uint8_t event_queue_tail;
01086
01087 /** @brief Number of occupied event queue slots */
01088 uint8_t event_queue_count;
01089 } rx_comm_manager_t;
01090
01091 /**
01092  * @struct rx_comm_send_params_t
01093  * @brief Parameters for rx_comm_manager_send
01094  *
01095  * @details
01096  * Wrapper structure providing type safety for send operations. Prevents
01097  * parameter confusion when passing multiple uint8_t/uint32_t values.
01098  *
01099  * **Design Rationale:**
01100  * Without this struct, function signature would be error-prone:
01101  * ```c
01102  * // BAD: Easy to mix up channel, type, flags (all similar types)
01103  * rx_err_t send(mgr, uint8_t ch, uint8_t type, uint8_t flags, ...);
01104  * ```
01105  *
01106  * With struct, parameters are self-documenting:
01107  * ```c
01108  * // GOOD: Clear naming prevents mistakes
01109  * rx_comm_send_params_t params = {
01110  *     .channel = k_comm_channel_usb,
01111  *     .type = k_frame_type_command,
01112  *     .flags = 0x00,
01113  *     // ...
01114  * };
01115  * ```
01116  */

```



```

01117 * @par Usage Example:
01118 * @code{.c}
01119 * rx_comm_send_params_t params = {
01120 *     .channel = k_comm_channel_spi,
01121 *     .type = k_frame_type_telemetry,
01122 *     .flags = 0,
01123 *     .payload = telemetry_data,
01124 *     .payload_len = sizeof(telemetry_data),
01125 * };
01126 * rx_comm_manager_send(&mgr, &params);
01127 * @endcode
01128 *
01129 * @see rx_comm_manager_send() Send function using this structure
01130 * @since Version 1.0.0
01131 */
01132 typedef struct {
01133     rx_comm_channel_t channel; /**< Channel to send on (USB or SPI) */
01134     rx_frame_type_t type; /**< Frame type (command, telemetry, etc.) */
01135     uint8_t flags; /**< Frame flags (protocol-specific) */
01136     const uint8_t* payload; /**< Payload data pointer */
01137     uint32_t payload_len; /**< Payload length in bytes */
01138 } rx_comm_send_params_t;
01139
01140 /*
=====
01141 * Lifecycle API
01142 *
=====
01143 */
01144 /**
01145 * @brief Initialize communication manager
01146 *
01147 * @details
01148 * Initializes the communication manager with the provided configuration.
01149 * Validates configuration parameters and sets up internal state for
01150 * USB and/or SPI channel operation.
01151 *
01152 * @param[out] mgr Pointer to manager handle
01153 * @param[in] cfg Configuration (NULL for defaults, both channels disabled)
01154 *
01155 * @return k_rx_ok on success
01156 * @return k_rx_err_invalid_arg if mgr is nullptr
01157 * @return k_rx_err_invalid_arg if spi_link is non-NULL but not initialized
01158 * @return k_rx_err_invalid_arg if spi_link->spi_handle != cfg->spi_handle
01159 *
01160 * @pre mgr must not be nullptr
01161 * @pre If cfg->spi_link is non-NULL, spi_link must be initialized via rx_spi_link_init()
01162 * @pre If cfg->spi_link is non-NULL, spi_link->spi_handle must equal cfg->spi_handle
01163 * @pre cfg may be nullptr (results in default configuration with both channels disabled)
01164 *
01165 * @post mgr->usb_handle == cfg->usb_handle (or nullptr if cfg is nullptr)
01166 * @post mgr->spi_handle == cfg->spi_handle (or nullptr if cfg is nullptr)
01167 * @post mgr->spi_link == cfg->spi_link (or nullptr if cfg is nullptr or cfg->spi_link is nullptr)
01168 * @post All internal state initialized (event queue empty, heartbeat timers reset)
01169 *
01170 * @note Thread-safe: May be called from any thread, but concurrent calls with same mgr are prohibited
01171 * @warning Must be called before any other manager operations on this handle
01172 *
01173 * @see rx_spi_link_init() Must call this before passing spi_link to manager
01174 * @see rx_comm_manager_deinit() Cleanup function
01175 *
01176 * @since Version 1.0.0
01177 */
01178 [[nodiscard]] rx_err_t rx_comm_manager_init(rx_comm_manager_t* mgr,
01179                                             const rx_comm_manager_config_t* cfg);
01180
01181 /**
01182 * @brief Deinitialize communication manager
01183 *
01184 * Does not deinitialize the underlying USB/SPI handles.
01185 *
01186 * @param[in,out] mgr Pointer to manager handle
01187 *
01188 * @return k_rx_ok on success
01189 */
01190 [[nodiscard]] rx_err_t rx_comm_manager_deinit(rx_comm_manager_t* mgr);
01191
01192 /*
=====
01193 * Polling API
01194 *
=====
01195 */
01196 /**
01197 * @brief Poll all enabled channels for incoming frames
01198 *
01199 */

```



```

01200 * Non-blocking check of USB and SPI channels. For each complete frame received:
01201 * 1. If decoded output enabled, format and send ASCII to Port 1
01202 * 2. Invoke user callback with frame data and channel
01203 *
01204 * Call this from main loop or dedicated communication task.
01205 *
01206 * @param[in,out] mgr Pointer to initialized manager
01207 *
01208 * @return k_rx_ok if at least one frame was received
01209 * @return k_rx_err_timeout if no frames available on any channel
01210 * @return k_rx_err_invalid_arg if mgr is nullptr
01211 * @return k_rx_err_invalid_state if not initialized
01212 */
01213 [[nodiscard]] rx_err_t rx_comm_manager_poll(rx_comm_manager_t* mgr);
01214
01215 /*
=====
01216 * Send API
01217 *
=====
01218 */
01219
01220 /**
01221 * @brief Send frame on specific channel
01222 *
01223 * Encodes and sends a frame on the specified channel. If decoded output
01224 * is enabled, also sends ASCII representation to Port 1.
01225 *
01226 * @param[in,out] mgr Pointer to initialized manager
01227 * @param[in] params Send parameters (channel, type, flags, payload, payload_len)
01228 *
01229 * @return k_rx_ok on success
01230 * @return k_rx_err_invalid_arg if mgr or payload is nullptr
01231 * @return k_rx_err_invalid_state if not initialized or channel not enabled
01232 */
01233 [[nodiscard]] rx_err_t rx_comm_manager_send(rx_comm_manager_t* mgr,
01234                                             const rx_comm_send_params_t* params);
01235
01236 /**
01237 * @brief Send response on same channel as received frame
01238 *
01239 * Convenience function for responding to commands. Sends a RESPONSE type
01240 * frame on the specified channel.
01241 *
01242 * @param[in,out] mgr Pointer to initialized manager
01243 * @param[in] channel Channel to respond on (from callback)
01244 * @param[in] payload Response payload data
01245 * @param[in] payload_len Response payload length
01246 *
01247 * @return k_rx_ok on success
01248 */
01249 [[nodiscard]] rx_err_t rx_comm_manager_respond(rx_comm_manager_t* mgr,
01250                                                rx_comm_channel_t channel,
01251                                                const uint8_t* payload,
01252                                                uint32_t payload_len);
01253
01254 /*
=====
01255 * Stream / Event Send API
01256 *
=====
01257 */
01258
01259 /**
01260 * @brief Send data on the stream path (fire-and-forget)
01261 *
01262 * @details
01263 * Stream path is for time-sensitive data like telemetry where stale data
01264 * has no value. Sends immediately with NO retries on any transport.
01265 * If the send fails, the data is simply dropped.
01266 *
01267 * - USB CDC: Direct send, no retries (USB HW provides reliability)
01268 * - SPI: Direct send, no retries (stale telemetry is worthless)
01269 *
01270 * @param[in,out] mgr Pointer to initialized manager
01271 * @param[in] params Send parameters (channel, type, flags, payload, payload_len)
01272 *
01273 * @return k_rx_ok on success
01274 * @return k_rx_err_invalid_arg if mgr or params is nullptr
01275 * @return k_rx_err_invalid_state if not initialized or channel not enabled
01276 *
01277 * @pre mgr must be initialized
01278 * @pre params must be valid
01279 * @post Data sent or silently dropped on failure
01280
01281 * @note This is a thin wrapper over rx_comm_manager_send() for semantic clarity
01282 */

```

```

01283 * @see rx_comm_manager_event_send() For reliable command delivery
01284 * @since Version 1.0.0
01285 */
01286 [[nodiscard]] rx_err_t rx_comm_manager_stream_send(rx_comm_manager_t* mgr,
01287                                                     const rx_comm_send_params_t* params);
01288
01289 /**
01290 * @brief Queue data on the event path (reliable, SPI retry)
01291 *
01292 * @details
01293 * Event path is for commands and other data requiring reliable delivery.
01294 * The payload is copied into a static FIFO queue and processed by the
01295 * poll loop. Retry behavior depends on transport:
01296 *
01297 * - **SPI**: Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
01298 * - **USB**: Fire-and-forget (USB HW reliability is sufficient)
01299 *
01300 * Queue is processed in FIFO order by rx_comm_manager_poll().
01301 *
01302 * @param[in,out] mgr Pointer to initialized manager
01303 * @param[in] params Send parameters (channel, type, flags, payload, payload_len)
01304 *
01305 * @return k_rx_ok on success (queued)
01306 * @return k_rx_err_invalid_arg if mgr or params is nullptr
01307 * @return k_rx_err_invalid_state if not initialized
01308 * @return k_rx_err_no_mem if event queue is full
01309 *
01310 * @pre mgr must be initialized
01311 * @pre params must be valid
01312 * @pre params->payload_len <= k_frame_max_payload
01313 * @post Payload copied to queue, will be sent by next poll()
01314 *
01315 * @note Payload is COPIED into queue - caller can reuse buffer immediately
01316 * @warning Queue depth is k_comm_event_queue_depth (8). If full, returns error.
01317 *
01318 * @see rx_comm_manager_stream_send() For fire-and-forget telemetry
01319 * @since Version 1.0.0
01320 */
01321 [[nodiscard]] rx_err_t rx_comm_manager_event_send(rx_comm_manager_t* mgr,
01322                                                    const rx_comm_send_params_t* params);
01323
01324 /*
=====
01325 * Heartbeat API
01326 *
=====
01327 */
01328
01329 /**
01330 * @brief Query link health status for a transport channel
01331 *
01332 * @details
01333 * Returns the current heartbeat-derived link status for the given channel.
01334 * Status is updated by rx_comm_manager_poll() based on the dual-detection
01335 * heartbeat model (200ms implicit timeout).
01336 *
01337 * @param[in] mgr Pointer to initialized manager
01338 * @param[in] channel Channel to query
01339 * @param[out] status Receives current link status
01340 *
01341 * @return k_rx_ok on success
01342 * @return k_rx_err_invalid_arg if mgr or status is nullptr
01343 *
01344 * @since Version 1.0.0
01345 */
01346 [[nodiscard]] rx_err_t rx_comm_manager_link_status(const rx_comm_manager_t* mgr,
01347                                                     rx_comm_channel_t channel,
01348                                                     rx_comm_link_status_t* status);
01349
01350 /*
=====
01351 * Status API
01352 *
=====
01353 */
01354
01355 /**
01356 * @brief Check if channel is ready for communication
01357 *
01358 * @param[in] mgr Pointer to initialized manager
01359 * @param[in] channel Channel to check
01360 * @param[out] ready Set to true if channel is ready
01361 *
01362 * @return k_rx_ok on success
01363 * @return k_rx_err_invalid_arg if mgr or ready is nullptr
01364 */
01365 [[nodiscard]] rx_err_t

```

```

01366 rx_comm_manager_channel_ready(rx_comm_manager_t* mgr, rx_comm_channel_t channel, bool* ready);
01367
01368 /**
01369  * @brief Get channel name string
01370  *
01371  * @param[in] channel Channel identifier
01372  * @return Human-readable channel name
01373  */
01374 const char* rx_comm_manager_channel_name(rx_comm_channel_t channel);
01375
01376 /*
=====
01377 * Retransmission Pass-Through
01378 *
=====
01379 */
01380
01381 /**
01382  * @brief Process pending retransmissions on SPI channel
01383  *
01384  * @details
01385  * Thin pass-through to rx_spi_comm_process_retransmits() on the SPI handle.
01386  * Call this from the communication task polling loop.
01387  *
01388  * @param[in,out] mgr Communication manager
01389  * @param[in] current_time_ms Current system time in milliseconds
01390  *
01391  * @return rx_err_t Error code from rx_spi_comm_process_retransmits()
01392  * @retval k_rx_ok No action needed or retransmit sent
01393  * @retval k_rx_err_invalid_arg mgr is nullptr
01394  * @retval k_rx_err_invalid_state mgr not initialized
01395  * @retval k_rx_err_retry_limit Max retries exceeded
01396  *
01397  * @since Version 1.0.0
01398  */
01399 [[nodiscard]] rx_err_t rx_comm_manager_process_retransmits(rx_comm_manager_t* mgr,
01400                                                             uint32_t          current_time_ms);
01401
01402 /**
01403  * @brief Enable or disable automatic retransmission on SPI channel
01404  *
01405  * @details
01406  * Thin pass-through to rx_spi_comm_set_auto_retransmit() on the SPI handle.
01407  *
01408  * @param[in,out] mgr Communication manager
01409  * @param[in] enabled true to enable, false to disable
01410  * @param[in] config Retransmit config (NULL to keep current/defaults)
01411  *
01412  * @return rx_err_t Error code from rx_spi_comm_set_auto_retransmit()
01413  *
01414  * @since Version 1.0.0
01415  */
01416 [[nodiscard]] rx_err_t
01417 rx_comm_manager_set_auto_retransmit(rx_comm_manager_t* mgr,
01418                                     bool              enabled,
01419                                     const rx_spi_comm_retransmit_config_t* config);
01420
01421 #ifdef __cplusplus
01422 }
01423 #endif

```

## 4.3 libs/rx'comm'manager/src/rx'comm'manager.c File Reference

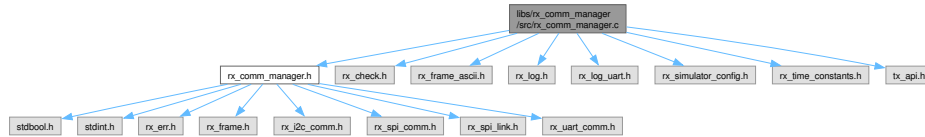
Unified Multi-Channel Communication Manager Implementation.

```

#include "rx_comm_manager.h"
#include "rx_check.h"
#include "rx_frame_ascii.h"
#include "rx_log.h"
#include "rx_log_uart.h"
#include "rx_simulator_config.h"
#include "rx_time_constants.h"
#include "tx_api.h"

```

Include dependency graph for rx\_comm\_manager.c:



## Enumerations

- enum `rx_comm_sim_time_t` : `uint32_t` { `k_sim_time_ms_zero` = 0 }  
Get current time in milliseconds.
- enum `internal_constants_t` : `uint32_t` {  
`k_receive_timeout_ms` = 0 , `k_frame_seq_placeholder` = 0 , `k_frame_crc_placeholder` = 0 ,  
`k_ascii_buffer_first_idx` = 0 ,  
`k_zero_u32` = 0 }  
Internal constants for communication manager implementation.

## Functions

- static `uint32_t` `internal_get_time_ms` (`void`)
- `RX_STATIC_TESTABLE` void `internal_output_decoded` (`rx_comm_manager_t` \*mgr, const `rx_frame_t` \*frame, bool is\_tx)  
Output decoded ASCII frame to USB Port 1 for debugging.
- static void `internal_output_decoded_from_params` (`rx_comm_manager_t` \*mgr, const `rx_comm_send_params_t` \*params)  
Build temporary frame from send params and output decoded ASCII (TX direction).
- `RX_STATIC_TESTABLE` void `internal_handle_frame` (`rx_comm_manager_t` \*mgr, `rx_comm_channel_t` channel, const `rx_frame_t` \*frame)  
Handle received frame by invoking user callback and optional ASCII output.
- static `rx_err_t` `internal_poll_spi` (`rx_comm_manager_t` \*mgr)  
Poll SPI channel for incoming frames (non-blocking).
- static `rx_err_t` `internal_poll_i2c` (`rx_comm_manager_t` \*mgr)  
Poll I2C channel for incoming frames (non-blocking).
- static `rx_err_t` `internal_poll_uart` (`rx_comm_manager_t` \*mgr)  
Poll UART channel for incoming frames (non-blocking).
- static void `internal_dequeue_event` (`rx_comm_manager_t` \*mgr, `rx_comm_event_entry_t` \*entry)  
Process one event from the FIFO queue.
- static void `internal_process_event_queue` (`rx_comm_manager_t` \*mgr)
- static void `internal_check_channel_heartbeat` (`rx_comm_manager_t` \*mgr, `uint8_t` ch, `uint32_t` now\_ms)  
Check heartbeat implicit timeout on all transports.
- static void `internal_check_heartbeat` (`rx_comm_manager_t` \*mgr)
- `rx_err_t` `rx_comm_manager_init` (`rx_comm_manager_t` \*mgr, const `rx_comm_manager_config_t` \*cfg)  
Initialize communication manager with channel configuration.
- `rx_err_t` `rx_comm_manager_deinit` (`rx_comm_manager_t` \*mgr)  
Deinitialize communication manager and release resources.
- static `rx_err_t` `internal_poll_all_channels` (`rx_comm_manager_t` \*mgr, bool \*received)  
Poll all enabled communication channels for incoming frames (non-blocking).
- `rx_err_t` `rx_comm_manager_poll` (`rx_comm_manager_t` \*mgr)  
Poll all enabled channels for incoming frames.

- static rx\_err\_t `internal_route_send`(rx\_comm\_manager\_t \*mgr, const rx\_comm\_send\_params\_t \*params)  
Send frame on specified communication channel.
- rx\_err\_t `rx_comm_manager_send`(rx\_comm\_manager\_t \*mgr, const rx\_comm\_send\_params\_t \*params)  
Send frame on specific channel.
- rx\_err\_t `rx_comm_manager_respond`(rx\_comm\_manager\_t \*mgr, rx\_comm\_channel\_t channel, const uint8\_t \*payload, uint32\_t payload\_len)  
Send response frame on specified channel (convenience wrapper).
- rx\_err\_t `rx_comm_manager_stream_send`(rx\_comm\_manager\_t \*mgr, const rx\_comm\_send\_params\_t \*params)  
Send data on the stream path (fire-and-forget).
- rx\_err\_t `rx_comm_manager_event_send`(rx\_comm\_manager\_t \*mgr, const rx\_comm\_send\_params\_t \*params)  
Queue data on the event path (reliable, SPI retry).
- rx\_err\_t `rx_comm_manager_link_status`(const rx\_comm\_manager\_t \*mgr, rx\_comm\_channel\_t channel, rx\_comm\_link\_status\_t \*status)  
Query link health status for a transport channel.
- rx\_err\_t `rx_comm_manager_channel_ready`(rx\_comm\_manager\_t \*mgr, rx\_comm\_channel\_t channel, bool \*ready)  
Query if communication channel is ready for send/receive operations.
- const char \* `rx_comm_manager_channel_name`(rx\_comm\_channel\_t channel)  
Get human-readable name for communication channel.
- rx\_err\_t `rx_comm_manager_process_retransmits`(rx\_comm\_manager\_t \*mgr, const uint32\_t current\_time\_ms)  
Process pending retransmissions on SPI channel.
- rx\_err\_t `rx_comm_manager_set_auto_retransmit`(rx\_comm\_manager\_t \*mgr, const bool enabled, const rx\_spi\_comm\_retransmit\_config\_t \*config)  
Enable or disable automatic retransmission on SPI channel.

## Variables

- static const char `s_tag`[] = "COMM\_MGR"
- static const rx\_comm\_manager\_t `s_zero_mgr` = {}  
Zero-initialized comm manager template for stack-safe initialization.

## 4.3.1 Detailed Description

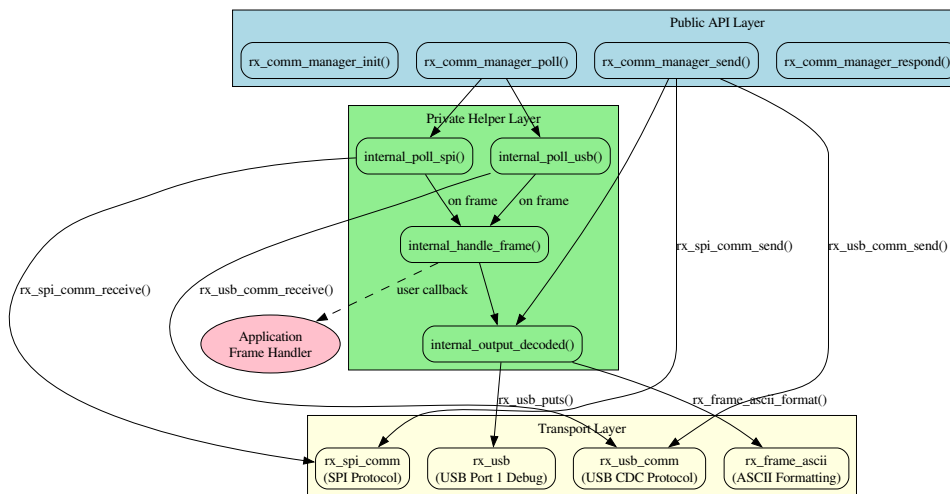
Unified Multi-Channel Communication Manager Implementation.

### 4.3.1.1 Overview

Production-quality implementation of unified communication manager coordinating multiple simultaneous channels (USB CDC and SPI) with automatic frame protocol handling, non-blocking polling architecture, optional decoded ASCII debugging output, and callback-based event notification.

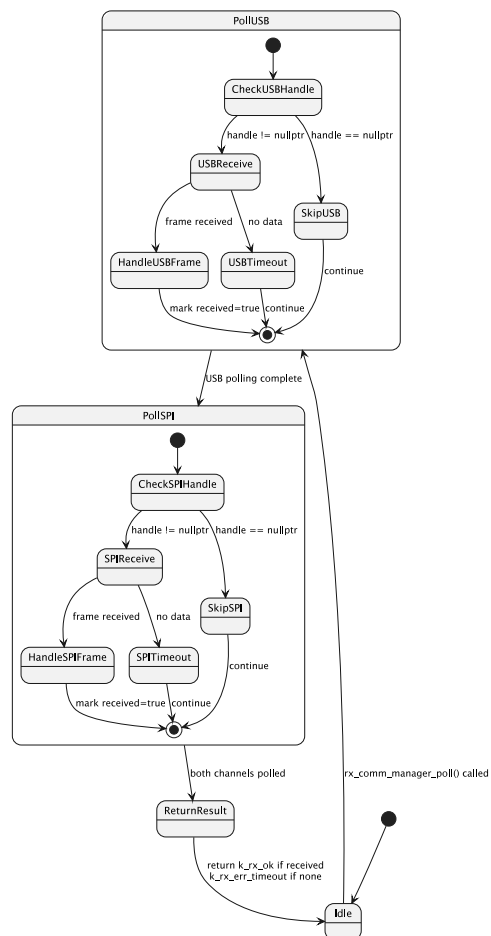
Purpose: Provide centralized, efficient coordination of independent communication channels between RX72N MCU and Raspberry Pi 5 host, abstracting low-level transport details while maintaining high performance and reliability.

### 4.3.1.2 Implementation Architecture



### 4.3.1.3 Polling State Machine

The `rx_comm_manager_poll()` function implements a non-blocking state machine that checks all enabled channels sequentially:



## 4.3.1.4 Performance Characteristics

| Operation                              | Avg Time   | Worst Case | Notes                                         |
|----------------------------------------|------------|------------|-----------------------------------------------|
| <code>rx'comm'manager'init()</code>    | ~5 us      | ~10 us     | memset dominates (328 bytes)                  |
| <code>rx'comm'manager'poll()</code>    | ~15-200 us | ~500 us    | Depends on channels enabled, frames available |
| <code>rx'comm'manager'send()</code>    | ~30-150 us | ~300 us    | Depends on payload size, channel              |
| <code>internal'poll'usb()</code>       | ~10-100 us | ~250 us    | USB CDC bulk transfer latency                 |
| <code>internal'poll'spi()</code>       | ~5-50 us   | ~150 us    | SPI hardware transfer latency                 |
| <code>internal'handle'frame()</code>   | ~2 us      | ~5 us      | Callback invocation overhead                  |
| <code>internal'output'decoded()</code> | ~50-150 us | ~300 us    | ASCII formatting + USB puts                   |

Timing Analysis:

- Measured @ 240 MHz CPU clock with -O2 optimization
- Poll time dominated by transport layer (USB/SPI receive)
- ASCII debugging adds ~50-150 us per frame (disable in production)
- Callback execution time not included (application-dependent)

## 4.3.1.5 Memory Usage

| Memory Type                           | Size      | Usage                            |
|---------------------------------------|-----------|----------------------------------|
| <code>rx'comm'manager't</code> handle | 328 bytes | Main state structure             |
| Stack (init)                          | ~32 bytes | Function locals                  |
| Stack (poll)                          | ~80 bytes | <code>rx_frame_t</code> + locals |
| Stack (send)                          | ~80 bytes | <code>rx_frame_t</code> + locals |
| ASCII buffer                          | 256 bytes | Inside handle (for debugging)    |

Memory Layout (`rx_comm_manager_t`):

| Offset                   | Size | Field                              | Purpose                      |
|--------------------------|------|------------------------------------|------------------------------|
| 0x00                     | 8    | <code>usb_handle</code>            | Pointer to USB comm handle   |
| 0x08                     | 8    | <code>spi_handle</code>            | Pointer to SPI comm handle   |
| 0x10                     | 8    | <code>callback</code>              | User frame callback function |
| 0x18                     | 8    | <code>callback_ctx</code>          | User context pointer         |
| 0x20                     | 1    | <code>enable_decoded_output</code> | ASCII debug flag             |
| 0x21                     | 1    | <code>initialized</code>           | Initialization flag          |
| 0x22                     | 2    | (padding)                          | Alignment                    |
| 0x24                     | 4    | (padding)                          | Alignment                    |
| 0x28                     | 256  | <code>ascii_buffer</code>          | ASCII formatting buffer      |
| Total: 328 bytes (0x148) |      |                                    |                              |

## 4.3.1.6 Algorithm Details

4.3.1.6.1 Polling Algorithm (`rx'comm'manager'poll`)

The polling algorithm maximizes throughput by checking all channels without blocking, returning immediately on the first error while continuing to poll remaining channels on timeout (no data available):

Steps:

1. Parameter validation (NULL check, initialized flag)
2. Poll USB channel:
  - If nullptr handle -> skip (`k_rx_err_timeout`)
  - If data available -> handle frame, mark received=true
  - If timeout -> continue to next channel
  - If error -> return error immediately (critical failure)
3. Poll SPI channel:
  - Same logic as USB
4. Return aggregated result:
  - `k_rx_ok` if ANY frame received on any channel
  - `k_rx_err_timeout` if NO frames received (normal, non-error condition)
  - Other error codes propagated immediately

Design Rationale:

- Non-blocking: Never blocks waiting for data
- Fair scheduling: Both channels polled each iteration
- Error prioritization: Critical errors returned immediately
- Timeout tolerance: Timeout is expected, not an error

#### 4.3.1.6.2 Send Algorithm (`rx_comm_manager.send`)

Routing algorithm with post-send ASCII debugging support:

Steps:

1. Validate parameters (NULL checks, payload length  $\leq$  `k_frame_max_payload`)
2. Route to channel:
  - USB: Call `rx_usb_comm_send()`
  - SPI: Call `rx_spi_comm_send()`
  - Invalid channel: Return `k_rx_err_invalid_arg`
3. On success, if ASCII output enabled:
  - Build temporary `rx_frame_t` from send parameters
  - Format as ASCII via `rx_frame_ascii_format()`
  - Send to USB Port 1 via `rx_usb_puts()`

Performance Note: ASCII formatting adds ~50-150 us overhead per send. Disable in production with `cfg->enable_decoded_output = false`.



#### 4.3.1.7 Thread Safety Analysis

Conditional Thread Safety - Safe if following rules are observed:

| Scenario                            | Thread Safety      | Mitigation Required                       |
|-------------------------------------|--------------------|-------------------------------------------|
| Single-threaded                     | [OK] Safe          | None                                      |
| Multi-threaded (different channels) | [OK] Safe          | Ensure each thread uses different channel |
| Multi-threaded (same channel)       | [X] Unsafe         | External mutex required                   |
| Multi-threaded (different handles)  | [OK] Safe          | None (independent handles)                |
| Init/Deinit from ISR                | [X] Unsafe         | Call only from task context               |
| Poll/Send from ISR                  | [WARN] Conditional | OK if callback is ISR-safe                |

Recommended Architecture:

- Dedicate one thread to poll all channels (e.g., ThreadX communication task)
- Send from any thread/ISR, but use external mutex if same channel
- Ensure user callback is reentrant if called from ISR

NASA Power of 10 Rule 5: All public functions validate preconditions with explicit checks (NULL pointers, initialized flag, parameter ranges).

#### 4.3.1.8 Integration Example

Complete System Setup with ThreadX:

```
// Global communication manager handle
static rx_comm_manager_t g_comm_mgr;
static rx_usb_comm_handle_t g_usb_handle;
static rx_spi_comm_handle_t g_spi_handle;

// User callback for frame processing
static void on_frame_received(rx_comm_channel_t channel,
                             const rx_frame_t* frame,
                             void* ctx)
{
    (void)ctx;

    // Dispatch based on frame type
    switch (frame->header.type) {
        case k_frame_type_command:
            handle_command(channel, frame);
            break;

        case k_frame_type_request:
            handle_request(channel, frame);
            break;

        default:
            // Unknown frame type, ignore
            break;
    }
}

// ThreadX communication task
static void comm_task_entry(ULONG thread_input)
{
    (void)thread_input;

    // Initialize USB CDC communication
    rx_usb_comm_config_t usb_cfg = {
        .port = k_usb_port_proto,
    };
}
```

```

rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
if (err != k_rx_ok) {
    // Handle error
    return;
}

// Initialize SPI communication
rx_spi_comm_config_t spi_cfg = {
    .spi_channel = 0,
};
err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
if (err != k_rx_ok) {
    // Handle error
    return;
}

// Initialize communication manager
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = &g_spi_handle,
    .callback = on_frame_received,
    .callback_ctx = nullptr,
    .enable_decoded_output = true, // Enable for development
};
err = rx_comm_manager_init(&g_comm_mgr, &cfg);
if (err != k_rx_ok) {
    // Handle error
    return;
}

// Main communication loop (100 Hz polling)
while (1) {
    // Poll all channels
    err = rx_comm_manager_poll(&g_comm_mgr);

    // Check for critical errors (not timeout)
    if (err != k_rx_ok && err != k_rx_err_timeout) {
        // Log error, attempt recovery
        tx_thread_sleep(100); // 1 second backoff
    }

    // Sleep 10 ticks (100 ms @ 100 Hz tick rate)
    tx_thread_sleep(10);
}

// Send response from application
void send_motor_telemetry(void)
{
    uint8_t telemetry_payload[16];
    // ... fill telemetry data ...

    // Check if USB channel is ready
    bool usb_ready = false;
    rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                                k_comm_channel_usb,
                                                &usb_ready);
    if (err == k_rx_ok && usb_ready) {
        // Send telemetry response
        err = rx_comm_manager_respond(&g_comm_mgr,
                                    k_comm_channel_usb,
                                    telemetry_payload,
                                    sizeof(telemetry_payload));
    }
}

```

#### 4.3.1.9 NASA Power of 10 Compliance

| Rule                          | Compliance | Implementation Notes                                 |
|-------------------------------|------------|------------------------------------------------------|
| Rule 1: Control Flow          | [OK]       | No goto, setjmp, longjmp, or recursion               |
| Rule 2: Loop Bounds           | [OK]       | All loops have statically provable bounds            |
| Rule 3: No Dynamic Allocation | [OK]       | Zero malloc/free - all static allocation             |
| Rule 4: Function Size         | [OK]       | All functions < 60 lines                             |
| Rule 5: Assertions            | [OK]       | Minimum 2 checks per function (nullptr, initialized) |

| Rule                       | Compliance | Implementation Notes                          |
|----------------------------|------------|-----------------------------------------------|
| Rule 6: Data Scope         | [OK]       | Variables declared at smallest scope          |
| Rule 7: Check Returns      | [OK]       | All return values validated or cast to (void) |
| Rule 8: Preprocessor       | [OK]       | Minimal macros, typed enums for constants     |
| Rule 9: Pointers           | [WARN]     | Function pointers used for callback (DIP)     |
| Rule 10: Compiler Warnings | [OK]       | -Wall -Wextra -Werror enabled                 |

Rule 9 Deviation: Function pointer used for user callback to enable Dependency Inversion Principle (SOLID) - allows application to register frame handler without tight coupling. This is intentional and documented per project policy.

#### 4.3.1.10 SOLID Principles

| Principle             | Implementation                                                                      |
|-----------------------|-------------------------------------------------------------------------------------|
| Single Responsibility | Module responsible ONLY for channel coordination; frame parsing/transport delegated |
| Open/Closed           | Extensible via callback; new frame types handled by application                     |
| Liskov Substitution   | Channels are interchangeable (same rx_frame_t interface)                            |
| Interface Segregation | Small focused API (7 functions); send/poll/ready separated                          |
| Dependency Inversion  | Depends on abstract rx_frame_t and channel handles, not concrete implementations    |

Key Design Decision: The manager does NOT parse frame payloads - it only routes frames to the user callback. This separation allows application-specific command handling without modifying the communication layer.

#### 4.3.1.11 Module Dependencies

Includes:

- [rx\\_comm\\_manager.h](#) - Public API definitions
- [rx\\_frame\\_ascii.h](#) - ASCII formatting for debugging
- [rx\\_usb.h](#) - USB port access (Port 1 debug output)
- [string.h](#) - Standard C library (memset, memcpy)

Links Against:

- [rx\\_usb\\_comm](#) - USB CDC frame protocol implementation
- [rx\\_spi\\_comm](#) - SPI frame protocol implementation
- [rx\\_frame\\_ascii](#) - ASCII frame formatting
- [rx\\_usb](#) - USB hardware abstraction

Date

2026-01-27

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

See also

[rx\\_comm\\_manager.h](#) Full API documentation  
[rx\\_spi\\_comm.h](#) SPI protocol implementation  
[rx\\_uart\\_comm.h](#) UART protocol implementation  
[rx\\_i2c\\_comm.h](#) I2C peripheral protocol implementation  
[rx\\_frame\\_ascii.h](#) ASCII debugging interface

Definition in file [rx\\_comm\\_manager.c](#).

## 4.3.2 Enumeration Type Documentation

### 4.3.2.1 internal\_constants\_t

enum [internal\\_constants\\_t](#) : uint32\_t

Internal constants for communication manager implementation.

Defines compile-time constants used internally by the communication manager for timeout values, placeholder values, and buffer indexing.

Design Notes:

- Non-blocking timeout (0 ms) ensures poll never blocks
- Placeholder values filled by transport layer (sequence, CRC)
- Explicit array index constant for bounds checking

Since

Version 1.0.0

#### Enumerator

|                          |                                                                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| k_receive_timeout_ms     | Non-blocking receive timeout for polling. Set to 0 ms to ensure <a href="#">rx_comm_manager_poll()</a> never blocks waiting for data. Transport layers (USB/SPI) return k_rx_err_timeout immediately if no data.<br><br>Value: 0 milliseconds |
| k_frame_seq_placeholder  | Placeholder for frame sequence number (filled by transport layer). Sequence number is managed by transport layer (rx_usb_comm/rx_spi_comm) to detect dropped frames. Manager does not manipulate sequence numbers.<br><br>Value: 0            |
| k_frame_crc_placeholder  | Placeholder for frame CRC-32 (filled by transport layer). CRC-32 calculated and verified by transport layer. Manager builds temp frame with placeholder CRC for ASCII formatting only.<br><br>Value: 0                                        |
| k_ascii_buffer_first_idx | First index of ascii_buffer for post-condition validation. Used in <a href="#">rx_comm_manager_init()</a> to verify memset properly cleared buffer. First byte must be '\0' after zero-fill.<br><br>Value: 0                                  |
| k_zero_u32               | Typed zero constant for uint32_t comparisons. Used for payload length checks and other uint32_t zero comparisons to avoid magic number 0. Provides type safety and self-documenting code.<br><br>Value: 0                                     |

Definition at line 462 of file [rx\\_comm\\_manager.c](#).

```

00462         : uint32_t {
00463     /**
00464     * @brief Non-blocking receive timeout for polling
00465     * @details
00466     * Set to 0 ms to ensure rx_comm_manager_poll() never blocks waiting for data.
00467     * Transport layers (USB/SPI) return k_rx_err_timeout immediately if no data.
00468     * @par Value: 0 milliseconds
00469     */
00470     k_receive_timeout_ms = 0,
00471
00472     /**
00473     * @brief Placeholder for frame sequence number (filled by transport layer)
00474     * @details
00475     * Sequence number is managed by transport layer (rx_usb_comm/rx_spi_comm)
00476     * to detect dropped frames. Manager does not manipulate sequence numbers.
00477     * @par Value: 0
00478     */
00479     k_frame_seq_placeholder = 0,

```

```

00480
00481 /**
00482  * @brief Placeholder for frame CRC-32 (filled by transport layer)
00483  * @details
00484  * CRC-32 calculated and verified by transport layer. Manager builds temp
00485  * frame with placeholder CRC for ASCII formatting only.
00486  * @par Value: 0
00487  */
00488 k_frame_crc_placeholder = 0,
00489
00490 /**
00491  * @brief First index of ascii_buffer for post-condition validation
00492  * @details
00493  * Used in rx_comm_manager_init() to verify memset properly cleared buffer.
00494  * First byte must be '\0' after zero-fill.
00495  * @par Value: 0
00496  */
00497 k_ascii_buffer_first_idx = 0,
00498
00499 /**
00500  * @brief Typed zero constant for uint32_t comparisons
00501  * @details
00502  * Used for payload length checks and other uint32_t zero comparisons to avoid
00503  * magic number 0. Provides type safety and self-documenting code.
00504  * @par Value: 0
00505  */
00506 k_zero_u32 = 0,
00507 } internal_constants_t;

```

#### 4.3.2.2 rx\_comm\_sim\_time\_t

```
enum rx_comm_sim_time_t : uint32_t
```

Get current time in milliseconds.

Returns the current system time in milliseconds. On RX72N hardware, this uses ThreadX tx\_time\_get() multiplied by tick period. In simulator builds, returns 0 (timing not available).

Since

Version 1.0.0

Simulator time stub value (time is unavailable in simulator)

Enumerator

|                    |                                             |
|--------------------|---------------------------------------------|
| k_sim_time_ms_zero | Simulator returns zero (no real-time clock) |
|--------------------|---------------------------------------------|

Definition at line 429 of file rx\_comm\_manager.c.

```

00429 : uint32_t {
00430 k_sim_time_ms_zero = 0, /**< Simulator returns zero (no real-time clock) */
00431 } rx_comm_sim_time_t;

```

### 4.3.3 Function Documentation

#### 4.3.3.1 internal\_check\_channel\_heartbeat()

```

void internal_check_channel_heartbeat (
    rx_comm_manager_t * mgr,
    uint8_t ch,
    uint32_t now_ms) [static]

```

Check heartbeat implicit timeout on all transports.

Evaluates the dual-detection heartbeat model. If no valid frame has been received on a transport within 200ms, the link is declared dead and the status callback is invoked.

## Precondition

mgr must be non-NULL and initialized

## Postcondition

Link status updated for each transport

## Since

Version 1.0.0

Check heartbeat timeout on a single channel

## Precondition

ch < k\_comm\_channel\_count

## Postcondition

Link status updated if timeout exceeded

## Since

Version 1.0.0

Definition at line 979 of file [rx\\_comm\\_manager.c](#).

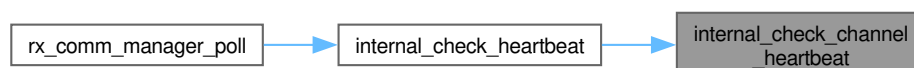
```

00980 {
00981   rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[ch];
00982
00983   /* Skip channels that haven't received any frame yet */
00984   if (hb->status != k_link_status_healthy) {
00985     return;
00986   }
00987
00988   /* Check implicit timeout: 200ms since last valid frame */
00989   const uint32_t elapsed = now_ms - hb->last_rx_ms;
00990   if (elapsed < k_heartbeat_implicit_timeout_ms) {
00991     return;
00992   }
00993
00994   hb->status = k_link_status_dead;
00995   rx_log_warn_val(s_tag, "Link dead: no frame (ms elapsed)", elapsed);
00996
00997   if (mgr->link_status_cb != nullptr) {
00998     mgr->link_status_cb((rx_comm_channel_t)ch, k_link_status_dead, mgr->link_status_ctx);
00999   }
01000 }
```

References [rx\\_comm\\_manager\\_t::heartbeat](#), [k\\_heartbeat\\_implicit\\_timeout\\_ms](#), [k\\_link\\_status\\_dead](#), [k\\_link\\_status\\_healthy](#), [rx\\_comm\\_heartbeat\\_state\\_t::last\\_rx\\_ms](#), [rx\\_comm\\_manager\\_t::link\\_status\\_cb](#), [rx\\_comm\\_manager\\_t::link\\_status\\_ctx](#), [s\\_tag](#), and [rx\\_comm\\_heartbeat\\_state\\_t::status](#).

Referenced by [internal\\_check\\_heartbeat\(\)](#).

Here is the caller graph for this function:



#### 4.3.3.2 internal\_check\_heartbeat()

```
void internal_check_heartbeat (
    rx_comm_manager_t * mgr) [static]
```

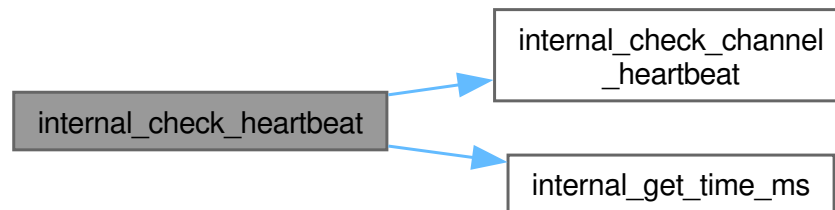
Definition at line 1002 of file [rx\\_comm\\_manager.c](#).

```
01003 {
01004     const uint32_t now = internal_get_time_ms();
01005
01006     for (uint8_t ch = 0; ch < k_comm_channel_count; ch++) {
01007         internal_check_channel_heartbeat(mgr, ch, now);
01008     }
01009 }
```

References [internal\\_check\\_channel\\_heartbeat\(\)](#), [internal\\_get\\_time\\_ms\(\)](#), and [k\\_comm\\_channel\\_count](#).

Referenced by [rx\\_comm\\_manager\\_poll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.3 internal\_dequeue\_event()

```
void internal_dequeue_event (
    rx_comm_manager_t * mgr,
    rx_comm_event_entry_t * entry) [static]
```

Process one event from the FIFO queue.

Dequeues the oldest event and attempts to send it. For SPI events, if the send fails, the event is re-queued with incremented retry count (up to `k_event_max_retries`). For USB events, failure is simply dropped (fire-and-forget).

Precondition

`mgr` must be non-NULL and initialized

Postcondition

One event processed (sent or dropped on max retries)



Since

Version 1.0.0

Dequeue the front event entry and advance the tail pointer

Precondition

`mgr->event_queue_count > 0`

Postcondition

entry cleared and tail pointer advanced

Since

Version 1.0.0

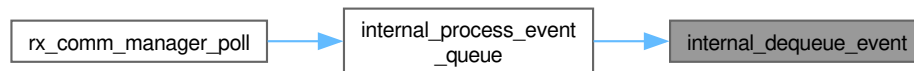
Definition at line 882 of file [rx\\_comm\\_manager.c](#).

```
00883 {
00884     entry->occupied      = false;
00885     entry->retries        = 0;
00886     entry->next_retry_time_ms = 0;
00887     mgr->event_queue_tail = (mgr->event_queue_tail + 1) % k_comm_event_queue_depth;
00888     mgr->event_queue_count--;
00889 }
```

References [rx\\_comm\\_manager\\_t::event\\_queue\\_count](#), [rx\\_comm\\_manager\\_t::event\\_queue\\_tail](#), [k\\_comm\\_event\\_queue\\_depth](#), [rx\\_comm\\_event\\_entry\\_t::next\\_retry\\_time\\_ms](#), [rx\\_comm\\_event\\_entry\\_t::occupied](#), and [rx\\_comm\\_event\\_entry\\_t::retries](#).

Referenced by [internal\\_process\\_event\\_queue\(\)](#).

Here is the caller graph for this function:



#### 4.3.3.4 internal\_get\_time\_ms()

```
uint32_t internal_get_time_ms (
    void ) [static]
```

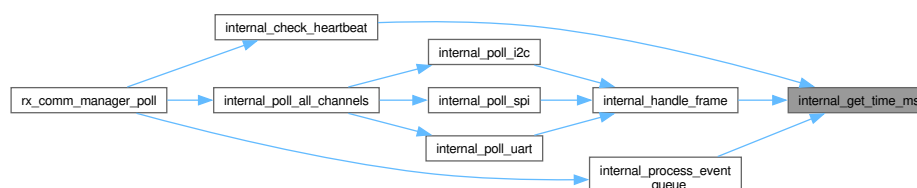
Definition at line 433 of file [rx\\_comm\\_manager.c](#).

```
00434 {
00435     #if !RX_IS_SIMULATOR
00436     return (uint32_t)tx_time_get() * k_threadx_ms_per_tick;
00437     #else
00438     return k_sim_time_ms_zero;
00439     #endif
00440 }
```

References [k\\_sim\\_time\\_ms\\_zero](#).

Referenced by [internal\\_check\\_heartbeat\(\)](#), [internal\\_handle\\_frame\(\)](#), and [internal\\_process\\_event\\_queue\(\)](#).

Here is the caller graph for this function:



#### 4.3.3.5 internal\_handle\_frame()

```
RX_STATIC_TESTABLE void internal_handle_frame (  
    rx_comm_manager_t * mgr,  
    rx_comm_channel_t channel,  
    const rx_frame_t * frame)
```

Handle received frame by invoking user callback and optional ASCII output.

Central frame dispatch point for all channels. Performs optional ASCII debugging output, then invokes user-registered callback with frame data. This function isolates callback invocation logic from channel-specific polling.

Algorithm:

1. Validate parameters (NULL checks, channel range)
2. Output decoded ASCII to Port 1 if enabled (RX direction)
3. Invoke user callback if registered

Design Rationale: Centralizing frame handling logic ensures consistent behavior across USB and SPI channels (single point of dispatch).

Precondition

mgr must be initialized  
frame must be valid and CRC-verified by transport layer  
channel must be in valid range [0, k\_comm\_channel\_count)

Postcondition

User callback invoked if registered (callback may be NULL)  
ASCII output sent to Port 1 if enabled

Note

Thread safety depends on user callback implementation  
Callback errors are not returned (callback should handle internally)

Warning

User callback must not block for extended periods (affects polling rate)

Performance:

- Without decoded output: ~2 us (callback overhead only)
- With decoded output: ~50-150 us (ASCII formatting + USB puts)

See also

[internal\\_output\\_decoded\(\)](#) ASCII debugging output  
[rx\\_comm\\_frame\\_callback\\_t](#) User callback type

Since

Version 1.0.0

Definition at line 648 of file [rx\\_comm\\_manager.c](#).

```

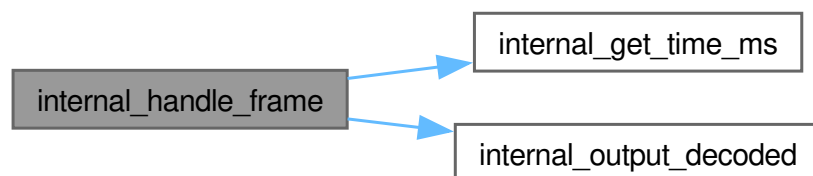
00649 {
00650  /* Update heartbeat: any valid frame resets the implicit timeout timer.
00651   * Per the dual-detection model, this is the primary link health
00652   * mechanism (200ms implicit timeout). */
00653  rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[channel];
00654  hb->last_rx_ms = internal_get_time_ms();
00655  if (hb->status != k_link_status_healthy) {
00656    rx_comm_link_status_t old_status = hb->status;
00657    hb->status = k_link_status_healthy;
00658    if (old_status == k_link_status_dead) {
00659      rx_log_info(s_tag, "Link recovered: frame received");
00660    } else {
00661      rx_log_info(s_tag, "Link established: first frame received");
00662    }
00663    if (mgr->link_status_cb != nullptr) {
00664      mgr->link_status_cb(channel, k_link_status_healthy, mgr->link_status_ctx);
00665    }
00666  }
00667  rx_log_debug_val(s_tag, "Frame received, type", (uint8_t)frame->header.type);
00668  if (frame->header.type == k_frame_type_command) {
00669    rx_log_info_val(s_tag, "CMD frame seq", frame->header.sequence);
00670    rx_log_info_val(s_tag, "CMD frame channel", (uint8_t)channel);
00671  }
00672  /* Output decoded ASCII to Port 1 */
00673  internal_output_decoded(mgr, frame, false);
00674  /* Invoke user callback if set */
00675  if (mgr->callback != nullptr) {
00676    mgr->callback(channel, frame, mgr->callback_ctx);
00677  }
00678 }
00679
00680
00681
00682

```

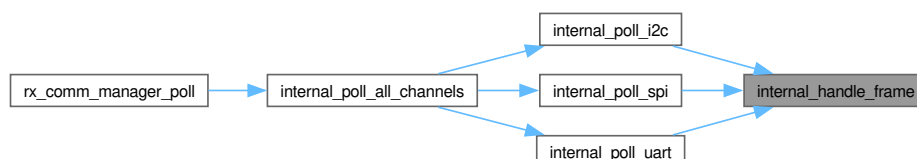
References [rx\\_comm\\_manager\\_t::callback](#), [rx\\_comm\\_manager\\_t::callback\\_ctx](#), [rx\\_comm\\_manager\\_t::heartbeat](#), [internal\\_get\\_time\\_ms\(\)](#), [internal\\_output\\_decoded\(\)](#), [k\\_link\\_status\\_dead](#), [k\\_link\\_status\\_healthy](#), [rx\\_comm\\_heartbeat\\_state\\_t::last\\_rx\\_ms](#), [rx\\_comm\\_manager\\_t::link\\_status\\_cb](#), [rx\\_comm\\_manager\\_t::link\\_status\\_s\\_tag](#), and [rx\\_comm\\_heartbeat\\_state\\_t::status](#).

Referenced by [internal\\_poll\\_i2c\(\)](#), [internal\\_poll\\_spi\(\)](#), and [internal\\_poll\\_uart\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.6 internal\_output\_decoded()

```
RX_STATIC_TESTABLE void internal_output_decoded (  
    rx_comm_manager_t * mgr,  
    const rx_frame_t * frame,  
    bool is_tx)
```

Output decoded ASCII frame to USB Port 1 for debugging.

Formats the given frame as human-readable ASCII text and outputs to USB Port 1 (decoded debug port) if decoded output is enabled. Used for development/debugging to monitor frame traffic without binary protocol parsing.

Algorithm:

1. Validate parameters (NULL checks)
2. Check if decoded output is enabled (cfg->enable\_decoded\_output)
3. Format frame as ASCII via rx\_frame\_ascii\_format()
4. Send ASCII string to USB Port 1 via rx\_usb\_puts()

Performance: ~50-150 us depending on frame size (ASCII formatting overhead). Disable in production for optimal performance.

Precondition

mgr must be initialized via rx\_comm\_manager\_init()  
frame must be a valid frame structure

Postcondition

mgr->ascii\_buffer content is modified (temporary)  
ASCII output sent to USB Port 1 if enabled and successful

Note

Not thread-safe - caller must ensure exclusive access to mgr->ascii\_buffer  
Errors are silently ignored (formatting/USB send failures)  
USB Port 1 must be configured for ASCII output to appear

Example Output:

```
RX: [USB] Type=0x01 Flags=0x00 Len=8 Seq=42 Payload=0102030405060708 CRC=ABCD1234  
TX: [SPI] Type=0x02 Flags=0x80 Len=4 Seq=43 Payload=01020304 CRC=5678CDEF
```

See also

rx\_frame\_ascii\_format() Frame-to-ASCII conversion  
rx\_usb\_puts() USB debug output



Since

Version 1.0.0

Definition at line 589 of file [rx\\_comm\\_manager.c](#).

```

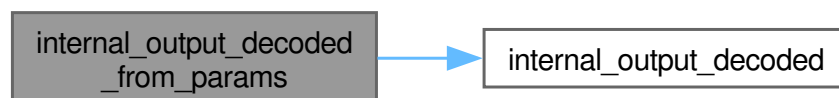
00591 {
00592     /* Build a temporary frame for ASCII formatting */
00593     rx_frame_t frame = {}; /* Zero-initialize to clear padding/unset fields */
00594     frame.header.sequence = k_frame_seq_placeholder;
00595     frame.header.length = (uint16_t)params->payload_len;
00596     frame.header.type = (uint8_t)params->type;
00597     frame.header.flags = params->flags;
00598
00599     /* payload_len <= k_frame_max_payload is guaranteed by caller validation */
00600     for (uint32_t i = 0; i < params->payload_len; i++) {
00601         frame.payload[i] = params->payload[i];
00602     }
00603     frame.crc = k_frame_crc_placeholder;
00604
00605     internal_output_decoded(mgr, &frame, true);
00606 }

```

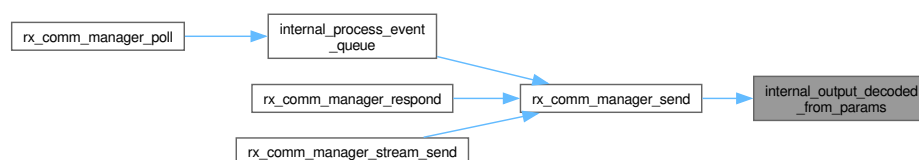
References [rx\\_comm\\_send\\_params\\_t::flags](#), [internal\\_output\\_decoded\(\)](#), [k\\_frame\\_crc\\_placeholder](#), [k\\_frame\\_seq\\_placeholder](#), [rx\\_comm\\_send\\_params\\_t::payload](#), [rx\\_comm\\_send\\_params\\_t::payload\\_len](#), and [rx\\_comm\\_send\\_params\\_t::type](#).

Referenced by [rx\\_comm\\_manager\\_send\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.8 internal\_poll\_all\_channels()

```

rx_err_t internal_poll_all_channels (
    rx_comm_manager_t * mgr,
    bool * received) [static]

```

Poll all enabled communication channels for incoming frames (non-blocking).

Performs non-blocking poll of all enabled channels (USB and SPI) sequentially, dispatching any received frames to the registered user callback. Returns aggregate result indicating whether any frames were received across all channels.

Algorithm:

1. Validate parameters: Check mgr pointer and initialized flag (NASA Rule 5)

2. Poll USB channel via `internal_poll_usb()`:
  - If `k_rx_ok`: Mark received=true, continue to SPI
  - If `k_rx_err_timeout`: Continue to SPI (no data is normal)
  - If other error: Return error immediately (critical failure)
3. Poll SPI channel via `internal_poll_spi()`:
  - If `k_rx_ok`: Mark received=true
  - If `k_rx_err_timeout`: Continue (no data is normal)
  - If other error: Return error immediately (critical failure)
4. Return result:
  - `k_rx_ok` if ANY channel received frame
  - `k_rx_err_timeout` if NO channels received (normal condition)
  - Other error codes propagated from transport layers

Performance: 15-200 us typical, depends on:

- Number of enabled channels (USB/SPI/both)
- Whether frames are available (data path vs fast timeout path)
- ASCII debugging enabled (adds ~50-150 us per frame)
- User callback execution time (not included in measurement)

Design Rationale:

- Non-blocking: Never blocks waiting for data (0 ms timeout)
- Fair scheduling: Both channels polled each iteration
- Timeout tolerance: Timeout is expected, not error condition
- Error prioritization: Critical errors returned immediately
- Aggregation: `k_rx_ok` if ANY channel received data

Precondition

`mgr` must be non-NULL

`mgr` must be initialized

At least one channel should be enabled (`usb_handle` or `spi_handle` non-NULL)

Postcondition

User callback invoked for each successfully received frame

ASCII output sent to Port 1 if enabled

## Note

Non-blocking - always returns immediately

k\_rx\_err\_timeout is NORMAL - means no data available (not an error)

User callback execution time is NOT included in performance measurements

## Warning

Call this function regularly (e.g., 100 Hz) to avoid buffer overruns

User callback must not block for extended periods

## Performance:

- No frames available: ~15 us (both channels timeout, fast path)
- USB frame received: ~100-250 us (USB bulk transfer + callback)
- SPI frame received: ~50-150 us (SPI hardware transfer + callback)
- Both frames: ~150-400 us (both channels + callbacks)
- With ASCII debugging: Add ~50-150 us per frame

## Typical Usage - ThreadX Communication Task:

```
static void comm_task_entry(ULONG thread_input)
{
    (void)thread_input;

    // Initialize communication manager
    // ... (see rx_comm_manager_init() documentation) ...

    // Main polling loop @ 100 Hz
    while (1) {
        // Poll all channels (non-blocking)
        rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);

        // Handle critical errors (not timeout)
        if (err != k_rx_ok && err != k_rx_err_timeout) {
            // Log error, implement recovery strategy
            log_error("Comm poll failed: %d", err);
            tx_thread_sleep(100); // 1 second backoff
            continue;
        }

        // Timeout is normal - just means no data available

        // Sleep 10 ticks (100 ms @ 100 Hz tick rate = 10 Hz polling)
        tx_thread_sleep(10);
    }
}
```

## Error Handling - Distinguishing Timeout from Errors:

```
rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
if (err == k_rx_ok) {
    // At least one frame received and processed
} else if (err == k_rx_err_timeout) {
    // No frames available - this is NORMAL, not an error
} else {
    // Critical error - handle appropriately
    handle_comm_error(err);
}
```



See also

[internal\\_poll\\_usb\(\)](#) USB channel polling implementation  
[internal\\_poll\\_spi\(\)](#) SPI channel polling implementation  
[internal\\_handle\\_frame\(\)](#) Frame dispatch  
[rx\\_comm\\_manager\\_init\(\)](#) Initialization with channel configuration

Since

Version 1.0.0

Poll all transport channels and aggregate results

Precondition

mgr must be non-NULL and initialized

Postcondition

\*received reflects whether any frames were received

Since

Version 1.0.0

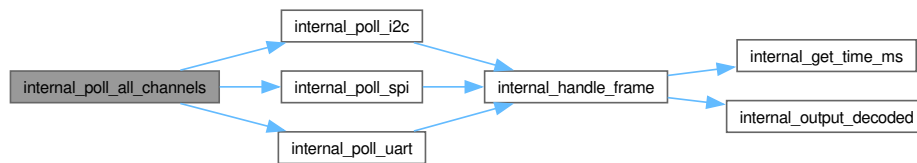
Definition at line 1351 of file [rx\\_comm\\_manager.c](#).

```
01352 {
01353     /* Poll UART channel (primary path to the Pi5 gateway) */
01354     rx_err_t uart_err = internal_poll_uart(mgr);
01355     if (uart_err == k_rx_ok) {
01356         *received = true;
01357     } else if (uart_err != k_rx_err_timeout && uart_err != k_rx_err_no_data) {
01358         rx_log_error_val(s_tag, "UART poll error", uart_err);
01359         return uart_err;
01360     }
01361
01362     /* Poll SPI channel */
01363     rx_err_t spi_err = internal_poll_spi(mgr);
01364     if (spi_err == k_rx_ok) {
01365         *received = true;
01366     } else if (spi_err != k_rx_err_timeout) {
01367         rx_log_error_val(s_tag, "SPI poll error", spi_err);
01368         return spi_err;
01369     }
01370
01371     /* Poll I2C channel */
01372     rx_err_t i2c_err = internal_poll_i2c(mgr);
01373     if (i2c_err == k_rx_ok) {
01374         *received = true;
01375     } else if (i2c_err != k_rx_err_timeout && i2c_err != k_rx_err_no_data) {
01376         rx_log_error_val(s_tag, "I2C poll error", i2c_err);
01377         return i2c_err;
01378     }
01379
01380     return k_rx_ok;
01381 }
```

References [internal\\_poll\\_i2c\(\)](#), [internal\\_poll\\_spi\(\)](#), [internal\\_poll\\_uart\(\)](#), and [s\\_tag](#).

Referenced by [rx\\_comm\\_manager\\_poll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.9 internal\_poll\_i2c()

```

rx_err_t internal_poll_i2c (
    rx_comm_manager_t * mgr) [static]
  
```

Poll I2C channel for incoming frames (non-blocking).

Attempts to receive one frame from I2C peripheral channel with zero timeout. If frame is successfully received, it is dispatched via [internal\\_handle\\_frame\(\)](#).

##### Precondition

mgr must be non-NULL

mgr->i2c\_handle must be initialized before calling [rx\\_comm\\_manager\\_poll\(\)](#)

##### Postcondition

On k\_rx\_ok: callback invoked for received frame

On k\_rx\_err\_timeout or k\_rx\_err\_no\_data: no state change

##### Note

Non-blocking - always returns immediately

##### See also

[internal\\_handle\\_frame\(\)](#) Frame dispatch

[rx\\_i2c\\_comm\\_receive\(\)](#) I2C peripheral receive implementation

Since

Version 1.0.0

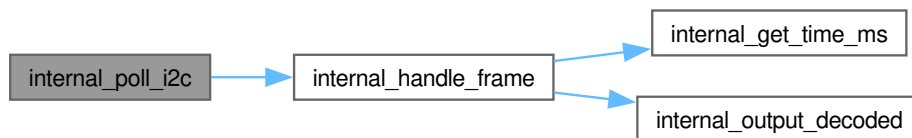
Definition at line 796 of file [rx\\_comm\\_manager.c](#).

```
00797 {
00798     if (mgr->i2c_handle == nullptr) {
00799         return k_rx_err_timeout;
00800     }
00801
00802     rx_frame_t frame;
00803     rx_err_t err = rx_i2c_comm_receive(mgr->i2c_handle, &frame, k_receive_timeout_ms);
00804
00805     if (err == k_rx_ok) {
00806         internal_handle_frame(mgr, k_comm_channel_i2c, &frame);
00807         return k_rx_ok;
00808     }
00809
00810     return err;
00811 }
```

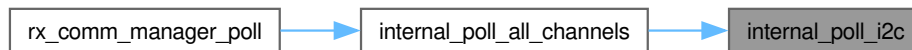
References [rx\\_comm\\_manager\\_t::i2c\\_handle](#), [internal\\_handle\\_frame\(\)](#), [k\\_comm\\_channel\\_i2c](#), and [k\\_receive\\_timeout\\_ms](#).

Referenced by [internal\\_poll\\_all\\_channels\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.10 internal\_poll\_spi()

```
rx_err_t internal_poll_spi (
    rx_comm_manager_t * mgr) [static]
```

Poll SPI channel for incoming frames (non-blocking).

Attempts to receive one frame from SPI channel with zero timeout (non-blocking). If frame is successfully received, it is dispatched via [internal\\_handle\\_frame\(\)](#). Timeout (no data available) is expected and normal - not an error condition.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Skip if SPI handle is nullptr (channel not configured)
3. Call `rx_spi_comm_receive()` with 0 ms timeout
4. On success: dispatch frame via [internal\\_handle\\_frame\(\)](#)
5. On timeout: return `k_rx_err_timeout` (expected, not error)
6. On other errors: propagate error code

Design Note: Identical logic to `internal_poll_usb()` but for SPI channel. Code duplication is intentional (NASA Rule 4: keep functions short and simple).

## Precondition

mgr must be non-NULL

If SPI channel enabled, spi\_handle must be initialized

## Postcondition

On k\_rx\_ok: callback invoked, ASCII output sent (if enabled)

On timeout: no side effects (no frame received)

## Note

Non-blocking - always returns immediately

Timeout is NOT an error - it means no data is available

## Warning

Do not call if spi\_handle is uninitialized (will return timeout)

## Performance:

- No data available: ~5 us (fast path)
- Frame received: ~50-150 us (SPI hardware transfer + callback)

## See also

[internal\\_handle\\_frame\(\)](#) Frame dispatch

[rx\\_spi\\_comm\\_receive\(\)](#) SPI receive implementation

## Since

Version 1.0.0

Definition at line 723 of file [rx\\_comm\\_manager.c](#).

```

00724 {
00725     if (mgr->spi_handle == nullptr) {
00726         return k_rx_err_timeout;
00727     }
00728
00729     /* If SPI link layer is available, use it for HARQ decoding
00730      * NOTE: Using static buffers to avoid stack overflow (~3KB total).
00731      * Safe because comm manager poll is single-threaded (called from comm_task). */
00732     if (mgr->spi_link != nullptr) {
00733         static rx_spi_link_receive_result_t s_link_result;
00734         rx_err_t err = rx_spi_link_receive(mgr->spi_link, &s_link_result, k_receive_timeout_ms);
00735
00736         if (err == k_rx_ok) {
00737             /* Convert link result to rx_frame_t for internal_handle_frame() */
00738             static rx_frame_t s_frame1;
00739             s_frame1 = (rx_frame_t){};
00740             s_frame1.header.sequence = s_link_result.sequence;
00741             s_frame1.header.length = (uint16_t)s_link_result.payload_len;
00742             s_frame1.header.type = s_link_result.frame_type;
00743
00744             /* Preserve retransmit and FEC flags from link layer result */
00745             s_frame1.header.flags = k_frame_flag_none;
00746             if (s_link_result.is_retransmit) {
00747                 s_frame1.header.flags |= k_frame_flag_retransmit;
00748             }
00749             if (s_link_result.fec_decoded) {

```

```

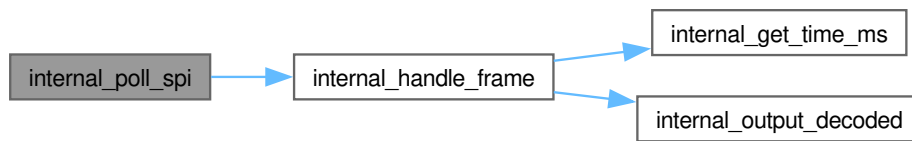
00750     s_frame1.header.flags |= k_frame_flag_fec_enabled;
00751 }
00752
00753 for (uint32_t i = 0; i < s_link_result.payload_len; i++) {
00754     s_frame1.payload[i] = s_link_result.payload[i];
00755 }
00756 internal_handle_frame(mgr, k_comm_channel_spi, &s_frame1);
00757 return k_rx_ok;
00758 }
00759
00760 return err;
00761 }
00762
00763 /* Fallback: raw SPI without HARQ */
00764 static rx_frame_t s_frame2;
00765 rx_err_t      err = rx_spi_comm_receive(mgr->spi_handle, &s_frame2, k_receive_timeout_ms);
00766
00767 if (err == k_rx_ok) {
00768     internal_handle_frame(mgr, k_comm_channel_spi, &s_frame2);
00769     return k_rx_ok;
00770 }
00771
00772 return err;
00773 }

```

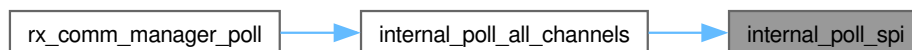
References [internal\\_handle\\_frame\(\)](#), [k\\_comm\\_channel\\_spi](#), [k\\_receive\\_timeout\\_ms](#), [rx\\_comm\\_manager\\_t::spi\\_handle](#) and [rx\\_comm\\_manager\\_t::spi\\_link](#).

Referenced by [internal\\_poll\\_all\\_channels\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.11 internal\_poll\_uart()

```

rx_err_t internal_poll_uart (
    rx_comm_manager_t * mgr) [static]

```

Poll UART channel for incoming frames (non-blocking).

Attempts to receive one frame from UART (SCI9) channel with zero timeout. If frame is successfully received, it is dispatched via [internal\\_handle\\_frame\(\)](#).

Precondition

`mgr` must be non-NULL

`mgr->uart_handle` must be initialized before calling [rx\\_comm\\_manager\\_poll\(\)](#)

## Postcondition

On `k_rx_ok`: callback invoked for received frame

On `k_rx_err_timeout` or `k_rx_err_no_data`: no state change

## Note

Non-blocking - always returns immediately

## See also

[internal\\_handle\\_frame\(\)](#) Frame dispatch

[rx\\_uart\\_comm\\_receive\(\)](#) UART receive implementation

## Since

Version 1.0.0

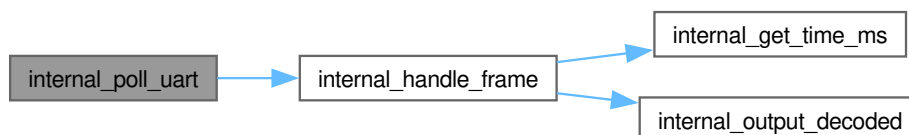
Definition at line 834 of file [rx\\_comm\\_manager.c](#).

```
00835 {
00836     if (mgr->uart_handle == nullptr) {
00837         return k_rx_err_timeout;
00838     }
00839     rx_frame_t frame;
00840     rx_err_t err = rx_uart_comm_receive(mgr->uart_handle, &frame, k_receive_timeout_ms);
00841     if (err == k_rx_ok) {
00842         if (frame.header.type == k_frame_type_command) {
00843             rx_log_info_val(s_tag, "UART RX CMD seq", frame.header.sequence);
00844             internal_handle_frame(mgr, k_comm_channel_uart, &frame);
00845             return k_rx_ok;
00846         }
00847     }
00848     return err;
00849 }
00850
00851 return err;
00852 }
```

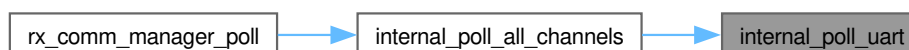
References [internal\\_handle\\_frame\(\)](#), [k\\_comm\\_channel\\_uart](#), [k\\_receive\\_timeout\\_ms](#), [s\\_tag](#), and [rx\\_comm\\_manager\\_t::uart\\_handle](#).

Referenced by [internal\\_poll\\_all\\_channels\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 4.3.3.12 internal\_process\_event\_queue()

```
void internal_process_event_queue (
    rx_comm_manager_t * mgr) [static]
```

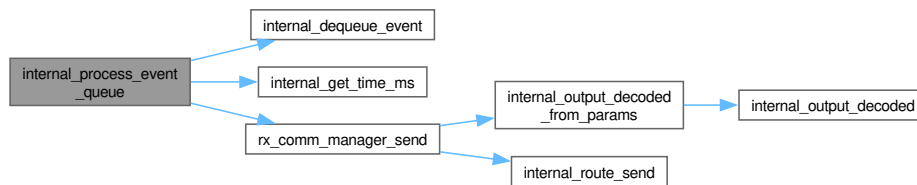
Definition at line 891 of file `rx_comm_manager.c`.

```
00892 {
00893     if (mgr->event_queue_count == 0) {
00894         return;
00895     }
00896
00897     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_tail];
00898     if (!entry->occupied) {
00899         return;
00900     }
00901
00902     /* Check backoff: skip if retry time hasn't been reached yet */
00903     const uint32_t now_ms = internal_get_time_ms();
00904     if (entry->retries > 0 && now_ms < entry->next_retry_time_ms) {
00905         return; /* Backoff period not elapsed, try again later */
00906     }
00907
00908     /* Build send params from queued entry */
00909     const rx_comm_send_params_t params = {
00910         .channel    = entry->channel,
00911         .type       = entry->type,
00912         .flags      = entry->flags,
00913         .payload     = entry->payload,
00914         .payload_len = entry->payload_len,
00915     };
00916
00917     rx_err_t err = rx_comm_manager_send(mgr, &params);
00918
00919     if (err == k_rx_ok || entry->channel == k_comm_channel_uart) {
00920         /* Success, or UART fire-and-forget: dequeue.
00921          * UART has no link-layer retry (the CY7C65213 bridge + USB CDC transport
00922          * on the host provides enough reliability for the command/response path). */
00923         internal_dequeue_event(mgr, entry);
00924         if (err == k_rx_ok) {
00925             rx_log_debug_val(s_tag, "Event sent, queue depth", mgr->event_queue_count);
00926         } else {
00927             rx_log_warn_val(s_tag, "UART event dropped, queue depth", mgr->event_queue_count);
00928         }
00929         return;
00930     }
00931
00932     /* SPI send failed: retry with exponential backoff */
00933     entry->retries++;
00934     if (entry->retries >= k_event_max_retries) {
00935         rx_log_error_val(s_tag, "SPI event failed after retries", (uint8_t)entry->retries);
00936         internal_dequeue_event(mgr, entry);
00937         return;
00938     }
00939
00940     /* Compute exponential backoff: initial_ms « (retries - 1), capped at max */
00941     uint32_t backoff_ms = (uint32_t)k_event_initial_backoff_ms « (entry->retries - 1);
00942     if (backoff_ms > k_event_max_backoff_ms) {
00943         backoff_ms = k_event_max_backoff_ms;
00944     }
00945     entry->next_retry_time_ms = now_ms + backoff_ms;
00946
00947     rx_log_warn_val(s_tag, "SPI event retry attempt", (uint8_t)entry->retries);
00948     /* Entry stays in queue, will be retried after backoff period */
00949 }
```

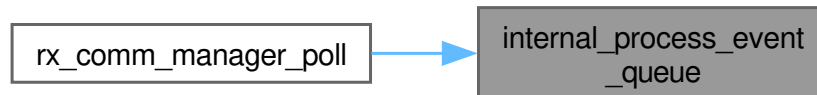
References `rx_comm_event_entry_t::channel`, `rx_comm_manager_t::event_queue`, `rx_comm_manager_t::event_queue_tail`, `rx_comm_event_entry_t::flags`, `internal_dequeue_event()`, `internal_get_time_ms()`, `k_comm_channel_uart`, `k_event_initial_backoff_ms`, `k_event_max_backoff_ms`, `k_event_max_retries`, `rx_comm_event_entry_t::next_retry_time_ms`, `rx_comm_event_entry_t::occupied`, `rx_comm_event_entry_t::payload`, `rx_comm_event_entry_t::payload_len`, `rx_comm_event_entry_t::retries`, `rx_comm_manager_send()`, `s_tag`, and `rx_comm_event_entry_t::type`.

Referenced by `rx_comm_manager_poll()`.

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.13 internal\_route\_send()

```

rx_err_t internal_route_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [static]
  
```

Send frame on specified communication channel.

Routes frame transmission to the appropriate transport layer (USB or SPI) based on specified channel. Supports configurable frame type, flags, and variable-length payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.

Algorithm:

1. Validate parameters (NASA Rule 5):
  - mgr and params pointers non-NULL
  - Manager initialized flag
  - Payload pointer valid if payload\_len > 0
  - payload\_len <= k\_frame\_max\_payload (255 bytes)
2. Route to channel:
  - USB: Verify usb\_handle non-NULL, call rx\_usb\_comm\_send()
  - SPI: Verify spi\_handle non-NULL, call rx\_spi\_comm\_send()
  - Invalid channel: Return k\_rx\_err\_invalid\_arg
3. On success, if ASCII debugging enabled:
  - Build temporary rx\_frame\_t from params
  - Format as ASCII via [internal\\_output\\_decoded\(\)](#)
  - Send to USB Port 1 (TX direction)

Performance:

- USB: ~30-150 us (depends on payload size, USB bulk transfer)
- SPI: ~20-100 us (depends on payload size, SPI hardware transfer)
- ASCII debugging: Add ~50-150 us if enabled



## Precondition

- mgr must be non-NULL and initialized
- params must be non-NULL with all valid fields
- Channel handle (USB/SPI) must be initialized
- If payload\_len > 0, payload must point to valid data

## Postcondition

- Frame transmitted on specified channel if successful
- ASCII output sent to Port 1 if enabled and successful

## Note

- Payload sequence number and CRC-32 managed by transport layer
- ASCII debugging adds overhead - disable in production

## Warning

- Ensure channel is ready before sending (use rx\_comm\_manager\_channel\_ready)

## Send Parameters Structure:

```
typedef struct {
    rx_comm_channel_t channel;    // USB or SPI
    uint8_t type;                // Frame type
    uint8_t flags;               // Frame flags
    const uint8_t* payload;      // Payload data
    uint32_t payload_len;        // Payload length [0, 255]
} rx_comm_send_params_t;
```

## Example - Send Command on USB:

```
uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_command,
    .flags = k_frame_flag_none,
    .payload = cmd_payload,
    .payload_len = sizeof(cmd_payload),
};

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err == k_rx_ok) {
    // Command sent successfully
}
```

## Example - Send Empty Frame (No Payload):

```
rx_comm_send_params_t params = {
    .channel = k_comm_channel_spi,
    .type = k_frame_type_response,
    .flags = k_frame_flag_ack,
    .payload = nullptr,        // No payload
    .payload_len = 0,          // Zero length
};

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
```

See also

[rx\\_comm\\_manager\\_respond\(\)](#) Convenience wrapper for response frames  
[rx\\_comm\\_send\\_params\\_t](#) Send parameters structure  
[rx\\_spi\\_comm\\_send\(\)](#) SPI send implementation  
[rx\\_uart\\_comm\\_send\(\)](#) UART send implementation  
[rx\\_i2c\\_comm\\_send\(\)](#) I2C peripheral send implementation

Since

Version 1.0.0

Route frame to transport layer based on channel

Precondition

mgr and params must be non-NULL  
 params validated by caller (payload length, etc.)

Postcondition

Frame sent on specified channel transport

Since

Version 1.0.0

Definition at line 1511 of file [rx\\_comm\\_manager.c](#).

```

1512 {
1513     switch (params->channel) {
1514     case k_comm_channel_spi:
1515         if (mgr->spi_handle == nullptr) {
1516             rx_log_error(s_tag, "Send failed: SPI handle NULL");
1517             return k_rx_err_invalid_state;
1518         }
1519         /* Use HARQ link layer if available, otherwise raw SPI */
1520         if (mgr->spi_link != nullptr) {
1521             /* NOTE: params->flags intentionally NOT forwarded to rx_spi_link_send().
1522              * The link layer manages its own flags internally (k_frame_flag_requires_ack,
1523              * k_frame_flag_fec_enabled, k_frame_flag_retransmit) based on HARQ state. */
1524             return rx_spi_link_send(mgr->spi_link, params->type, params->payload, params->payload_len);
1525         }
1526         return rx_spi_comm_send(mgr->spi_handle,
1527                                params->type,
1528                                params->flags,
1529                                params->payload,
1530                                params->payload_len);
1531     case k_comm_channel_i2c:
1532         if (mgr->i2c_handle == nullptr) {
1533             rx_log_error(s_tag, "Send failed: I2C handle NULL");
1534             return k_rx_err_invalid_state;
1535         }
1536         return rx_i2c_comm_send(mgr->i2c_handle,
1537                                params->type,
1538                                params->flags,
1539                                params->payload,
1540                                params->payload_len);
1541     case k_comm_channel_uart:
1542         if (mgr->uart_handle == nullptr) {
1543             rx_log_error(s_tag, "Send failed: UART handle NULL");
1544             return k_rx_err_invalid_state;
1545         }
1546         return rx_uart_comm_send(mgr->uart_handle,
1547                                params->type,

```

```

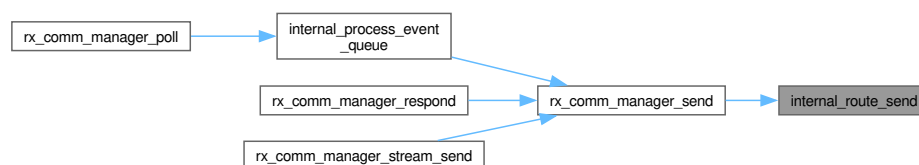
01550             params->flags,
01551             params->payload,
01552             params->payload_len);
01553
01554     default:
01555         rx_log_error(s_tag, "Send failed: invalid channel");
01556         return k_rx_err_invalid_arg;
01557     }
01558 }

```

References [rx\\_comm\\_send\\_params\\_t::channel](#), [rx\\_comm\\_send\\_params\\_t::flags](#), [rx\\_comm\\_manager\\_t::i2c\\_handle](#), [k\\_comm\\_channel\\_i2c](#), [k\\_comm\\_channel\\_spi](#), [k\\_comm\\_channel\\_uart](#), [rx\\_comm\\_send\\_params\\_t::payload](#), [rx\\_comm\\_send\\_params\\_t::payload\\_len](#), [s\\_tag](#), [rx\\_comm\\_manager\\_t::spi\\_handle](#), [rx\\_comm\\_manager\\_t::spi\\_link](#), [rx\\_comm\\_send\\_params\\_t::type](#), and [rx\\_comm\\_manager\\_t::uart\\_handle](#).

Referenced by [rx\\_comm\\_manager\\_send\(\)](#).

Here is the caller graph for this function:



#### 4.3.3.14 rx'comm'manager'channel'name()

```

const char * rx_comm_manager_channel_name (
    rx_comm_channel_t channel)

```

Get human-readable name for communication channel.

Get channel name string.

Returns static string name for given channel enum value. Used for logging, debugging, and user interface display. Always returns a valid string (never NULL).

Returned Strings:

- `k_comm_channel_usb` -> "USB"
- `k_comm_channel_spi` -> "SPI"
- Invalid/unknown -> "UNKNOWN"

Use Cases:

- Logging channel activity
- Debugging frame traffic
- User interface display
- Error messages

## Precondition

None (accepts any input)

## Postcondition

Returns pointer to static string (valid for program lifetime)

## Note

Returned string is static - do not free

Thread-safe (returns read-only static strings)

Never returns NULL - always safe to use in printf

## Example - Logging:

```
static void on_frame_received(rx_comm_channel_t channel,
                             const rx_frame_t* frame,
                             void* ctx)
{
    printf("Frame received on %s: type=%d len=%d\n",
          rx_comm_manager_channel_name(channel),
          frame->header.type,
          frame->header.length);
}
```

## Example - Error Messages:

```
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err != k_rx_ok) {
    fprintf(stderr, "Failed to send on %s: error %d\n",
            rx_comm_manager_channel_name(params.channel), err);
}
```

## See also

[rx\\_comm\\_channel\\_t](#) Channel enumeration

## Since

Version 1.0.0

Definition at line 1946 of file [rx\\_comm\\_manager.c](#).

```
01947 {
01948     switch (channel) {
01949         case k_comm_channel_uart:
01950             return "UART";
01951         case k_comm_channel_spi:
01952             return "SPI";
01953         case k_comm_channel_i2c:
01954             return "I2C";
01955         default:
01956             return "UNKNOWN";
01957     }
01958 }
```

References [k\\_comm\\_channel\\_i2c](#), [k\\_comm\\_channel\\_spi](#), and [k\\_comm\\_channel\\_uart](#).

#### 4.3.3.15 rx'comm'manager'channel'ready()

```
rx_err_t rx_comm_manager_channel_ready (  
    rx_comm_manager_t * mgr,  
    rx_comm_channel_t channel,  
    bool * ready)  [nodiscard]
```

Query if communication channel is ready for send/receive operations.

Check if channel is ready for communication.

Checks if specified channel is configured and ready for communication. USB channel readiness depends on USB enumeration and configuration by host. SPI channel is ready immediately if handle is configured (no enumeration required).

Readiness Criteria:

- USB: usb\_handle non-NULL AND USB device enumerated and configured by host
- SPI: spi\_handle non-NULL (always ready if configured)

Use Cases:

- Check readiness before sending to avoid errors
- Implement fallback logic (try USB, fall back to SPI)
- Log channel availability status

Precondition

mgr must be non-NULL and initialized  
ready must be non-NULL  
channel must be valid (USB or SPI)

Postcondition

\*ready set to true or false based on channel state  
On error, \*ready set to false (safe default)

Note

USB readiness can change at runtime (host connects/disconnects)  
SPI readiness is static (always ready if handle configured)  
This function does NOT initiate communication - only queries state

Example - Send with Fallback:

```
uint8_t payload[16] = { ... };

// Try USB first
bool usb_ready = false;
rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                             k_comm_channel_usb,
                                             &usb_ready);
if (err == k_rx_ok && usb_ready) {
    err = rx_comm_manager_respond(&g_comm_mgr,
                                  k_comm_channel_usb,
                                  payload,
                                  sizeof(payload));
} else {
    // Fallback to SPI
    bool spi_ready = false;
    err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                       k_comm_channel_spi,
                                       &spi_ready);
    if (err == k_rx_ok && spi_ready) {
        err = rx_comm_manager_respond(&g_comm_mgr,
                                      k_comm_channel_spi,
                                      payload,
                                      sizeof(payload));
    }
}
```

Example - Log Channel Status:

```
void log_channel_status(void)
{
    bool usb_ready = false, spi_ready = false;

    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_usb, &usb_ready);
    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_spi, &spi_ready);

    printf("USB: %s, SPI: %s\n",
          usb_ready ? "READY" : "NOT READY",
          spi_ready ? "READY" : "NOT READY");
}
```

See also

[rx\\_usb\\_is\\_configured\(\)](#) USB enumeration status  
[rx\\_comm\\_manager\\_send\(\)](#) Send frame on channel

Since

Version 1.0.0

Definition at line 1857 of file [rx\\_comm\\_manager.c](#).

```
01858 {
01859     /* Rule 5: Pre-condition validation */
01860     if (mgr == nullptr || ready == nullptr) {
01861         if (ready != nullptr) {
01862             *ready = false;
01863         }
01864         return k_rx_err_invalid_arg;
01865     }
01866     if (!mgr->initialized) {
01867         *ready = false;
01868         return k_rx_err_invalid_state;
01869     }
01870
01871     switch (channel) {
01872     case k_comm_channel_uart:
01873         *ready = (bool)(mgr->uart_handle != nullptr);
01874         break;
01875
01876     case k_comm_channel_spi:
01877         *ready = (bool)(mgr->spi_handle != nullptr);
01878         break;
```

```

01879
01880     case k_comm_channel_i2c:
01881         *ready = (bool)(mgr->i2c_handle != nullptr);
01882         break;
01883
01884     default:
01885         *ready = false;
01886         rx_log_error(s_tag, "Channel ready: invalid channel");
01887         return k_rx_err_invalid_arg;
01888     }
01889
01890     return k_rx_ok;
01891 }

```

References `rx_comm_manager_t::i2c_handle`, `rx_comm_manager_t::initialized`, `k_comm_channel_i2c`, `k_comm_channel_spi`, `k_comm_channel_uart`, `s_tag`, `rx_comm_manager_t::spi_handle`, and `rx_comm_manager_t::uart_handle`.

#### 4.3.3.16 rx'comm'manager'deinit()

```

rx_err_t rx_comm_manager_deinit (
    rx_comm_manager_t * mgr) [nodiscard]

```

Deinitialize communication manager and release resources.

Deinitialize communication manager.

Marks communication manager as uninitialized, preventing further operations. This function does NOT deinitialize USB or SPI transport handles - caller must separately call `rx_usb_comm_deinit()` and `rx_spi_comm_deinit()` if needed.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated). Function primarily serves as safety guard to prevent use-after-deinit.

Precondition

mgr must be non-NULL  
mgr must be initialized (mgr->initialized == true)

Postcondition

mgr->initialized = false  
Subsequent calls to `rx_comm_manager_poll/send` will return `k_rx_err_invalid_state`

Note

Does NOT free memory (handle is statically allocated)  
Does NOT deinitialize transport handles (USB/SPI)  
Thread-safe if no concurrent operations on same mgr handle

Example:

```
// Deinitialize manager
rx_err_t err = rx_comm_manager_deinit(&g_comm_mgr);
if (err == k_rx_ok) {
    // Manager no longer usable

    // Separately deinitialize transports if needed
    rx_usb_comm_deinit(&g_usb_handle);
    rx_spi_comm_deinit(&g_spi_handle);
}
```

See also

[rx\\_comm\\_manager\\_init\(\)](#) Initialization  
[rx\\_spi\\_comm\\_deinit\(\)](#) SPI transport cleanup

Since

Version 1.0.0

Definition at line 1216 of file [rx\\_comm\\_manager.c](#).

```
01217 {
01218     /* Rule 5: Pre-condition validation */
01219     if (mgr == nullptr) {
01220         rx_log_error(s_tag, "Deinit failed: NULL mgr");
01221         return k_rx_err_invalid_arg;
01222     }
01223     if (!mgr->initialized) {
01224         rx_log_warn(s_tag, "Deinit called on uninitialized mgr");
01225         return k_rx_err_invalid_state;
01226     }
01227
01228     mgr->initialized = false;
01229
01230     rx_log_info(s_tag, "Comm manager deinitialized");
01231     return k_rx_ok;
01232 }
```

References [rx\\_comm\\_manager\\_t::initialized](#), and [s\\_tag](#).

#### 4.3.3.17 rx'comm'manager'event'send()

```
rx_err_t rx_comm_manager_event_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Queue data on the event path (reliable, SPI retry).

Event path is for commands and other data requiring reliable delivery. The payload is copied into a static FIFO queue and processed by the poll loop. Retry behavior depends on transport:

- SPI: Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
- USB: Fire-and-forget (USB HW reliability is sufficient)



Queue is processed in FIFO order by [rx\\_comm\\_manager\\_poll\(\)](#).

#### Parameters

|        |        |                                                              |
|--------|--------|--------------------------------------------------------------|
| in,out | mgr    | Pointer to initialized manager                               |
| in     | params | Send parameters (channel, type, flags, payload, payload_len) |

#### Returns

k\_rx\_ok on success (queued)  
 k\_rx\_err\_invalid\_arg if mgr or params is nullptr  
 k\_rx\_err\_invalid\_state if not initialized  
 k\_rx\_err\_no\_mem if event queue is full

#### Precondition

mgr must be initialized  
 params must be valid  
 params->payload\_len <= k\_frame\_max\_payload

#### Postcondition

Payload copied to queue, will be sent by next poll()

#### Note

Payload is COPIED into queue - caller can reuse buffer immediately

#### Warning

Queue depth is k\_comm\_event\_queue\_depth (8). If full, returns error.

#### See also

[rx\\_comm\\_manager\\_stream\\_send\(\)](#) For fire-and-forget telemetry

#### Since

Version 1.0.0

Definition at line 1698 of file [rx\\_comm\\_manager.c](#).

```

01699 {
01700   if (mgr == nullptr || params == nullptr) {
01701     rx_log_error(s_tag, "Event enqueue failed: NULL arg");
01702     return k_rx_err_invalid_arg;
01703   }
01704
01705   if (!mgr->initialized) {
01706     rx_log_error(s_tag, "Event enqueue failed: not initialized");
01707     return k_rx_err_invalid_state;
01708   }
01709
01710   if (params->payload_len > k_frame_max_payload) {

```

```

01711 rx_log_error_val(s_tag, "Event enqueue failed: payload too large", params->payload_len);
01712 return k_rx_err_invalid_arg;
01713 }
01714
01715 if (params->payload_len > 0 && params->payload == nullptr) {
01716 rx_log_error(s_tag, "Event enqueue failed: NULL payload with nonzero len");
01717 return k_rx_err_invalid_arg;
01718 }
01719
01720 /* Check queue capacity */
01721 if (mgr->event_queue_count >= k_comm_event_queue_depth) {
01722 rx_log_error_val(s_tag, "Event queue full, depth", (uint8_t)mgr->event_queue_count);
01723 return k_rx_err_no_mem;
01724 }
01725
01726 /* Enqueue: copy payload into static slot */
01727 rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_head];
01728 entry->channel = params->channel;
01729 entry->type = params->type;
01730 entry->flags = params->flags;
01731 entry->payload_len = params->payload_len;
01732 entry->retries = 0;
01733 entry->occupied = true;
01734
01735 if (params->payload != nullptr && params->payload_len > 0) {
01736 for (uint32_t i = 0; i < params->payload_len; i++) {
01737 entry->payload[i] = params->payload[i];
01738 }
01739 }
01740
01741 mgr->event_queue_head = (mgr->event_queue_head + 1) % k_comm_event_queue_depth;
01742 mgr->event_queue_count++;
01743
01744 rx_log_debug_val(s_tag, "Event enqueued, queue depth", mgr->event_queue_count);
01745 return k_rx_ok;
01746 }

```

References [rx\\_comm\\_event\\_entry\\_t::channel](#), [rx\\_comm\\_send\\_params\\_t::channel](#), [rx\\_comm\\_manager\\_t::event\\_queue](#), [rx\\_comm\\_manager\\_t::event\\_queue\\_count](#), [rx\\_comm\\_manager\\_t::event\\_queue\\_head](#), [rx\\_comm\\_event\\_entry\\_t::flags](#), [rx\\_comm\\_send\\_params\\_t::flags](#), [rx\\_comm\\_manager\\_t::initialized](#), [k\\_comm\\_event\\_queue\\_depth](#), [rx\\_comm\\_event\\_entry\\_t::occupied](#), [rx\\_comm\\_event\\_entry\\_t::payload](#), [rx\\_comm\\_send\\_params\\_t::payload](#), [rx\\_comm\\_event\\_entry\\_t::payload\\_len](#), [rx\\_comm\\_send\\_params\\_t::payload\\_len](#), [rx\\_comm\\_event\\_entry\\_t::retries](#), [s\\_tag](#), [rx\\_comm\\_event\\_entry\\_t::type](#), and [rx\\_comm\\_send\\_params\\_t::type](#).

#### 4.3.3.18 rx\_comm\_manager\_init()

```

rx_err_t rx_comm_manager_init (
    rx_comm_manager_t * mgr,
    const rx_comm_manager_config_t * cfg) [nodiscard]

```

Initialize communication manager with channel configuration.

Initialize communication manager.

Performs complete initialization of communication manager handle, configuring enabled channels, callback registration, and ASCII debugging output. This function must be called before any other communication manager operations.

Algorithm:

1. Validate parameters: Check mgr pointer for nullptr (NASA Rule 5)
2. Clear handle: Zero-fill entire structure via memset (328 bytes)
3. Apply configuration: Copy config fields if cfg is non-NULL
  - usb\_handle: Pointer to initialized USB comm handle (may be NULL)
  - spi\_handle: Pointer to initialized SPI comm handle (may be NULL)

- callback: User frame handler function (may be NULL)
  - callback\_ctx: User context pointer (may be NULL)
  - enable\_decoded\_output: ASCII debugging flag
4. Mark initialized: Set mgr->initialized = true
  5. Post-condition validation: Verify ascii\_buffer properly cleared

#### Configuration Options:

- cfg = NULL: All channels disabled, no callback, ASCII output off
- usb\_handle = NULL: USB channel disabled (poll skips USB)
- spi\_handle = NULL: SPI channel disabled (poll skips SPI)
- callback = NULL: No callback invoked (silent frame consumption)
- enable\_decoded\_output = true: ASCII debugging enabled (Port 1)

#### Precondition

mgr must point to allocated memory (uninitialized content OK)  
USB/SPI handles (if provided) must already be initialized  
Callback function (if provided) must be valid and reentrant

#### Postcondition

mgr->initialized = true on success  
All mgr fields set to config values or zero  
mgr->ascii\_buffer is zero-filled (256 bytes)

#### Note

This function does NOT initialize USB or SPI transports - caller must initialize rx\_usb\_comm and rx\_spi\_comm separately before passing handles to this function

On success, at least one channel should be enabled (usb\_handle or spi\_handle non-NULL)

Thread-safe ONLY if each thread uses a different mgr handle

#### Performance:

- Execution time: ~5 us @ 240 MHz
- Dominated by memset (328 bytes)

## Configuration Example - Both Channels Enabled:

```

static rx_comm_manager_t g_comm_mgr;
static rx_usb_comm_handle_t g_usb_handle;
static rx_spi_comm_handle_t g_spi_handle;

// Initialize transport layers first
rx_usb_comm_config_t usb_cfg = { .port = k_usb_port_proto };
rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
if (err != k_rx_ok) return err;

rx_spi_comm_config_t spi_cfg = { .spi_channel = 0 };
err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
if (err != k_rx_ok) return err;

// Initialize communication manager with both channels
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = &g_spi_handle,
    .callback = on_frame_received,
    .callback_ctx = nullptr,
    .enable_decoded_output = true,
};
err = rx_comm_manager_init(&g_comm_mgr, &cfg);

```

## Configuration Example - USB Only, No Callback:

```

rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = nullptr,           // SPI disabled
    .callback = nullptr,           // No callback
    .callback_ctx = nullptr,
    .enable_decoded_output = false, // No ASCII debugging
};
rx_err_t err = rx_comm_manager_init(&g_comm_mgr, &cfg);

```

See also

[rx\\_comm\\_manager\\_deinit\(\)](#) Cleanup and resource release  
[rx\\_comm\\_manager\\_poll\(\)](#) Poll for incoming frames  
[rx\\_spi\\_comm\\_init\(\)](#) Initialize SPI transport  
[rx\\_uart\\_comm\\_init\(\)](#) Initialize UART transport  
[rx\\_i2c\\_comm\\_init\(\)](#) Initialize I2C peripheral transport

Since

Version 1.0.0

Definition at line 1120 of file [rx\\_comm\\_manager.c](#).

```

01121 {
01122     /* Rule 5: Pre-condition validation - nullptr check */
01123     if (mgr == nullptr) {
01124         rx_log_error(s_tag, "Init failed: NULL mgr");
01125         return k_rx_err_invalid_arg;
01126     }
01127
01128     /* Rule 5: Pre-condition validation - SPI link consistency checks */
01129     if (cfg != nullptr && cfg->spi_link != nullptr) {
01130         /* If spi_link is provided, spi_handle must also be provided */
01131         if (cfg->spi_handle == nullptr) {
01132             rx_log_error(s_tag, "Init failed: spi_link requires non-NULL spi_handle");
01133             return k_rx_err_invalid_arg;
01134         }
01135
01136         /* If spi_link is provided, it must be initialized */
01137         if (!cfg->spi_link->initialized) {
01138             rx_log_error(s_tag, "Init failed: spi_link not initialized (call rx_spi_link_init first)");
01139             return k_rx_err_invalid_arg;
01140         }
01141     }

```

```

01142  /* If spi_link is provided, its spi_handle must match config spi_handle */
01143  if (cfg->spi_link->spi_handle != cfg->spi_handle) {
01144      rx_log_error(s_tag, "Init failed: spi_link->spi_handle != cfg->spi_handle");
01145      return k_rx_err_invalid_arg;
01146  }
01147  }
01148
01149  *mgr = s_zero_mgr;
01150
01151  /* Apply configuration */
01152  if (cfg != nullptr) {
01153      mgr->uart_handle      = cfg->uart_handle;
01154      mgr->spi_handle       = cfg->spi_handle;
01155      mgr->i2c_handle       = cfg->i2c_handle;
01156      mgr->spi_link        = cfg->spi_link;
01157      mgr->callback         = cfg->callback;
01158      mgr->callback_ctx     = cfg->callback_ctx;
01159      mgr->enable_decoded_output = cfg->enable_decoded_output;
01160      mgr->link_status_cb   = cfg->link_status_cb;
01161      mgr->link_status_ctx  = cfg->link_status_ctx;
01162  }
01163
01164  mgr->initialized = true;
01165
01166  rx_log_info(s_tag, "Comm manager initialized");
01167  return k_rx_ok;
01168 }

```

References [rx\\_comm\\_manager\\_config\\_t::callback](#), [rx\\_comm\\_manager\\_t::callback](#), [rx\\_comm\\_manager\\_config\\_t::callback\\_ctx](#), [rx\\_comm\\_manager\\_t::callback\\_ctx](#), [rx\\_comm\\_manager\\_config\\_t::enable\\_decoded\\_output](#), [rx\\_comm\\_manager\\_t::enable\\_decoded\\_output](#), [rx\\_comm\\_manager\\_config\\_t::i2c\\_handle](#), [rx\\_comm\\_manager\\_t::i2c\\_handle](#), [rx\\_comm\\_manager\\_t::initialized](#), [rx\\_comm\\_manager\\_config\\_t::link\\_status\\_cb](#), [rx\\_comm\\_manager\\_t::link\\_status\\_cb](#), [rx\\_comm\\_manager\\_config\\_t::link\\_status\\_ctx](#), [rx\\_comm\\_manager\\_t::link\\_status\\_ctx](#), [s\\_tag](#), [s\\_zero\\_mgr](#), [rx\\_comm\\_manager\\_config\\_t::spi\\_handle](#), [rx\\_comm\\_manager\\_t::spi\\_handle](#), [rx\\_comm\\_manager\\_config\\_t::spi\\_link](#), [rx\\_comm\\_manager\\_t::spi\\_link](#), [rx\\_comm\\_manager\\_config\\_t::uart\\_handle](#), and [rx\\_comm\\_manager\\_t::uart\\_handle](#).

#### 4.3.3.19 rx'comm'manager'link'status()

```

rx_err_t rx_comm_manager_link_status (
    const rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    rx_comm_link_status_t * status) [nodiscard]

```

Query link health status for a transport channel.

Returns the current heartbeat-derived link status for the given channel. Status is updated by [rx\\_comm\\_manager\\_poll\(\)](#) based on the dual-detection heartbeat model (200ms implicit timeout).

##### Parameters

|     |         |                                |
|-----|---------|--------------------------------|
| in  | mgr     | Pointer to initialized manager |
| in  | channel | Channel to query               |
| out | status  | Receives current link status   |

##### Returns

- k\_rx\_ok on success
- k\_rx\_err\_invalid\_arg if mgr or status is nullptr

Since

Version 1.0.0

Definition at line 1753 of file `rx_comm_manager.c`.

```

01756 {
01757     if (mgr == nullptr || status == nullptr) {
01758         rx_log_error(s_tag, "Link status query: NULL arg");
01759         return k_rx_err_invalid_arg;
01760     }
01761
01762     if (!mgr->initialized) {
01763         rx_log_error(s_tag, "Link status query: uninitialized manager");
01764         return k_rx_err_invalid_state;
01765     }
01766
01767     if (channel >= k_comm_channel_count) {
01768         rx_log_error(s_tag, "Link status query: invalid channel");
01769         return k_rx_err_invalid_arg;
01770     }
01771
01772     *status = mgr->heartbeat[channel].status;
01773     return k_rx_ok;
01774 }
```

References `rx_comm_manager_t::heartbeat`, `rx_comm_manager_t::initialized`, `k_comm_channel_count`, `s_tag`, and `rx_comm_heartbeat_state_t::status`.

#### 4.3.3.20 rx'comm'manager'poll()

```

rx_err_t rx_comm_manager_poll (
    rx_comm_manager_t * mgr) [nodiscard]
```

Poll all enabled channels for incoming frames.

Non-blocking check of USB and SPI channels. For each complete frame received:

1. If decoded output enabled, format and send ASCII to Port 1
2. Invoke user callback with frame data and channel

Call this from main loop or dedicated communication task.

#### Parameters

|        |     |                                |
|--------|-----|--------------------------------|
| in,out | mgr | Pointer to initialized manager |
|--------|-----|--------------------------------|

#### Returns

- `k_rx_ok` if at least one frame was received
- `k_rx_err_timeout` if no frames available on any channel
- `k_rx_err_invalid_arg` if mgr is nullptr
- `k_rx_err_invalid_state` if not initialized

Definition at line 1383 of file `rx_comm_manager.c`.

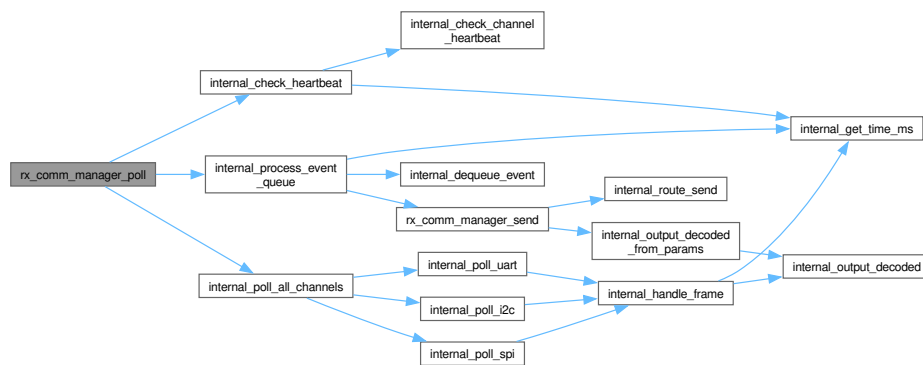
```

01384 {
01385     /* Rule 5: Pre-condition validation */
01386     if (mgr == nullptr) {
01387         return k_rx_err_invalid_arg;
01388     }
01389     if (!mgr->initialized) {
01390         return k_rx_err_invalid_state;
01391     }
01392
01393     bool    received = false;
01394     rx_err_t poll_err = internal_poll_all_channels(mgr, &received);
01395     if (poll_err != k_rx_ok) {
01396         return poll_err;
01397     }
01398
01399     /* Process event queue: send one queued event per poll cycle */
01400     internal_process_event_queue(mgr);
01401
01402     /* Heartbeat: check implicit timeout on each transport */
01403     internal_check_heartbeat(mgr);
01404
01405     return (int)received ? k_rx_ok : k_rx_err_timeout;
01406 }

```

References `rx_comm_manager_t::initialized`, `internal_check_heartbeat()`, `internal_poll_all_channels()`, and `internal_process_event_queue()`.

Here is the call graph for this function:



#### 4.3.3.21 rx'comm'manager'process'retransmits()

```

rx_err_t rx_comm_manager_process_retransmits (
    rx_comm_manager_t * mgr,
    uint32_t current_time_ms) [nodiscard]

```

Process pending retransmissions on SPI channel.

Thin pass-through to `rx_spi_comm_process_retransmits()` on the SPI handle. Call this from the communication task polling loop.

Parameters

|        |                 |                                     |
|--------|-----------------|-------------------------------------|
| in,out | mgr             | Communication manager               |
| in     | current_time_ms | Current system time in milliseconds |

## Returns

`rx_err_t` Error code from `rx_spi_comm_process_retransmits()`

## Return values

|                                     |                                     |
|-------------------------------------|-------------------------------------|
| <code>k_rx_ok</code>                | No action needed or retransmit sent |
| <code>k_rx_err_invalid_arg</code>   | mgr is nullptr                      |
| <code>k_rx_err_invalid_state</code> | mgr not initialized                 |
| <code>k_rx_err_retry_limit</code>   | Max retries exceeded                |

## Since

Version 1.0.0

Definition at line 1965 of file `rx_comm_manager.c`.

```

01966 {
01967     if (mgr == nullptr) {
01968         return k_rx_err_invalid_arg;
01969     }
01970
01971     if (!mgr->initialized) {
01972         return k_rx_err_invalid_state;
01973     }
01974
01975     if (mgr->spi_handle == nullptr) {
01976         return k_rx_ok; /* No SPI channel configured */
01977     }
01978
01979     return rx_spi_comm_process_retransmits(mgr->spi_handle, current_time_ms);
01980 }

```

References `rx_comm_manager_t::initialized`, and `rx_comm_manager_t::spi_handle`.

4.3.3.22 `rx_comm_manager_respond()`

```

rx_err_t rx_comm_manager_respond (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    const uint8_t * payload,
    uint32_t payload_len) [nodiscard]

```

Send response frame on specified channel (convenience wrapper).

Send response on same channel as received frame.

Convenience function for sending response frames. Automatically sets frame type to `k_frame_type_response` and flags to `k_frame_flag_none`. Simplifies common use case of responding to commands/requests.

## Algorithm:

1. Build `rx_comm_send_params_t` with:
  - channel: User-specified
  - type: `k_frame_type_response` (fixed)
  - flags: `k_frame_flag_none` (fixed)
  - payload: User-specified



- payload\_len: User-specified
2. Call `rx_comm_manager_send()` with constructed params
  3. Return result from `rx_comm_manager_send()`

When to Use:

- Responding to commands with telemetry data
- Sending acknowledgments
- Replying to requests

When NOT to Use:

- Need custom frame type (use `rx_comm_manager_send` instead)
- Need custom flags (use `rx_comm_manager_send` instead)

Precondition

Same preconditions as `rx_comm_manager_send()`

Postcondition

Response frame transmitted with type=response, flags=none

Note

Equivalent to calling `rx_comm_manager_send()` with type=k\_frame\_type\_response

This is a thin wrapper - no additional validation performed

Example - Send Telemetry Response:

```
// Received a telemetry request on USB, send response
static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
{
    uint8_t telemetry[16];

    // Gather telemetry data
    telemetry[0] = get_temperature_celsius();
    telemetry[1] = get_current_ma();
    // ... fill remaining fields ...

    // Send response on same channel request came from
    rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                           channel,
                                           telemetry,
                                           sizeof(telemetry));

    if (err != k_rx_ok) {
        // Handle send error
    }
}
```

Example - Send Empty Acknowledgment:

```
// Send empty ACK response
rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                       k_comm_channel_usb,
                                       NULL, // No payload
                                       0);   // Zero length
```

See also

[rx\\_comm\\_manager\\_send\(\)](#) Full send function with all parameters

[rx\\_comm\\_send\\_params\\_t](#) Send parameters structure

Since

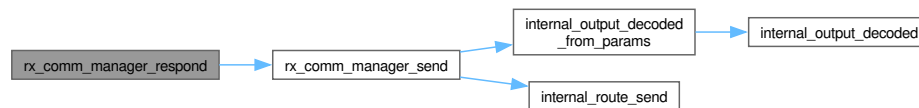
Version 1.0.0

Definition at line 1670 of file [rx\\_comm\\_manager.c](#).

```
01674 {
01675     const rx_comm_send_params_t params = {
01676         .channel    = channel,
01677         .type       = k_frame_type_response,
01678         .flags      = k_frame_flag_none,
01679         .payload    = payload,
01680         .payload_len = payload_len,
01681     };
01682     return rx_comm_manager_send(mgr, &params);
01683 }
```

References [rx\\_comm\\_manager\\_send\(\)](#).

Here is the call graph for this function:



#### 4.3.3.23 rx'comm'manager'send()

```
rx_err_t rx_comm_manager_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send frame on specific channel.

Encodes and sends a frame on the specified channel. If decoded output is enabled, also sends ASCII representation to Port 1.

Parameters

|        |        |                                                              |
|--------|--------|--------------------------------------------------------------|
| in,out | mgr    | Pointer to initialized manager                               |
| in     | params | Send parameters (channel, type, flags, payload, payload_len) |

## Returns

[k\\_rx\\_ok](#) on success  
[k\\_rx\\_err\\_invalid\\_arg](#) if mgr or payload is nullptr  
[k\\_rx\\_err\\_invalid\\_state](#) if not initialized or channel not enabled

Definition at line 1560 of file [rx\\_comm\\_manager.c](#).

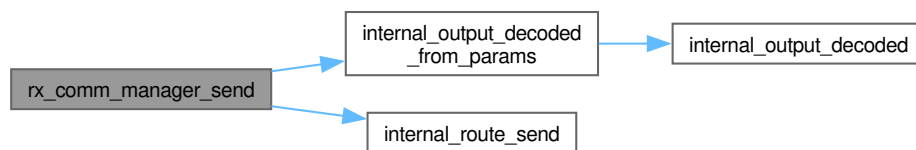
```

01561 {
01562     /* Rule 5: Pre-condition validation */
01563     if (mgr == nullptr || params == nullptr) {
01564         rx_log_error(s_tag, "Send failed: NULL arg");
01565         return k_rx_err_invalid_arg;
01566     }
01567     if (!mgr->initialized) {
01568         rx_log_error(s_tag, "Send failed: not initialized");
01569         return k_rx_err_invalid_state;
01570     }
01571     if (params->payload == nullptr && params->payload_len > 0) {
01572         rx_log_error(s_tag, "Send failed: NULL payload with nonzero len");
01573         return k_rx_err_invalid_arg;
01574     }
01575     /* Validate payload length is within frame limits */
01576     if (params->payload_len > k_frame_max_payload) {
01577         rx_log_error_val(s_tag, "Send failed: payload too large", params->payload_len);
01578         return k_rx_err_invalid_arg;
01579     }
01580
01581     rx_err_t err = internal_route_send(mgr, params);
01582
01583     if (err != k_rx_ok) {
01584         rx_log_error_val(s_tag, "Transport send failed", err);
01585     }
01586
01587     /* Output decoded ASCII on success */
01588     if (err == k_rx_ok && (int)mgr->enable_decoded_output) {
01589         internal_output_decoded_from_params(mgr, params);
01590     }
01591
01592     return err;
01593 }
  
```

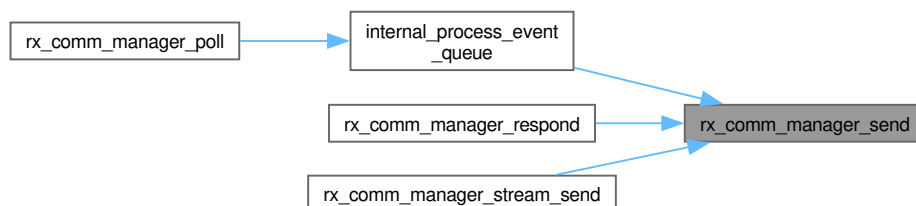
References [rx\\_comm\\_manager\\_t::enable\\_decoded\\_output](#), [rx\\_comm\\_manager\\_t::initialized](#), [internal\\_output\\_decoded\\_from\\_params\(\)](#), [internal\\_route\\_send\(\)](#), [rx\\_comm\\_send\\_params\\_t::payload](#), [rx\\_comm\\_send\\_params\\_t::payload\\_len](#), and [s\\_tag](#).

Referenced by [internal\\_process\\_event\\_queue\(\)](#), [rx\\_comm\\_manager\\_respond\(\)](#), and [rx\\_comm\\_manager\\_stream\\_send\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 4.3.3.24 rx\_comm\_manager\_set\_auto\_retransmit()

```
rx_err_t rx_comm_manager_set_auto_retransmit (
    rx_comm_manager_t * mgr,
    bool enabled,
    const rx_spi_comm_retransmit_config_t * config)  [nodiscard]
```

Enable or disable automatic retransmission on SPI channel.

Thin pass-through to `rx_spi_comm_set_auto_retransmit()` on the SPI handle.

##### Parameters

|        |         |                                                   |
|--------|---------|---------------------------------------------------|
| in,out | mgr     | Communication manager                             |
| in     | enabled | true to enable, false to disable                  |
| in     | config  | Retransmit config (NULL to keep current/defaults) |

##### Returns

rx\_err\_t Error code from `rx_spi_comm_set_auto_retransmit()`

##### Since

Version 1.0.0

Definition at line 1982 of file `rx_comm_manager.c`.

```
01985 {
01986     if (mgr == nullptr) {
01987         return k_rx_err_invalid_arg;
01988     }
01989
01990     if (!mgr->initialized) {
01991         return k_rx_err_invalid_state;
01992     }
01993
01994     if (mgr->spi_handle == nullptr) {
01995         return k_rx_err_not_supported;
01996     }
01997     return rx_spi_comm_set_auto_retransmit(mgr->spi_handle, enabled, config);
01999 }
```

References `rx_comm_manager_t::initialized`, and `rx_comm_manager_t::spi_handle`.

#### 4.3.3.25 rx\_comm\_manager\_stream\_send()

```
rx_err_t rx_comm_manager_stream_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params)  [nodiscard]
```

Send data on the stream path (fire-and-forget).

Stream path is for time-sensitive data like telemetry where stale data has no value. Sends immediately with NO retries on any transport. If the send fails, the data is simply dropped.

- USB CDC: Direct send, no retries (USB HW provides reliability)

- SPI: Direct send, no retries (stale telemetry is worthless)

## Parameters

|        |        |                                                              |
|--------|--------|--------------------------------------------------------------|
| in,out | mgr    | Pointer to initialized manager                               |
| in     | params | Send parameters (channel, type, flags, payload, payload_len) |

## Returns

k\_rx\_ok on success  
 k\_rx\_err\_invalid\_arg if mgr or params is nullptr  
 k\_rx\_err\_invalid\_state if not initialized or channel not enabled

## Precondition

mgr must be initialized  
 params must be valid

## Postcondition

Data sent or silently dropped on failure

## Note

This is a thin wrapper over [rx\\_comm\\_manager\\_send\(\)](#) for semantic clarity

## See also

[rx\\_comm\\_manager\\_event\\_send\(\)](#) For reliable command delivery

## Since

Version 1.0.0

Definition at line 1690 of file [rx\\_comm\\_manager.c](#).

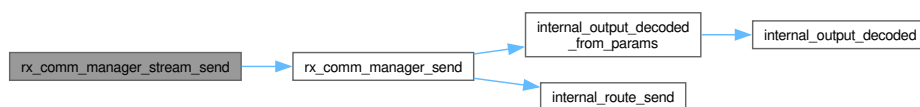
```

01691 {
01692     /* Stream path: fire-and-forget, no retries, no queuing.
01693      * Telemetry and other time-sensitive data goes here.
01694      * Stale data has no value, so failure is silently accepted. */
01695     return rx_comm_manager_send(mgr, params);
01696 }

```

References [rx\\_comm\\_manager\\_send\(\)](#).

Here is the call graph for this function:



### 4.3.4 Variable Documentation

#### 4.3.4.1 s`tag

```
const char s_tag[] = "COMM_MGR" [static]
```

Definition at line 410 of file [rx\\_comm\\_manager.c](#).

Referenced by [internal\\_check\\_channel\\_heartbeat\(\)](#), [internal\\_handle\\_frame\(\)](#), [internal\\_poll\\_all\\_channels\(\)](#), [internal\\_poll\\_uart\(\)](#), [internal\\_process\\_event\\_queue\(\)](#), [internal\\_route\\_send\(\)](#), [rx\\_comm\\_manager\\_channel\\_ready\(\)](#), [rx\\_comm\\_manager\\_deinit\(\)](#), [rx\\_comm\\_manager\\_event\\_send\(\)](#), [rx\\_comm\\_manager\\_init\(\)](#), [rx\\_comm\\_manager\\_link](#) and [rx\\_comm\\_manager\\_send\(\)](#).

#### 4.3.4.2 s`zero`mgr

```
const rx_comm_manager_t s_zero_mgr = {} [static]
```

Zero-initialized comm manager template for stack-safe initialization.

Static const instance used to clear [rx\\_comm\\_manager\\_t](#) without creating a large compound literal on the stack. Lives in .rodata section.

Note

Read-only; never modified after static initialization

Warning

Do not remove - prevents stack overflow in init function

See also

[rx\\_comm\\_manager\\_init\(\)](#) Uses this template to zero-initialize the manager handle

Since

Version 1.0.0

Definition at line 1026 of file [rx\\_comm\\_manager.c](#).

```
01026 {};
```

Referenced by [rx\\_comm\\_manager\\_init\(\)](#).

## 4.4 rx\_comm\_manager.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_comm_manager.c
00003  * @brief Unified Multi-Channel Communication Manager Implementation
00004  *
00005  * @details
00006  * ## Overview
00007  * Production-quality implementation of unified communication manager coordinating
00008  * multiple simultaneous channels (USB CDC and SPI) with automatic frame protocol
00009  * handling, non-blocking polling architecture, optional decoded ASCII debugging
00010  * output, and callback-based event notification.
00011  *
00012  * **Purpose**: Provide centralized, efficient coordination of independent communication
00013  * channels between RX72N MCU and Raspberry Pi 5 host, abstracting low-level transport
00014  * details while maintaining high performance and reliability.
00015  *
00016  * ## Implementation Architecture
00017  *
00018  * @dot
00019  * digraph comm_manager_impl {
00020  *   rankdir=TB;
00021  *   node [shape=box, style=rounded];
00022  *
00023  *   subgraph cluster_public {
00024  *     label="Public API Layer";
00025  *     style=filled;
00026  *     fillcolor=lightblue;
00027  *     init [label="rx_comm_manager_init()"];
00028  *     poll [label="rx_comm_manager_poll()"];
00029  *     send [label="rx_comm_manager_send()"];
00030  *     respond [label="rx_comm_manager_respond()"];
00031  *     ready [label="rx_comm_manager_channel_ready()"];
00032  *   }
00033  *
00034  *   subgraph cluster_private {
00035  *     label="Private Helper Layer";
00036  *     style=filled;
00037  *     fillcolor=lightgreen;
00038  *     poll_usb [label="internal_poll_usb()"];
00039  *     poll_spi [label="internal_poll_spi()"];
00040  *     handle_frame [label="internal_handle_frame()"];
00041  *     output_decoded [label="internal_output_decoded()"];
00042  *   }
00043  *
00044  *   subgraph cluster_transport {
00045  *     label="Transport Layer";
00046  *     style=filled;
00047  *     fillcolor=lightyellow;
00048  *     usb_comm [label="rx_usb_comm\n(USB CDC Protocol)"];
00049  *     spi_comm [label="rx_spi_comm\n(SPI Protocol)"];
00050  *     usb_hw [label="rx_usb\n(USB Port 1 Debug)"];
00051  *     frame_ascii [label="rx_frame_ascii\n(ASCII Formatting)"];
00052  *   }
00053  *
00054  *   // Public API to Private Helpers
00055  *   poll -> poll_usb;
00056  *   poll -> poll_spi;
00057  *   send -> output_decoded;
00058  *
00059  *   // Private Helpers to Transport
00060  *   poll_usb -> usb_comm [label="rx_usb_comm_receive()"];
00061  *   poll_spi -> spi_comm [label="rx_spi_comm_receive()"];
00062  *   poll_usb -> handle_frame [label="on frame"];
00063  *   poll_spi -> handle_frame [label="on frame"];
00064  *   handle_frame -> output_decoded;
00065  *   output_decoded -> frame_ascii [label="rx_frame_ascii_format()"];
00066  *   output_decoded -> usb_hw [label="rx_usb_puts()"];
00067  *   send -> usb_comm [label="rx_usb_comm_send()"];
00068  *   send -> spi_comm [label="rx_spi_comm_send()"];
00069  *
00070  *   // User callback
00071  *   handle_frame -> callback [label="user callback", style=dashed];
00072  *   callback [label="Application\nFrame Handler", shape=ellipse, fillcolor=pink, style=filled];
00073  * }
00074  * @enddot
00075  *
00076  * ## Polling State Machine
00077  *
00078  * The rx_comm_manager_poll() function implements a non-blocking state machine
00079  * that checks all enabled channels sequentially:
00080  *
00081  * @startuml
00082  * [*] --> Idle

```

```

00083 * Idle --> PollUSB : rx_comm_manager_poll() called
00084 *
00085 * state PollUSB {
00086 *     [*] --> CheckUSBHandle
00087 *     CheckUSBHandle --> USBReceive : handle != nullptr
00088 *     CheckUSBHandle --> SkipUSB : handle == nullptr
00089 *     USBReceive --> HandleUSBFrame : frame received
00090 *     USBReceive --> USBTimeout : no data
00091 *     HandleUSBFrame --> [*] : mark received=true
00092 *     USBTimeout --> [*] : continue
00093 *     SkipUSB --> [*] : continue
00094 * }
00095 *
00096 * PollUSB --> PollSPI : USB polling complete
00097 *
00098 * state PollSPI {
00099 *     [*] --> CheckSPIHandle
00100 *     CheckSPIHandle --> SPIReceive : handle != nullptr
00101 *     CheckSPIHandle --> SkipSPI : handle == nullptr
00102 *     SPIReceive --> HandleSPIFrame : frame received
00103 *     SPIReceive --> SPITimeout : no data
00104 *     HandleSPIFrame --> [*] : mark received=true
00105 *     SPITimeout --> [*] : continue
00106 *     SkipSPI --> [*] : continue
00107 * }
00108 *
00109 * PollSPI --> ReturnResult : both channels polled
00110 * ReturnResult --> Idle : return k_rx_ok if received\nk_rx_err_timeout if none
00111 * @enduml
00112 *
00113 * ## Performance Characteristics
00114 *
00115 * | Operation | Avg Time | Worst Case | Notes |
00116 * |-----|-----|-----|-----|
00117 * | **rx_comm_manager_init()** | ~5 us | ~10 us | memset dominates (328 bytes) |
00118 * | **rx_comm_manager_poll()** | ~15-200 us | ~500 us | Depends on channels enabled, frames available |
00119 * | **rx_comm_manager_send()** | ~30-150 us | ~300 us | Depends on payload size, channel |
00120 * | **internal_poll_usb()** | ~10-100 us | ~250 us | USB CDC bulk transfer latency |
00121 * | **internal_poll_spi()** | ~5-50 us | ~150 us | SPI hardware transfer latency |
00122 * | **internal_handle_frame()** | ~2 us | ~5 us | Callback invocation overhead |
00123 * | **internal_output_decoded()** | ~50-150 us | ~300 us | ASCII formatting + USB puts |
00124 *
00125 * **Timing Analysis**:
00126 * - Measured @ 240 MHz CPU clock with -O2 optimization
00127 * - Poll time dominated by transport layer (USB/SPI receive)
00128 * - ASCII debugging adds ~50-150 us per frame (disable in production)
00129 * - Callback execution time not included (application-dependent)
00130 *
00131 * ## Memory Usage
00132 *
00133 * | Memory Type | Size | Usage |
00134 * |-----|-----|-----|
00135 * | **rx_comm_manager_t handle** | 328 bytes | Main state structure |
00136 * | **Stack (init)** | ~32 bytes | Function locals |
00137 * | **Stack (poll)** | ~80 bytes | rx_frame_t + locals |
00138 * | **Stack (send)** | ~80 bytes | rx_frame_t + locals |
00139 * | **ASCII buffer** | 256 bytes | Inside handle (for debugging) |
00140 *
00141 * **Memory Layout** (rx_comm_manager_t):
00142 * ```
00143 * Offset | Size | Field | Purpose
00144 * -----|-----|-----|-----
00145 * 0x00 | 8 | usb_handle | Pointer to USB comm handle
00146 * 0x08 | 8 | spi_handle | Pointer to SPI comm handle
00147 * 0x10 | 8 | callback | User frame callback function
00148 * 0x18 | 8 | callback_ctx | User context pointer
00149 * 0x20 | 1 | enable_decoded_output | ASCII debug flag
00150 * 0x21 | 1 | initialized | Initialization flag
00151 * 0x22 | 2 | (padding) | Alignment
00152 * 0x24 | 4 | (padding) | Alignment
00153 * 0x28 | 256 | ascii_buffer | ASCII formatting buffer
00154 * -----|-----|-----|-----
00155 * Total: 328 bytes (0x148)
00156 * ```
00157 *
00158 * ## Algorithm Details
00159 *
00160 * ### Polling Algorithm (rx_comm_manager_poll)
00161 *
00162 * The polling algorithm maximizes throughput by checking all channels without
00163 * blocking, returning immediately on the first error while continuing to poll
00164 * remaining channels on timeout (no data available):
00165 *
00166 * **Steps**:
00167 * 1. **Parameter validation** (NULL check, initialized flag)
00168 * 2. **Poll USB channel**:
00169 * - If nullptr handle -> skip (k_rx_err_timeout)

```



```

00170 * - If data available -> handle frame, mark received=true
00171 * - If timeout -> continue to next channel
00172 * - If error -> return error immediately (critical failure)
00173 * 3. **Poll SPI channel**:
00174 * - Same logic as USB
00175 * 4. **Return aggregated result**:
00176 * - k_rx_ok if ANY frame received on any channel
00177 * - k_rx_err_timeout if NO frames received (normal, non-error condition)
00178 * - Other error codes propagated immediately
00179 *
00180 * **Design Rationale**:
00181 * - Non-blocking: Never blocks waiting for data
00182 * - Fair scheduling: Both channels polled each iteration
00183 * - Error prioritization: Critical errors returned immediately
00184 * - Timeout tolerance: Timeout is expected, not an error
00185 *
00186 * ### Send Algorithm (rx_comm_manager_send)
00187 *
00188 * Routing algorithm with post-send ASCII debugging support:
00189 *
00190 * **Steps**:
00191 * 1. **Validate parameters** (NULL checks, payload length <= k_frame_max_payload)
00192 * 2. **Route to channel**:
00193 * - USB: Call rx_usb_comm_send()
00194 * - SPI: Call rx_spi_comm_send()
00195 * - Invalid channel: Return k_rx_err_invalid_arg
00196 * 3. **On success, if ASCII output enabled**:
00197 * - Build temporary rx_frame_t from send parameters
00198 * - Format as ASCII via rx_frame_ascii_format()
00199 * - Send to USB Port 1 via rx_usb_puts()
00200 *
00201 * **Performance Note**: ASCII formatting adds ~50-150 us overhead per send.
00202 * Disable in production with `cfg->enable_decoded_output = false`.
00203 *
00204 * ## Thread Safety Analysis
00205 *
00206 * **Conditional Thread Safety** - Safe if following rules are observed:
00207 *
00208 * | Scenario | Thread Safety | Mitigation Required |
00209 * |-----|-----|-----|
00210 * | **Single-threaded** | [OK] Safe | None |
00211 * | **Multi-threaded (different channels)** | [OK] Safe | Ensure each thread uses different channel |
00212 * | **Multi-threaded (same channel)** | [X] Unsafe | External mutex required |
00213 * | **Multi-threaded (different handles)** | [OK] Safe | None (independent handles) |
00214 * | **Init/Deinit from ISR** | [X] Unsafe | Call only from task context |
00215 * | **Poll/Send from ISR** | [WARN] Conditional | OK if callback is ISR-safe |
00216 *
00217 * **Recommended Architecture**:
00218 * - Dedicate one thread to poll all channels (e.g., ThreadX communication task)
00219 * - Send from any thread/ISR, but use external mutex if same channel
00220 * - Ensure user callback is reentrant if called from ISR
00221 *
00222 * **NASA Power of 10 Rule 5**: All public functions validate preconditions with
00223 * explicit checks (NULL pointers, initialized flag, parameter ranges).
00224 *
00225 * ## Integration Example
00226 *
00227 * @par Complete System Setup with ThreadX:
00228 * @code{.c}
00229 * // Global communication manager handle
00230 * static rx_comm_manager_t g_comm_mgr;
00231 * static rx_usb_comm_handle_t g_usb_handle;
00232 * static rx_spi_comm_handle_t g_spi_handle;
00233 *
00234 * // User callback for frame processing
00235 * static void on_frame_received(rx_comm_channel_t channel,
00236 *                             const rx_frame_t* frame,
00237 *                             void* ctx)
00238 * {
00239 *     (void)ctx;
00240 *
00241 *     // Dispatch based on frame type
00242 *     switch (frame->header.type) {
00243 *     case k_frame_type_command:
00244 *         handle_command(channel, frame);
00245 *         break;
00246 *
00247 *     case k_frame_type_request:
00248 *         handle_request(channel, frame);
00249 *         break;
00250 *
00251 *     default:
00252 *         // Unknown frame type, ignore
00253 *         break;
00254 *     }
00255 * }
00256 *

```

```

00257 * // ThreadX communication task
00258 * static void comm_task_entry(ULONG thread_input)
00259 * {
00260 *     (void)thread_input;
00261 *
00262 *     // Initialize USB CDC communication
00263 *     rx_usb_comm_config_t usb_cfg = {
00264 *         .port = k_usb_port_proto,
00265 *     };
00266 *     rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
00267 *     if (err != k_rx_ok) {
00268 *         // Handle error
00269 *         return;
00270 *     }
00271 *
00272 *     // Initialize SPI communication
00273 *     rx_spi_comm_config_t spi_cfg = {
00274 *         .spi_channel = 0,
00275 *     };
00276 *     err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
00277 *     if (err != k_rx_ok) {
00278 *         // Handle error
00279 *         return;
00280 *     }
00281 *
00282 *     // Initialize communication manager
00283 *     rx_comm_manager_config_t cfg = {
00284 *         .usb_handle = &g_usb_handle,
00285 *         .spi_handle = &g_spi_handle,
00286 *         .callback = on_frame_received,
00287 *         .callback_ctx = nullptr,
00288 *         .enable_decoded_output = true, // Enable for development
00289 *     };
00290 *     err = rx_comm_manager_init(&g_comm_mgr, &cfg);
00291 *     if (err != k_rx_ok) {
00292 *         // Handle error
00293 *         return;
00294 *     }
00295 *
00296 *     // Main communication loop (100 Hz polling)
00297 *     while (1) {
00298 *         // Poll all channels
00299 *         err = rx_comm_manager_poll(&g_comm_mgr);
00300 *
00301 *         // Check for critical errors (not timeout)
00302 *         if (err != k_rx_ok && err != k_rx_err_timeout) {
00303 *             // Log error, attempt recovery
00304 *             tx_thread_sleep(100); // 1 second backoff
00305 *         }
00306 *
00307 *         // Sleep 10 ticks (100 ms @ 100 Hz tick rate)
00308 *         tx_thread_sleep(10);
00309 *     }
00310 * }
00311 *
00312 * // Send response from application
00313 * void send_motor_telemetry(void)
00314 * {
00315 *     uint8_t telemetry_payload[16];
00316 *     // ... fill telemetry data ...
00317 *
00318 *     // Check if USB channel is ready
00319 *     bool usb_ready = false;
00320 *     rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
00321 *                                                  k_comm_channel_usb,
00322 *                                                  &usb_ready);
00323 *     if (err == k_rx_ok && usb_ready) {
00324 *         // Send telemetry response
00325 *         err = rx_comm_manager_respond(&g_comm_mgr,
00326 *                                       k_comm_channel_usb,
00327 *                                       telemetry_payload,
00328 *                                       sizeof(telemetry_payload));
00329 *     }
00330 * }
00331 * @endcode
00332 *
00333 * ## NASA Power of 10 Compliance
00334 *
00335 * | Rule | Compliance | Implementation Notes |
00336 * |-----|-----|-----|
00337 * | **Rule 1: Control Flow** | [OK] | No goto, setjmp, longjmp, or recursion |
00338 * | **Rule 2: Loop Bounds** | [OK] | All loops have statically provable bounds |
00339 * | **Rule 3: No Dynamic Allocation** | [OK] | Zero malloc/free - all static allocation |
00340 * | **Rule 4: Function Size** | [OK] | All functions < 60 lines |
00341 * | **Rule 5: Assertions** | [OK] | Minimum 2 checks per function (nullptr, initialized) |
00342 * | **Rule 6: Data Scope** | [OK] | Variables declared at smallest scope |
00343 * | **Rule 7: Check Returns** | [OK] | All return values validated or cast to (void) |

```

```

00344 * | **Rule 8: Preprocessor** | [OK] | Minimal macros, typed enums for constants |
00345 * | **Rule 9: Pointers** | [WARN] | Function pointers used for callback (DIP) |
00346 * | **Rule 10: Compiler Warnings** | [OK] | -Wall -Wextra -Werror enabled |
00347 *
00348 * **Rule 9 Deviation**: Function pointer used for user callback to enable Dependency
00349 * Inversion Principle (SOLID) - allows application to register frame handler without
00350 * tight coupling. This is intentional and documented per project policy.
00351 *
00352 * ## SOLID Principles
00353 *
00354 * | Principle | Implementation |
00355 * |-----|-----|
00356 * | **Single Responsibility** | Module responsible ONLY for channel coordination; frame parsing/transport delegated |
00357 * | **Open/Closed** | Extensible via callback; new frame types handled by application |
00358 * | **Liskov Substitution** | Channels are interchangeable (same rx_frame_t interface) |
00359 * | **Interface Segregation** | Small focused API (7 functions); send/poll/ready separated |
00360 * | **Dependency Inversion** | Depends on abstract rx_frame_t and channel handles, not concrete implementations |
00361 *
00362 * **Key Design Decision**: The manager does NOT parse frame payloads - it only routes
00363 * frames to the user callback. This separation allows application-specific command
00364 * handling without modifying the communication layer.
00365 *
00366 * ## Module Dependencies
00367 *
00368 * **Includes**
00369 * - rx_comm_manager.h - Public API definitions
00370 * - rx_frame_ascii.h - ASCII formatting for debugging
00371 * - rx_usb.h - USB port access (Port 1 debug output)
00372 * - string.h - Standard C library (memset, memcpy)
00373 *
00374 * **Links Against**
00375 * - rx_usb_comm - USB CDC frame protocol implementation
00376 * - rx_spi_comm - SPI frame protocol implementation
00377 * - rx_frame_ascii - ASCII frame formatting
00378 * - rx_usb - USB hardware abstraction
00379 *
00380 * @date 2026-01-27
00381 * @copyright Copyright (c) 2026 Locked Inc.
00382 * SPDX-License-Identifier: MIT
00383 * @since Version 1.0.0
00384 *
00385 * @see rx_comm_manager.h Full API documentation
00386 * @see rx_spi_comm.h SPI protocol implementation
00387 * @see rx_uart_comm.h UART protocol implementation
00388 * @see rx_i2c_comm.h I2C peripheral protocol implementation
00389 * @see rx_frame_ascii.h ASCII debugging interface
00390 */
00391
00392 #include "rx_comm_manager.h"
00393
00394 #include "rx_check.h"
00395 #include "rx_frame_ascii.h"
00396 #include "rx_log.h"
00397 #include "rx_log_uart.h"
00398 #include "rx_simulator_config.h"
00399 #include "rx_time_constants.h"
00400
00401 #if !RX_IS_SIMULATOR
00402 #include "tx_api.h"
00403 #endif
00404
00405 /*
=====
00406 * Logging
00407 *
=====
00408 */
00409
00410 static const char s_tag[] = "COMM_MGR";
00411
00412 /*
=====
00413 * Private Helpers
00414 *
=====
00415 */
00416
00417 /**
00418 * @brief Get current time in milliseconds
00419 *
00420 * @details
00421 * Returns the current system time in milliseconds. On RX72N hardware, this
00422 * uses ThreadX tx_time_get() multiplied by tick period. In simulator builds,
00423 * returns 0 (timing not available).
00424 *
00425 *
00426 * @since Version 1.0.0

```

```

00427 */
00428 /** @brief Simulator time stub value (time is unavailable in simulator) */
00429 typedef enum : uint32_t {
00430     k_sim_time_ms_zero = 0, /**< Simulator returns zero (no real-time clock) */
00431 } rx_comm_sim_time_t;
00432
00433 static uint32_t internal_get_time_ms(void)
00434 {
00435     #if !RX_IS_SIMULATOR
00436         return (uint32_t)tx_time_get() * k_threadx_ms_per_tick;
00437     #else
00438         return k_sim_time_ms_zero;
00439     #endif
00440 }
00441
00442 /*
=====
00443  * Private Constants
00444  *
=====
00445  */
00446
00447 /**
00448  * @enum internal_constants_t
00449  * @brief Internal constants for communication manager implementation
00450  *
00451  * @details
00452  * Defines compile-time constants used internally by the communication manager
00453  * for timeout values, placeholder values, and buffer indexing.
00454  *
00455  * **Design Notes:**
00456  * - Non-blocking timeout (0 ms) ensures poll never blocks
00457  * - Placeholder values filled by transport layer (sequence, CRC)
00458  * - Explicit array index constant for bounds checking
00459  *
00460  * @since Version 1.0.0
00461  */
00462 typedef enum : uint32_t {
00463     /**
00464      * @brief Non-blocking receive timeout for polling
00465      * @details
00466      * Set to 0 ms to ensure rx_comm_manager_poll() never blocks waiting for data.
00467      * Transport layers (USB/SPI) return k_rx_err_timeout immediately if no data.
00468      * @par Value: 0 milliseconds
00469      */
00470     k_receive_timeout_ms = 0,
00471
00472     /**
00473      * @brief Placeholder for frame sequence number (filled by transport layer)
00474      * @details
00475      * Sequence number is managed by transport layer (rx_usb_comm/rx_spi_comm)
00476      * to detect dropped frames. Manager does not manipulate sequence numbers.
00477      * @par Value: 0
00478      */
00479     k_frame_seq_placeholder = 0,
00480
00481     /**
00482      * @brief Placeholder for frame CRC-32 (filled by transport layer)
00483      * @details
00484      * CRC-32 calculated and verified by transport layer. Manager builds temp
00485      * frame with placeholder CRC for ASCII formatting only.
00486      * @par Value: 0
00487      */
00488     k_frame_crc_placeholder = 0,
00489
00490     /**
00491      * @brief First index of ascii_buffer for post-condition validation
00492      * @details
00493      * Used in rx_comm_manager_init() to verify memset properly cleared buffer.
00494      * First byte must be '\0' after zero-fill.
00495      * @par Value: 0
00496      */
00497     k_ascii_buffer_first_idx = 0,
00498
00499     /**
00500      * @brief Typed zero constant for uint32_t comparisons
00501      * @details
00502      * Used for payload length checks and other uint32_t zero comparisons to avoid
00503      * magic number 0. Provides type safety and self-documenting code.
00504      * @par Value: 0
00505      */
00506     k_zero_u32 = 0,
00507 } internal_constants_t;
00508
00509 /*
=====
00510  * Private Functions

```

```

00511 *
00512 =====
00513 */
00514 /**
00515  * @brief Output decoded ASCII frame to USB Port 1 for debugging
00516  *
00517  * @details
00518  * Formats the given frame as human-readable ASCII text and outputs to USB Port 1
00519  * (decoded debug port) if decoded output is enabled. Used for development/debugging
00520  * to monitor frame traffic without binary protocol parsing.
00521  *
00522  * **Algorithm:**
00523  * 1. Validate parameters (NULL checks)
00524  * 2. Check if decoded output is enabled (cfg->enable_decoded_output)
00525  * 3. Format frame as ASCII via rx_frame_ascii_format()
00526  * 4. Send ASCII string to USB Port 1 via rx_usb_puts()
00527  *
00528  * **Performance**: ~50-150 us depending on frame size (ASCII formatting overhead).
00529  * Disable in production for optimal performance.
00530  *
00531  *
00532  *
00533  *
00534  *
00535  * @pre mgr must be initialized via rx_comm_manager_init()
00536  * @pre frame must be a valid frame structure
00537  * @post mgr->ascii_buffer content is modified (temporary)
00538  * @post ASCII output sent to USB Port 1 if enabled and successful
00539  *
00540  * @note Not thread-safe - caller must ensure exclusive access to mgr->ascii_buffer
00541  * @note Errors are silently ignored (formatting/USB send failures)
00542  * @note USB Port 1 must be configured for ASCII output to appear
00543  *
00544  * @par Example Output:
00545  * ```
00546  * RX: [USB] Type=0x01 Flags=0x00 Len=8 Seq=42 Payload=0102030405060708 CRC=ABCD1234
00547  * TX: [SPI] Type=0x02 Flags=0x80 Len=4 Seq=43 Payload=01020304 CRC=5678CDEF
00548  * ```
00549  *
00550  * @see rx_frame_ascii_format() Frame-to-ASCII conversion
00551  * @see rx_usb_puts() USB debug output
00552  *
00553  * @since Version 1.0.0
00554  */
00555 RX_STATIC_TESTABLE void
00556 internal_output_decoded(rx_comm_manager_t* mgr, const rx_frame_t* frame, bool is_tx)
00557 {
00558     if (!mgr->enable_decoded_output) {
00559         return;
00560     }
00561
00562     uint32_t len = 0;
00563     /* rx_frame_ascii_format always succeeds and produces output for any valid frame
00564      * and adequately-sized buffer (guaranteed by ascii_buffer sizing in the manager). */
00565     (void)rx_frame_ascii_format(frame, is_tx, mgr->ascii_buffer, sizeof(mgr->ascii_buffer), &len);
00566     /* The old decoded-debug path wrote to USB CDC port 1; with USB0 gone, we
00567      * route the ASCII dump through the UART log ring so it still shows up on
00568      * the gateway prefixed with [RX72N]. */
00569     rx_log_uart_puts(mgr->ascii_buffer);
00570 }
00571
00572 /**
00573  * @brief Build temporary frame from send params and output decoded ASCII (TX direction)
00574  *
00575  * @details
00576  * Helper for rx_comm_manager_send() to output ASCII-formatted frame after a successful
00577  * send. Constructs a temporary rx_frame_t from the send parameters and delegates to
00578  * internal_output_decoded().
00579  *
00580  *
00581  * @pre mgr must be non-NULL and initialized
00582  * @pre params must be non-NULL with valid fields
00583  * @post ASCII output sent to USB Port 1 if decoded output is enabled
00584  *
00585  * @note Not thread-safe - caller must ensure exclusive access to mgr->ascii_buffer
00586  *
00587  * @since Version 1.0.0
00588  */
00589 static void internal_output_decoded_from_params(rx_comm_manager_t* mgr,
00590 const rx_comm_send_params_t* params)
00591 {
00592     /* Build a temporary frame for ASCII formatting */
00593     rx_frame_t frame = {}; /* Zero-initialize to clear padding/unset fields */
00594     frame.header.sequence = k_frame_seq_placeholder;
00595     frame.header.length = (uint16_t)params->payload_len;
00596     frame.header.type = (uint8_t)params->type;

```

```

00597 frame.header.flags = params->flags;
00598
00599 /* payload_len <= k_frame_max_payload is guaranteed by caller validation */
00600 for (uint32_t i = 0; i < params->payload_len; i++) {
00601     frame.payload[i] = params->payload[i];
00602 }
00603 frame.crc = k_frame_crc_placeholder;
00604
00605 internal_output_decoded(mgr, &frame, true);
00606 }
00607
00608 /**
00609  * @brief Handle received frame by invoking user callback and optional ASCII output
00610  *
00611  * @details
00612  * Central frame dispatch point for all channels. Performs optional ASCII debugging
00613  * output, then invokes user-registered callback with frame data. This function
00614  * isolates callback invocation logic from channel-specific polling.
00615  *
00616  * **Algorithm:**
00617  * 1. Validate parameters (NULL checks, channel range)
00618  * 2. Output decoded ASCII to Port 1 if enabled (RX direction)
00619  * 3. Invoke user callback if registered
00620  *
00621  * **Design Rationale:** Centralizing frame handling logic ensures consistent
00622  * behavior across USB and SPI channels (single point of dispatch).
00623  *
00624  *
00625  *
00626  *
00627  *
00628  * @pre mgr must be initialized
00629  * @pre frame must be valid and CRC-verified by transport layer
00630  * @pre channel must be in valid range [0, k_comm_channel_count)
00631  * @post User callback invoked if registered (callback may be NULL)
00632  * @post ASCII output sent to Port 1 if enabled
00633  *
00634  * @note Thread safety depends on user callback implementation
00635  * @note Callback errors are not returned (callback should handle internally)
00636  * @warning User callback must not block for extended periods (affects polling rate)
00637  *
00638  * @par Performance:
00639  * - Without decoded output: ~2 us (callback overhead only)
00640  * - With decoded output: ~50-150 us (ASCII formatting + USB puts)
00641  *
00642  * @see internal_output_decoded() ASCII debugging output
00643  * @see rx_comm_frame_callback_t User callback type
00644  *
00645  * @since Version 1.0.0
00646  */
00647 RX_STATIC_TESTABLE void
00648 internal_handle_frame(rx_comm_manager_t* mgr, rx_comm_channel_t channel, const rx_frame_t* frame)
00649 {
00650     /* Update heartbeat: any valid frame resets the implicit timeout timer.
00651      * Per the dual-detection model, this is the primary link health
00652      * mechanism (200ms implicit timeout). */
00653     rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[channel];
00654     hb->last_rx_ms = internal_get_time_ms();
00655     if (hb->status != k_link_status_healthy) {
00656         rx_comm_link_status_t old_status = hb->status;
00657         hb->status = k_link_status_healthy;
00658         if (old_status == k_link_status_dead) {
00659             rx_log_info(s_tag, "Link recovered: frame received");
00660         } else {
00661             rx_log_info(s_tag, "Link established: first frame received");
00662         }
00663     }
00664     if (mgr->link_status_cb != nullptr) {
00665         mgr->link_status_cb(channel, k_link_status_healthy, mgr->link_status_ctx);
00666     }
00667
00668     rx_log_debug_val(s_tag, "Frame received, type", (uint8_t)frame->header.type);
00669
00670     if (frame->header.type == k_frame_type_command) {
00671         rx_log_info_val(s_tag, "CMD frame seq", frame->header.sequence);
00672         rx_log_info_val(s_tag, "CMD frame channel", (uint8_t)channel);
00673     }
00674
00675     /* Output decoded ASCII to Port 1 */
00676     internal_output_decoded(mgr, frame, false);
00677
00678     /* Invoke user callback if set */
00679     if (mgr->callback != nullptr) {
00680         mgr->callback(channel, frame, mgr->callback_ctx);
00681     }
00682 }
00683

```

```

00684 /**
00685  * @brief Poll SPI channel for incoming frames (non-blocking)
00686  *
00687  * @details
00688  * Attempts to receive one frame from SPI channel with zero timeout (non-blocking).
00689  * If frame is successfully received, it is dispatched via internal_handle_frame().
00690  * Timeout (no data available) is expected and normal - not an error condition.
00691  *
00692  * **Algorithm:**
00693  * 1. Validate mgr pointer (NULL check)
00694  * 2. Skip if SPI handle is nullptr (channel not configured)
00695  * 3. Call rx_spi_comm_receive() with 0 ms timeout
00696  * 4. On success: dispatch frame via internal_handle_frame()
00697  * 5. On timeout: return k_rx_err_timeout (expected, not error)
00698  * 6. On other errors: propagate error code
00699  *
00700  * **Design Note**: Identical logic to internal_poll_usb() but for SPI channel.
00701  * Code duplication is intentional (NASA Rule 4: keep functions short and simple).
00702  *
00703  *
00704  *
00705  * @pre mgr must be non-NULL
00706  * @pre If SPI channel enabled, spi_handle must be initialized
00707  * @post On k_rx_ok: callback invoked, ASCII output sent (if enabled)
00708  * @post On timeout: no side effects (no frame received)
00709  *
00710  * @note Non-blocking - always returns immediately
00711  * @note Timeout is NOT an error - it means no data is available
00712  * @warning Do not call if spi_handle is uninitialized (will return timeout)
00713  *
00714  * @par Performance:
00715  * - No data available: ~5 us (fast path)
00716  * - Frame received: ~50-150 us (SPI hardware transfer + callback)
00717  *
00718  * @see internal_handle_frame() Frame dispatch
00719  * @see rx_spi_comm_receive() SPI receive implementation
00720  *
00721  * @since Version 1.0.0
00722  */
00723 static rx_err_t internal_poll_spi(rx_comm_manager_t* mgr)
00724 {
00725     if (mgr->spi_handle == nullptr) {
00726         return k_rx_err_timeout;
00727     }
00728
00729     /* If SPI link layer is available, use it for HARQ decoding
00730     * NOTE: Using static buffers to avoid stack overflow (~3KB total).
00731     * Safe because comm_manager_poll is single-threaded (called from comm_task). */
00732     if (mgr->spi_link != nullptr) {
00733         static rx_spi_link_receive_result_t s_link_result;
00734         rx_err_t err = rx_spi_link_receive(mgr->spi_link, &s_link_result, k_receive_timeout_ms);
00735
00736         if (err == k_rx_ok) {
00737             /* Convert link result to rx_frame_t for internal_handle_frame() */
00738             static rx_frame_t s_frame1;
00739             s_frame1 = (rx_frame_t){};
00740             s_frame1.header.sequence = s_link_result.sequence;
00741             s_frame1.header.length = (uint16_t)s_link_result.payload_len;
00742             s_frame1.header.type = s_link_result.frame_type;
00743
00744             /* Preserve retransmit and FEC flags from link layer result */
00745             s_frame1.header.flags = k_frame_flag_none;
00746             if (s_link_result.is_retransmit) {
00747                 s_frame1.header.flags |= k_frame_flag_retransmit;
00748             }
00749             if (s_link_result.fec_decoded) {
00750                 s_frame1.header.flags |= k_frame_flag_fec_enabled;
00751             }
00752
00753             for (uint32_t i = 0; i < s_link_result.payload_len; i++) {
00754                 s_frame1.payload[i] = s_link_result.payload[i];
00755             }
00756             internal_handle_frame(mgr, k_comm_channel_spi, &s_frame1);
00757             return k_rx_ok;
00758         }
00759
00760         return err;
00761     }
00762
00763     /* Fallback: raw SPI without HARQ */
00764     static rx_frame_t s_frame2;
00765     rx_err_t err = rx_spi_comm_receive(mgr->spi_handle, &s_frame2, k_receive_timeout_ms);
00766
00767     if (err == k_rx_ok) {
00768         internal_handle_frame(mgr, k_comm_channel_spi, &s_frame2);
00769         return k_rx_ok;
00770     }

```



```

00771
00772     return err;
00773 }
00774
00775 /**
00776  * @brief Poll I2C channel for incoming frames (non-blocking)
00777  *
00778  * @details
00779  * Attempts to receive one frame from I2C peripheral channel with zero timeout.
00780  * If frame is successfully received, it is dispatched via internal_handle_frame().
00781  *
00782  *
00783  *
00784  * @pre mgr must be non-NULL
00785  * @pre mgr->i2c_handle must be initialized before calling rx_comm_manager_poll()
00786  * @post On k_rx_ok: callback invoked for received frame
00787  * @post On k_rx_err_timeout or k_rx_err_no_data: no state change
00788  *
00789  * @note Non-blocking - always returns immediately
00790  *
00791  * @see internal_handle_frame() Frame dispatch
00792  * @see rx_i2c_comm_receive() I2C peripheral receive implementation
00793  *
00794  * @since Version 1.0.0
00795  */
00796 static rx_err_t internal_poll_i2c(rx_comm_manager_t* mgr)
00797 {
00798     if (mgr->i2c_handle == nullptr) {
00799         return k_rx_err_timeout;
00800     }
00801
00802     rx_frame_t frame;
00803     rx_err_t err = rx_i2c_comm_receive(mgr->i2c_handle, &frame, k_receive_timeout_ms);
00804
00805     if (err == k_rx_ok) {
00806         internal_handle_frame(mgr, k_comm_channel_i2c, &frame);
00807         return k_rx_ok;
00808     }
00809
00810     return err;
00811 }
00812
00813 /**
00814  * @brief Poll UART channel for incoming frames (non-blocking)
00815  *
00816  * @details
00817  * Attempts to receive one frame from UART (SCI9) channel with zero timeout.
00818  * If frame is successfully received, it is dispatched via internal_handle_frame().
00819  *
00820  *
00821  *
00822  * @pre mgr must be non-NULL
00823  * @pre mgr->uart_handle must be initialized before calling rx_comm_manager_poll()
00824  * @post On k_rx_ok: callback invoked for received frame
00825  * @post On k_rx_err_timeout or k_rx_err_no_data: no state change
00826  *
00827  * @note Non-blocking - always returns immediately
00828  *
00829  * @see internal_handle_frame() Frame dispatch
00830  * @see rx_uart_comm_receive() UART receive implementation
00831  *
00832  * @since Version 1.0.0
00833  */
00834 static rx_err_t internal_poll_uart(rx_comm_manager_t* mgr)
00835 {
00836     if (mgr->uart_handle == nullptr) {
00837         return k_rx_err_timeout;
00838     }
00839
00840     rx_frame_t frame;
00841     rx_err_t err = rx_uart_comm_receive(mgr->uart_handle, &frame, k_receive_timeout_ms);
00842
00843     if (err == k_rx_ok) {
00844         if (frame.header.type == k_frame_type_command) {
00845             rx_log_info_val(s_tag, "UART RX CMD seq", frame.header.sequence);
00846         }
00847         internal_handle_frame(mgr, k_comm_channel_uart, &frame);
00848         return k_rx_ok;
00849     }
00850
00851     return err;
00852 }
00853
00854 /**
00855  * =====
00856  * Event Queue Helpers

```



```

=====
00857 */
00858
00859 /**
00860  * @brief Process one event from the FIFO queue
00861  *
00862  * @details
00863  * Dequeues the oldest event and attempts to send it. For SPI events, if the
00864  * send fails, the event is re-enqueued with incremented retry count (up to
00865  * k_event_max_retries). For USB events, failure is simply dropped (fire-and-forget).
00866  *
00867  *
00868  * @pre mgr->event_queue_count > 0
00869  * @post One event processed (sent or dropped on max retries)
00870  *
00871  * @since Version 1.0.0
00872  */
00873 /**
00874  * @brief Dequeue the front event entry and advance the tail pointer
00875  *
00876  *
00877  * @pre mgr->event_queue_count > 0
00878  * @post entry cleared and tail pointer advanced
00879  *
00880  * @since Version 1.0.0
00881  */
00882 static void internal_dequeue_event(rx_comm_manager_t* mgr, rx_comm_event_entry_t* entry)
00883 {
00884     entry->occupied      = false;
00885     entry->retries        = 0;
00886     entry->next_retry_time_ms = 0;
00887     mgr->event_queue_tail = (mgr->event_queue_tail + 1) % k_comm_event_queue_depth;
00888     mgr->event_queue_count--;
00889 }
00890
00891 static void internal_process_event_queue(rx_comm_manager_t* mgr)
00892 {
00893     if (mgr->event_queue_count == 0) {
00894         return;
00895     }
00896
00897     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_tail];
00898     if (!entry->occupied) {
00899         return;
00900     }
00901
00902     /* Check backoff: skip if retry time hasn't been reached yet */
00903     const uint32_t now_ms = internal_get_time_ms();
00904     if (entry->retries > 0 && now_ms < entry->next_retry_time_ms) {
00905         return; /* Backoff period not elapsed, try again later */
00906     }
00907
00908     /* Build send params from queued entry */
00909     const rx_comm_send_params_t params = {
00910         .channel    = entry->channel,
00911         .type       = entry->type,
00912         .flags      = entry->flags,
00913         .payload     = entry->payload,
00914         .payload_len = entry->payload_len,
00915     };
00916
00917     rx_err_t err = rx_comm_manager_send(mgr, &params);
00918
00919     if (err == k_rx_ok || entry->channel == k_comm_channel_uart) {
00920         /* Success, or UART fire-and-forget: dequeue.
00921          * UART has no link-layer retry (the CY7C65213 bridge + USB CDC transport
00922          * on the host provides enough reliability for the command/response path). */
00923         internal_dequeue_event(mgr, entry);
00924         if (err == k_rx_ok) {
00925             rx_log_debug_val(s_tag, "Event sent, queue depth", mgr->event_queue_count);
00926         } else {
00927             rx_log_warn_val(s_tag, "UART event dropped, queue depth", mgr->event_queue_count);
00928         }
00929         return;
00930     }
00931
00932     /* SPI send failed: retry with exponential backoff */
00933     entry->retries++;
00934     if (entry->retries >= k_event_max_retries) {
00935         rx_log_error_val(s_tag, "SPI event failed after retries", (uint8_t)entry->retries);
00936         internal_dequeue_event(mgr, entry);
00937         return;
00938     }
00939
00940     /* Compute exponential backoff: initial_ms « (retries - 1), capped at max */
00941     uint32_t backoff_ms = (uint32_t)k_event_initial_backoff_ms « (entry->retries - 1);
00942     if (backoff_ms > k_event_max_backoff_ms) {

```

```

00943     backoff_ms = k_event_max_backoff_ms;
00944 }
00945 entry->next_retry_time_ms = now_ms + backoff_ms;
00946
00947 rx_log_warn_val(s_tag, "SPI event retry attempt", (uint8_t)entry->retries);
00948 /* Entry stays in queue, will be retried after backoff period */
00949 }
00950
00951 /*
=====
00952 * Heartbeat Helpers
00953 *
=====
00954 */
00955
00956 /**
00957  * @brief Check heartbeat implicit timeout on all transports
00958  *
00959  * @details
00960  * Evaluates the dual-detection heartbeat model.
00961  * If no valid frame has been received on a transport within 200ms, the link
00962  * is declared dead and the status callback is invoked.
00963  *
00964  *
00965  * @pre mgr must be non-NULL and initialized
00966  * @post Link status updated for each transport
00967  *
00968  * @since Version 1.0.0
00969  */
00970 /**
00971  * @brief Check heartbeat timeout on a single channel
00972  *
00973  *
00974  * @pre ch < k_comm_channel_count
00975  * @post Link status updated if timeout exceeded
00976  *
00977  * @since Version 1.0.0
00978  */
00979 static void internal_check_channel_heartbeat(rx_comm_manager_t* mgr, uint8_t ch, uint32_t now_ms)
00980 {
00981     rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[ch];
00982
00983     /* Skip channels that haven't received any frame yet */
00984     if (hb->status != k_link_status_healthy) {
00985         return;
00986     }
00987
00988     /* Check implicit timeout: 200ms since last valid frame */
00989     const uint32_t elapsed = now_ms - hb->last_rx_ms;
00990     if (elapsed < k_heartbeat_implicit_timeout_ms) {
00991         return;
00992     }
00993
00994     hb->status = k_link_status_dead;
00995     rx_log_warn_val(s_tag, "Link dead: no frame (ms elapsed)", elapsed);
00996
00997     if (mgr->link_status_cb != nullptr) {
00998         mgr->link_status_cb((rx_comm_channel_t)ch, k_link_status_dead, mgr->link_status_ctx);
00999     }
01000 }
01001
01002 static void internal_check_heartbeat(rx_comm_manager_t* mgr)
01003 {
01004     const uint32_t now = internal_get_time_ms();
01005
01006     for (uint8_t ch = 0; ch < k_comm_channel_count; ch++) {
01007         internal_check_channel_heartbeat(mgr, ch, now);
01008     }
01009 }
01010
01011 /*
=====
01012 * Public Functions
01013 *
=====
01014 */
01015
01016 /**
01017  * @var s_zero_mgr
01018  * @brief Zero-initialized comm manager template for stack-safe initialization
01019  * @details Static const instance used to clear rx_comm_manager_t without creating
01020  *         a large compound literal on the stack. Lives in .rodata section.
01021  * @note Read-only; never modified after static initialization
01022  * @warning Do not remove - prevents stack overflow in init function
01023  * @see rx_comm_manager_init() Uses this template to zero-initialize the manager handle
01024  * @since Version 1.0.0
01025  */

```

```

01026 static const rx_comm_manager_t s_zero_mgr = {};
01027
01028 /**
01029  * @brief Initialize communication manager with channel configuration
01030  *
01031  * @details
01032  * Performs complete initialization of communication manager handle, configuring
01033  * enabled channels, callback registration, and ASCII debugging output. This function
01034  * must be called before any other communication manager operations.
01035  *
01036  * **Algorithm**:
01037  * 1. **Validate parameters**: Check mgr pointer for nullptr (NASA Rule 5)
01038  * 2. **Clear handle**: Zero-fill entire structure via memset (328 bytes)
01039  * 3. **Apply configuration**: Copy config fields if cfg is non-NULL
01040  *   - usb_handle: Pointer to initialized USB comm handle (may be NULL)
01041  *   - spi_handle: Pointer to initialized SPI comm handle (may be NULL)
01042  *   - callback: User frame handler function (may be NULL)
01043  *   - callback_ctx: User context pointer (may be NULL)
01044  *   - enable_decoded_output: ASCII debugging flag
01045  * 4. **Mark initialized**: Set mgr->initialized = true
01046  * 5. **Post-condition validation**: Verify ascii_buffer properly cleared
01047  *
01048  * **Configuration Options**:
01049  * - **cfg = NULL**: All channels disabled, no callback, ASCII output off
01050  * - **usb_handle = NULL**: USB channel disabled (poll skips USB)
01051  * - **spi_handle = NULL**: SPI channel disabled (poll skips SPI)
01052  * - **callback = NULL**: No callback invoked (silent frame consumption)
01053  * - **enable_decoded_output = true**: ASCII debugging enabled (Port 1)
01054  *
01055  *
01056  *
01057  *
01058  * @pre mgr must point to allocated memory (uninitialized content OK)
01059  * @pre USB/SPI handles (if provided) must already be initialized
01060  * @pre Callback function (if provided) must be valid and reentrant
01061  * @post mgr->initialized = true on success
01062  * @post All mgr fields set to config values or zero
01063  * @post mgr->ascii_buffer is zero-filled (256 bytes)
01064  *
01065  * @note This function does NOT initialize USB or SPI transports - caller must initialize
01066  *       rx_usb_comm and rx_spi_comm separately before passing handles to this function
01067  * @note On success, at least one channel should be enabled (usb_handle or spi_handle non-NULL)
01068  * @note Thread-safe ONLY if each thread uses a different mgr handle
01069  *
01070  * @par Performance:
01071  * - Execution time: ~5 us @ 240 MHz
01072  * - Dominated by memset (328 bytes)
01073  *
01074  * @par Configuration Example - Both Channels Enabled:
01075  * @code{.c}
01076  * static rx_comm_manager_t g_comm_mgr;
01077  * static rx_usb_comm_handle_t g_usb_handle;
01078  * static rx_spi_comm_handle_t g_spi_handle;
01079  *
01080  * // Initialize transport layers first
01081  * rx_usb_comm_config_t usb_cfg = { .port = k_usb_port_proto };
01082  * rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
01083  * if (err != k_rx_ok) return err;
01084  *
01085  * rx_spi_comm_config_t spi_cfg = { .spi_channel = 0 };
01086  * err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
01087  * if (err != k_rx_ok) return err;
01088  *
01089  * // Initialize communication manager with both channels
01090  * rx_comm_manager_config_t cfg = {
01091  *   .usb_handle = &g_usb_handle,
01092  *   .spi_handle = &g_spi_handle,
01093  *   .callback = on_frame_received,
01094  *   .callback_ctx = nullptr,
01095  *   .enable_decoded_output = true,
01096  * };
01097  * err = rx_comm_manager_init(&g_comm_mgr, &cfg);
01098  * @endcode
01099  *
01100  * @par Configuration Example - USB Only, No Callback:
01101  * @code{.c}
01102  * rx_comm_manager_config_t cfg = {
01103  *   .usb_handle = &g_usb_handle,
01104  *   .spi_handle = nullptr,           // SPI disabled
01105  *   .callback = nullptr,           // No callback
01106  *   .callback_ctx = nullptr,
01107  *   .enable_decoded_output = false, // No ASCII debugging
01108  * };
01109  * rx_err_t err = rx_comm_manager_init(&g_comm_mgr, &cfg);
01110  * @endcode
01111  *
01112  * @see rx_comm_manager_deinit() Cleanup and resource release

```

```

01113 * @see rx_comm_manager_poll() Poll for incoming frames
01114 * @see rx_spi_comm_init() Initialize SPI transport
01115 * @see rx_uart_comm_init() Initialize UART transport
01116 * @see rx_i2c_comm_init() Initialize I2C peripheral transport
01117 *
01118 * @since Version 1.0.0
01119 */
01120 rx_err_t rx_comm_manager_init(rx_comm_manager_t* mgr, const rx_comm_manager_config_t* cfg)
01121 {
01122     /* Rule 5: Pre-condition validation - nullptr check */
01123     if (mgr == nullptr) {
01124         rx_log_error(s_tag, "Init failed: NULL mgr");
01125         return k_rx_err_invalid_arg;
01126     }
01127
01128     /* Rule 5: Pre-condition validation - SPI link consistency checks */
01129     if (cfg != nullptr && cfg->spi_link != nullptr) {
01130         /* If spi_link is provided, spi_handle must also be provided */
01131         if (cfg->spi_handle == nullptr) {
01132             rx_log_error(s_tag, "Init failed: spi_link requires non-NULL spi_handle");
01133             return k_rx_err_invalid_arg;
01134         }
01135
01136         /* If spi_link is provided, it must be initialized */
01137         if (!cfg->spi_link->initialized) {
01138             rx_log_error(s_tag, "Init failed: spi_link not initialized (call rx_spi_link_init first)");
01139             return k_rx_err_invalid_arg;
01140         }
01141
01142         /* If spi_link is provided, its spi_handle must match config spi_handle */
01143         if (cfg->spi_link->spi_handle != cfg->spi_handle) {
01144             rx_log_error(s_tag, "Init failed: spi_link->spi_handle != cfg->spi_handle");
01145             return k_rx_err_invalid_arg;
01146         }
01147     }
01148
01149     *mgr = s_zero_mgr;
01150
01151     /* Apply configuration */
01152     if (cfg != nullptr) {
01153         mgr->uart_handle = cfg->uart_handle;
01154         mgr->spi_handle = cfg->spi_handle;
01155         mgr->i2c_handle = cfg->i2c_handle;
01156         mgr->spi_link = cfg->spi_link;
01157         mgr->callback = cfg->callback;
01158         mgr->callback_ctx = cfg->callback_ctx;
01159         mgr->enable_decoded_output = cfg->enable_decoded_output;
01160         mgr->link_status_cb = cfg->link_status_cb;
01161         mgr->link_status_ctx = cfg->link_status_ctx;
01162     }
01163
01164     mgr->initialized = true;
01165
01166     rx_log_info(s_tag, "Comm manager initialized");
01167     return k_rx_ok;
01168 }
01169 /**
01170 * @brief Deinitialize communication manager and release resources
01171 *
01172 * @details
01173 * Marks communication manager as uninitialized, preventing further operations.
01174 * This function does NOT deinitialize USB or SPI transport handles - caller
01175 * must separately call rx_usb_comm_deinit() and rx_spi_comm_deinit() if needed.
01176 *
01177 * **Algorithm:**
01178 * 1. Validate mgr pointer (NULL check)
01179 * 2. Validate initialized flag (must be initialized to deinitialize)
01180 * 3. Clear initialized flag (marks handle as invalid)
01181 *
01182 * **Design Note**: Minimal deinitialization - no dynamic resources to free
01183 * (all memory is statically allocated). Function primarily serves as safety
01184 * guard to prevent use-after-deinit.
01185 *
01186 *
01187 *
01188 *
01189 * @pre mgr must be non-NULL
01190 * @pre mgr must be initialized (mgr->initialized == true)
01191 * @post mgr->initialized = false
01192 * @post Subsequent calls to rx_comm_manager_poll/send will return k_rx_err_invalid_state
01193 *
01194 * @note Does NOT free memory (handle is statically allocated)
01195 * @note Does NOT deinitialize transport handles (USB/SPI)
01196 * @note Thread-safe if no concurrent operations on same mgr handle
01197 *
01198 * @par Example:
01199 * @code{.c}

```

```

01200 * // Deinitialize manager
01201 * rx_err_t err = rx_comm_manager_deinit(&g_comm_mgr);
01202 * if (err == k_rx_ok) {
01203 *     // Manager no longer usable
01204 *
01205 *     // Separately deinitialize transports if needed
01206 *     rx_usb_comm_deinit(&g_usb_handle);
01207 *     rx_spi_comm_deinit(&g_spi_handle);
01208 * }
01209 * @endcode
01210
01211 * @see rx_comm_manager_init() Initialization
01212 * @see rx_spi_comm_deinit() SPI transport cleanup
01213 *
01214 * @since Version 1.0.0
01215 */
01216 rx_err_t rx_comm_manager_deinit(rx_comm_manager_t* mgr)
01217 {
01218     /* Rule 5: Pre-condition validation */
01219     if (mgr == nullptr) {
01220         rx_log_error(s_tag, "Deinit failed: NULL mgr");
01221         return k_rx_err_invalid_arg;
01222     }
01223     if (!mgr->initialized) {
01224         rx_log_warn(s_tag, "Deinit called on uninitialized mgr");
01225         return k_rx_err_invalid_state;
01226     }
01227     mgr->initialized = false;
01228
01229     rx_log_info(s_tag, "Comm manager deinitialized");
01230     return k_rx_ok;
01231 }
01232
01233 /**
01234 * @brief Poll all enabled communication channels for incoming frames (non-blocking)
01235 *
01236 * @details
01237 * Performs non-blocking poll of all enabled channels (USB and SPI) sequentially,
01238 * dispatching any received frames to the registered user callback. Returns aggregate
01239 * result indicating whether any frames were received across all channels.
01240 *
01241 * **Algorithm:**
01242 * 1. **Validate parameters**: Check mgr pointer and initialized flag (NASA Rule 5)
01243 * 2. **Poll USB channel** via internal_poll_usb():
01244 *   - If k_rx_ok: Mark received=true, continue to SPI
01245 *   - If k_rx_err_timeout: Continue to SPI (no data is normal)
01246 *   - If other error: Return error immediately (critical failure)
01247 * 3. **Poll SPI channel** via internal_poll_spi():
01248 *   - If k_rx_ok: Mark received=true
01249 *   - If k_rx_err_timeout: Continue (no data is normal)
01250 *   - If other error: Return error immediately (critical failure)
01251 * 4. **Return result**:
01252 *   - k_rx_ok if ANY channel received frame
01253 *   - k_rx_err_timeout if NO channels received (normal condition)
01254 *   - Other error codes propagated from transport layers
01255 *
01256 * **Performance**: 15-200 us typical, depends on:
01257 * - Number of enabled channels (USB/SPI/both)
01258 * - Whether frames are available (data path vs fast timeout path)
01259 * - ASCII debugging enabled (adds ~50-150 us per frame)
01260 * - User callback execution time (not included in measurement)
01261 *
01262 * **Design Rationale**:
01263 * - **Non-blocking**: Never blocks waiting for data (0 ms timeout)
01264 * - **Fair scheduling**: Both channels polled each iteration
01265 * - **Timeout tolerance**: Timeout is expected, not error condition
01266 * - **Error prioritization**: Critical errors returned immediately
01267 * - **Aggregation**: k_rx_ok if ANY channel received data
01268 *
01269 *
01270 *
01271 * @pre mgr must be non-NULL
01272 * @pre mgr must be initialized
01273 * @pre At least one channel should be enabled (usb_handle or spi_handle non-NULL)
01274 * @post User callback invoked for each successfully received frame
01275 * @post ASCII output sent to Port 1 if enabled
01276 *
01277 * @note Non-blocking - always returns immediately
01278 * @note k_rx_err_timeout is NORMAL - means no data available (not an error)
01279 * @note User callback execution time is NOT included in performance measurements
01280 * @warning Call this function regularly (e.g., 100 Hz) to avoid buffer overruns
01281 * @warning User callback must not block for extended periods
01282 *
01283 * @par Performance:
01284 * - No frames available: ~15 us (both channels timeout, fast path)
01285 * - USB frame received: ~100-250 us (USB bulk transfer + callback)

```

```

01287 * - SPI frame received: ~50-150 us (SPI hardware transfer + callback)
01288 * - Both frames: ~150-400 us (both channels + callbacks)
01289 * - With ASCII debugging: Add ~50-150 us per frame
01290 *
01291 * @par Typical Usage - ThreadX Communication Task:
01292 * @code{.c}
01293 * static void comm_task_entry(ULONG thread_input)
01294 * {
01295 *     (void)thread_input;
01296 *
01297 *     // Initialize communication manager
01298 *     // ... (see rx_comm_manager_init() documentation) ...
01299 *
01300 *     // Main polling loop @ 100 Hz
01301 *     while (1) {
01302 *         // Poll all channels (non-blocking)
01303 *         rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
01304 *
01305 *         // Handle critical errors (not timeout)
01306 *         if (err != k_rx_ok && err != k_rx_err_timeout) {
01307 *             // Log error, implement recovery strategy
01308 *             log_error("Comm poll failed: %d", err);
01309 *             tx_thread_sleep(100); // 1 second backoff
01310 *             continue;
01311 *         }
01312 *
01313 *         // Timeout is normal - just means no data available
01314 *
01315 *         // Sleep 10 ticks (100 ms @ 100 Hz tick rate = 10 Hz polling)
01316 *         tx_thread_sleep(10);
01317 *     }
01318 * }
01319 * @endcode
01320 *
01321 * @par Error Handling - Distinguishing Timeout from Errors:
01322 * @code{.c}
01323 * rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
01324 * if (err == k_rx_ok) {
01325 *     // At least one frame received and processed
01326 * } else if (err == k_rx_err_timeout) {
01327 *     // No frames available - this is NORMAL, not an error
01328 * } else {
01329 *     // Critical error - handle appropriately
01330 *     handle_comm_error(err);
01331 * }
01332 * @endcode
01333 *
01334 * @see internal_poll_usb() USB channel polling implementation
01335 * @see internal_poll_spi() SPI channel polling implementation
01336 * @see internal_handle_frame() Frame dispatch
01337 * @see rx_comm_manager_init() Initialization with channel configuration
01338 *
01339 * @since Version 1.0.0
01340 */
01341 /**
01342 * @brief Poll all transport channels and aggregate results
01343 *
01344 *
01345 *
01346 * @pre mgr must be non-NULL and initialized
01347 * @post *received reflects whether any frames were received
01348 *
01349 * @since Version 1.0.0
01350 */
01351 static rx_err_t internal_poll_all_channels(rx_comm_manager_t* mgr, bool* received)
01352 {
01353     /* Poll UART channel (primary path to the Pi5 gateway) */
01354     rx_err_t uart_err = internal_poll_uart(mgr);
01355     if (uart_err == k_rx_ok) {
01356         *received = true;
01357     } else if (uart_err != k_rx_err_timeout && uart_err != k_rx_err_no_data) {
01358         rx_log_error_val(s_tag, "UART poll error", uart_err);
01359         return uart_err;
01360     }
01361
01362     /* Poll SPI channel */
01363     rx_err_t spi_err = internal_poll_spi(mgr);
01364     if (spi_err == k_rx_ok) {
01365         *received = true;
01366     } else if (spi_err != k_rx_err_timeout) {
01367         rx_log_error_val(s_tag, "SPI poll error", spi_err);
01368         return spi_err;
01369     }
01370
01371     /* Poll I2C channel */
01372     rx_err_t i2c_err = internal_poll_i2c(mgr);
01373     if (i2c_err == k_rx_ok) {

```

```

01374     *received = true;
01375 } else if (i2c_err != k_rx_err_timeout && i2c_err != k_rx_err_no_data) {
01376     rx_log_error_val(s_tag, "I2C poll error", i2c_err);
01377     return i2c_err;
01378 }
01379
01380 return k_rx_ok;
01381 }
01382
01383 rx_err_t rx_comm_manager_poll(rx_comm_manager_t* mgr)
01384 {
01385     /* Rule 5: Pre-condition validation */
01386     if (mgr == nullptr) {
01387         return k_rx_err_invalid_arg;
01388     }
01389     if (!mgr->initialized) {
01390         return k_rx_err_invalid_state;
01391     }
01392
01393     bool    received = false;
01394     rx_err_t poll_err = internal_poll_all_channels(mgr, &received);
01395     if (poll_err != k_rx_ok) {
01396         return poll_err;
01397     }
01398
01399     /* Process event queue: send one queued event per poll cycle */
01400     internal_process_event_queue(mgr);
01401
01402     /* Heartbeat: check implicit timeout on each transport */
01403     internal_check_heartbeat(mgr);
01404
01405     return (int)received ? k_rx_ok : k_rx_err_timeout;
01406 }
01407
01408 /**
01409  * @brief Send frame on specified communication channel
01410  *
01411  * @details
01412  * Routes frame transmission to the appropriate transport layer (USB or SPI) based
01413  * on specified channel. Supports configurable frame type, flags, and variable-length
01414  * payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.
01415  *
01416  * **Algorithm:**
01417  * 1. **Validate parameters** (NASA Rule 5):
01418  *    - mgr and params pointers non-NULL
01419  *    - Manager initialized flag
01420  *    - Payload pointer valid if payload_len > 0
01421  *    - payload_len <= k_frame_max_payload (255 bytes)
01422  * 2. **Route to channel**::
01423  *    - USB: Verify usb_handle non-NULL, call rx_usb_comm_send()
01424  *    - SPI: Verify spi_handle non-NULL, call rx_spi_comm_send()
01425  *    - Invalid channel: Return k_rx_err_invalid_arg
01426  * 3. **On success, if ASCII debugging enabled**::
01427  *    - Build temporary rx_frame_t from params
01428  *    - Format as ASCII via internal_output_decoded()
01429  *    - Send to USB Port 1 (TX direction)
01430  *
01431  * **Performance**::
01432  * - USB: ~30-150 us (depends on payload size, USB bulk transfer)
01433  * - SPI: ~20-100 us (depends on payload size, SPI hardware transfer)
01434  * - ASCII debugging: Add ~50-150 us if enabled
01435  *
01436  *
01437  *
01438  *
01439  * @pre mgr must be non-NULL and initialized
01440  * @pre params must be non-NULL with all valid fields
01441  * @pre Channel handle (USB/SPI) must be initialized
01442  * @pre If payload_len > 0, payload must point to valid data
01443  * @post Frame transmitted on specified channel if successful
01444  * @post ASCII output sent to Port 1 if enabled and successful
01445  *
01446  * @note Payload sequence number and CRC-32 managed by transport layer
01447  * @note ASCII debugging adds overhead - disable in production
01448  * @warning Ensure channel is ready before sending (use rx_comm_manager_channel_ready)
01449  *
01450  * @par Send Parameters Structure:
01451  * @code{.c}
01452  * typedef struct {
01453  *     rx_comm_channel_t channel;    // USB or SPI
01454  *     uint8_t          type;        // Frame type
01455  *     uint8_t          flags;       // Frame flags
01456  *     const uint8_t*    payload;    // Payload data
01457  *     uint32_t          payload_len; // Payload length [0, 255]
01458  * } rx_comm_send_params_t;
01459  * @endcode
01460  */

```



```

01461 * @par Example - Send Command on USB:
01462 * @code{.c}
01463 * uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
01464 *
01465 * rx_comm_send_params_t params = {
01466 *     .channel = k_comm_channel_usb,
01467 *     .type = k_frame_type_command,
01468 *     .flags = k_frame_flag_none,
01469 *     .payload = cmd_payload,
01470 *     .payload_len = sizeof(cmd_payload),
01471 * };
01472 *
01473 * rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
01474 * if (err == k_rx_ok) {
01475 *     // Command sent successfully
01476 * }
01477 * @endcode
01478 *
01479 * @par Example - Send Empty Frame (No Payload):
01480 * @code{.c}
01481 * rx_comm_send_params_t params = {
01482 *     .channel = k_comm_channel_spi,
01483 *     .type = k_frame_type_response,
01484 *     .flags = k_frame_flag_ack,
01485 *     .payload = nullptr, // No payload
01486 *     .payload_len = 0, // Zero length
01487 * };
01488 *
01489 * rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
01490 * @endcode
01491 *
01492 * @see rx_comm_manager_respond() Convenience wrapper for response frames
01493 * @see rx_comm_send_params_t Send parameters structure
01494 * @see rx_spi_comm_send() SPI send implementation
01495 * @see rx_uart_comm_send() UART send implementation
01496 * @see rx_i2c_comm_send() I2C peripheral send implementation
01497 *
01498 * @since Version 1.0.0
01499 */
01500 /**
01501 * @brief Route frame to transport layer based on channel
01502 *
01503 *
01504 *
01505 * @pre mgr and params must be non-NULL
01506 * @pre params validated by caller (payload length, etc.)
01507 * @post Frame sent on specified channel transport
01508 *
01509 * @since Version 1.0.0
01510 */
01511 static rx_err_t internal_route_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01512 {
01513     switch (params->channel) {
01514     case k_comm_channel_spi:
01515         if (mgr->spi_handle == nullptr) {
01516             rx_log_error(s_tag, "Send failed: SPI handle NULL");
01517             return k_rx_err_invalid_state;
01518         }
01519         /* Use HARQ link layer if available, otherwise raw SPI */
01520         if (mgr->spi_link != nullptr) {
01521             /* NOTE: params->flags intentionally NOT forwarded to rx_spi_link_send().
01522              * The link layer manages its own flags internally (k_frame_flag_requires_ack,
01523              * k_frame_flag_fec_enabled, k_frame_flag_retransmit) based on HARQ state. */
01524             return rx_spi_link_send(mgr->spi_link, params->type, params->payload, params->payload_len);
01525         }
01526         return rx_spi_comm_send(mgr->spi_handle,
01527                                 params->type,
01528                                 params->flags,
01529                                 params->payload,
01530                                 params->payload_len);
01531     case k_comm_channel_i2c:
01532         if (mgr->i2c_handle == nullptr) {
01533             rx_log_error(s_tag, "Send failed: I2C handle NULL");
01534             return k_rx_err_invalid_state;
01535         }
01536         return rx_i2c_comm_send(mgr->i2c_handle,
01537                                 params->type,
01538                                 params->flags,
01539                                 params->payload,
01540                                 params->payload_len);
01541     case k_comm_channel_uart:
01542         if (mgr->uart_handle == nullptr) {
01543             rx_log_error(s_tag, "Send failed: UART handle NULL");
01544             return k_rx_err_invalid_state;
01545         }
01546     }
01547 }

```



```

01548     return rx_uart_comm_send(mgr->uart_handle,
01549                             params->type,
01550                             params->flags,
01551                             params->payload,
01552                             params->payload_len);
01553
01554     default:
01555         rx_log_error(s_tag, "Send failed: invalid channel");
01556         return k_rx_err_invalid_arg;
01557 }
01558 }
01559
01560 rx_err_t rx_comm_manager_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01561 {
01562     /* Rule 5: Pre-condition validation */
01563     if (mgr == nullptr || params == nullptr) {
01564         rx_log_error(s_tag, "Send failed: NULL arg");
01565         return k_rx_err_invalid_arg;
01566     }
01567     if (!mgr->initialized) {
01568         rx_log_error(s_tag, "Send failed: not initialized");
01569         return k_rx_err_invalid_state;
01570     }
01571     if (params->payload == nullptr && params->payload_len > 0) {
01572         rx_log_error(s_tag, "Send failed: NULL payload with nonzero len");
01573         return k_rx_err_invalid_arg;
01574     }
01575     /* Validate payload length is within frame limits */
01576     if (params->payload_len > k_frame_max_payload) {
01577         rx_log_error_val(s_tag, "Send failed: payload too large", params->payload_len);
01578         return k_rx_err_invalid_arg;
01579     }
01580
01581     rx_err_t err = internal_route_send(mgr, params);
01582
01583     if (err != k_rx_ok) {
01584         rx_log_error_val(s_tag, "Transport send failed", err);
01585     }
01586
01587     /* Output decoded ASCII on success */
01588     if (err == k_rx_ok && (int)mgr->enable_decoded_output) {
01589         internal_output_decoded_from_params(mgr, params);
01590     }
01591
01592     return err;
01593 }
01594
01595 /**
01596  * @brief Send response frame on specified channel (convenience wrapper)
01597  *
01598  * @details
01599  * Convenience function for sending response frames. Automatically sets frame type
01600  * to k_frame_type_response and flags to k_frame_flag_none. Simplifies common use
01601  * case of responding to commands/requests.
01602  *
01603  * **Algorithm:**
01604  * 1. Build rx_comm_send_params_t with:
01605  *   - channel: User-specified
01606  *   - type: k_frame_type_response (fixed)
01607  *   - flags: k_frame_flag_none (fixed)
01608  *   - payload: User-specified
01609  *   - payload_len: User-specified
01610  * 2. Call rx_comm_manager_send() with constructed params
01611  * 3. Return result from rx_comm_manager_send()
01612  *
01613  * **When to Use:**
01614  * - Responding to commands with telemetry data
01615  * - Sending acknowledgments
01616  * - Replying to requests
01617  *
01618  * **When NOT to Use:**
01619  * - Need custom frame type (use rx_comm_manager_send instead)
01620  * - Need custom flags (use rx_comm_manager_send instead)
01621  *
01622  *
01623  *
01624  *
01625  *
01626  *
01627  * @pre Same preconditions as rx_comm_manager_send()
01628  * @post Response frame transmitted with type=response, flags=none
01629  *
01630  * @note Equivalent to calling rx_comm_manager_send() with type=k_frame_type_response
01631  * @note This is a thin wrapper - no additional validation performed
01632  *
01633  * @par Example - Send Telemetry Response:
01634  * @code{.c}

```

```

01635 * // Received a telemetry request on USB, send response
01636 * static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
01637 * {
01638 *     uint8_t telemetry[16];
01639 *
01640 *     // Gather telemetry data
01641 *     telemetry[0] = get_temperature_celsius();
01642 *     telemetry[1] = get_current_ma();
01643 *     // ... fill remaining fields ...
01644 *
01645 *     // Send response on same channel request came from
01646 *     rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
01647 *                                           channel,
01648 *                                           telemetry,
01649 *                                           sizeof(telemetry));
01650 *     if (err != k_rx_ok) {
01651 *         // Handle send error
01652 *     }
01653 * }
01654 * @endcode
01655 *
01656 * @par Example - Send Empty Acknowledgment:
01657 * @code{.c}
01658 * // Send empty ACK response
01659 * rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
01660 *                                       k_comm_channel_usb,
01661 *                                       NULL, // No payload
01662 *                                       0); // Zero length
01663 * @endcode
01664 *
01665 * @see rx_comm_manager_send() Full send function with all parameters
01666 * @see rx_comm_send_params_t Send parameters structure
01667 *
01668 * @since Version 1.0.0
01669 */
01670 rx_err_t rx_comm_manager_respond(rx_comm_manager_t* mgr,
01671                                 rx_comm_channel_t channel,
01672                                 const uint8_t* payload,
01673                                 uint32_t payload_len)
01674 {
01675     const rx_comm_send_params_t params = {
01676         .channel = channel,
01677         .type = k_frame_type_response,
01678         .flags = k_frame_flag_none,
01679         .payload = payload,
01680         .payload_len = payload_len,
01681     };
01682     return rx_comm_manager_send(mgr, &params);
01683 }
01684
01685 /=====
01686 * Stream / Event Send API
01687 *
01688 *=====
01689 */
01690 rx_err_t rx_comm_manager_stream_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01691 {
01692     /* Stream path: fire-and-forget, no retries, no queuing.
01693     * Telemetry and other time-sensitive data goes here.
01694     * Stale data has no value, so failure is silently accepted. */
01695     return rx_comm_manager_send(mgr, params);
01696 }
01697
01698 rx_err_t rx_comm_manager_event_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01699 {
01700     if (mgr == nullptr || params == nullptr) {
01701         rx_log_error(s_tag, "Event enqueue failed: NULL arg");
01702         return k_rx_err_invalid_arg;
01703     }
01704
01705     if (!mgr->initialized) {
01706         rx_log_error(s_tag, "Event enqueue failed: not initialized");
01707         return k_rx_err_invalid_state;
01708     }
01709
01710     if (params->payload_len > k_frame_max_payload) {
01711         rx_log_error_val(s_tag, "Event enqueue failed: payload too large", params->payload_len);
01712         return k_rx_err_invalid_arg;
01713     }
01714
01715     if (params->payload_len > 0 && params->payload == nullptr) {
01716         rx_log_error(s_tag, "Event enqueue failed: NULL payload with nonzero len");
01717         return k_rx_err_invalid_arg;
01718     }
01719 }

```

```

01720 /* Check queue capacity */
01721 if (mgr->event_queue_count >= k_comm_event_queue_depth) {
01722     rx_log_error_val(s_tag, "Event queue full, depth", (uint8_t)mgr->event_queue_count);
01723     return k_rx_err_no_mem;
01724 }
01725
01726 /* Enqueue: copy payload into static slot */
01727 rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_head];
01728 entry->channel = params->channel;
01729 entry->type = params->type;
01730 entry->flags = params->flags;
01731 entry->payload_len = params->payload_len;
01732 entry->retries = 0;
01733 entry->occupied = true;
01734
01735 if (params->payload != nullptr && params->payload_len > 0) {
01736     for (uint32_t i = 0; i < params->payload_len; i++) {
01737         entry->payload[i] = params->payload[i];
01738     }
01739 }
01740
01741 mgr->event_queue_head = (mgr->event_queue_head + 1) % k_comm_event_queue_depth;
01742 mgr->event_queue_count++;
01743
01744 rx_log_debug_val(s_tag, "Event enqueued, queue depth", mgr->event_queue_count);
01745 return k_rx_ok;
01746 }
01747
01748 /*
=====
01749 * Heartbeat API
01750 *
=====
01751 */
01752
01753 rx_err_t rx_comm_manager_link_status(const rx_comm_manager_t* mgr,
01754                                     rx_comm_channel_t channel,
01755                                     rx_comm_link_status_t* status)
01756 {
01757     if (mgr == nullptr || status == nullptr) {
01758         rx_log_error(s_tag, "Link status query: NULL arg");
01759         return k_rx_err_invalid_arg;
01760     }
01761
01762     if (!mgr->initialized) {
01763         rx_log_error(s_tag, "Link status query: uninitialized manager");
01764         return k_rx_err_invalid_state;
01765     }
01766
01767     if (channel >= k_comm_channel_count) {
01768         rx_log_error(s_tag, "Link status query: invalid channel");
01769         return k_rx_err_invalid_arg;
01770     }
01771
01772     *status = mgr->heartbeat[channel].status;
01773     return k_rx_ok;
01774 }
01775
01776 /**
01777  * @brief Query if communication channel is ready for send/receive operations
01778  *
01779  * @details
01780  * Checks if specified channel is configured and ready for communication. USB channel
01781  * readiness depends on USB enumeration and configuration by host. SPI channel is
01782  * ready immediately if handle is configured (no enumeration required).
01783  *
01784  * **Readiness Criteria**:
01785  * - **USB**: usb_handle non-NULL AND USB device enumerated and configured by host
01786  * - **SPI**: spi_handle non-NULL (always ready if configured)
01787  *
01788  * **Use Cases**:
01789  * - Check readiness before sending to avoid errors
01790  * - Implement fallback logic (try USB, fall back to SPI)
01791  * - Log channel availability status
01792  *
01793  *
01794  *
01795  *
01796  *
01797  * @pre mgr must be non-NULL and initialized
01798  * @pre ready must be non-NULL
01799  * @pre channel must be valid (USB or SPI)
01800  * @post *ready set to true or false based on channel state
01801  * @post On error, *ready set to false (safe default)
01802  *
01803  * @note USB readiness can change at runtime (host connects/disconnects)
01804  * @note SPI readiness is static (always ready if handle configured)

```

```

01805 * @note This function does NOT initiate communication - only queries state
01806 *
01807 * @par Example - Send with Fallback:
01808 * @code{.c}
01809 * uint8_t payload[16] = { ... };
01810 *
01811 * // Try USB first
01812 * bool usb_ready = false;
01813 * rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
01814 *                                             k_comm_channel_usb,
01815 *                                             &usb_ready);
01816 * if (err == k_rx_ok && usb_ready) {
01817 *     err = rx_comm_manager_respond(&g_comm_mgr,
01818 *                                 k_comm_channel_usb,
01819 *                                 payload,
01820 *                                 sizeof(payload));
01821 * } else {
01822 *     // Fallback to SPI
01823 *     bool spi_ready = false;
01824 *     err = rx_comm_manager_channel_ready(&g_comm_mgr,
01825 *                                       k_comm_channel_spi,
01826 *                                       &spi_ready);
01827 *     if (err == k_rx_ok && spi_ready) {
01828 *         err = rx_comm_manager_respond(&g_comm_mgr,
01829 *                                     k_comm_channel_spi,
01830 *                                     payload,
01831 *                                     sizeof(payload));
01832 *     }
01833 * }
01834 * @endcode
01835 *
01836 * @par Example - Log Channel Status:
01837 * @code{.c}
01838 * void log_channel_status(void)
01839 * {
01840 *     bool usb_ready = false, spi_ready = false;
01841 *
01842 *     rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_usb, &usb_ready);
01843 *     rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_spi, &spi_ready);
01844 *
01845 *     printf("USB: %s, SPI: %s\n",
01846 *          usb_ready ? "READY" : "NOT READY",
01847 *          spi_ready ? "READY" : "NOT READY");
01848 * }
01849 * @endcode
01850 *
01851 * @see rx_usb_is_configured() USB enumeration status
01852 * @see rx_comm_manager_send() Send frame on channel
01853 *
01854 * @since Version 1.0.0
01855 */
01856 rx_err_t
01857 rx_comm_manager_channel_ready(rx_comm_manager_t* mgr, rx_comm_channel_t channel, bool* ready)
01858 {
01859     /* Rule 5: Pre-condition validation */
01860     if (mgr == nullptr || ready == nullptr) {
01861         if (ready != nullptr) {
01862             *ready = false;
01863         }
01864         return k_rx_err_invalid_arg;
01865     }
01866     if (!mgr->initialized) {
01867         *ready = false;
01868         return k_rx_err_invalid_state;
01869     }
01870     switch (channel) {
01871     case k_comm_channel_uart:
01872         *ready = (bool)(mgr->uart_handle != nullptr);
01873         break;
01874     case k_comm_channel_spi:
01875         *ready = (bool)(mgr->spi_handle != nullptr);
01876         break;
01877     case k_comm_channel_i2c:
01878         *ready = (bool)(mgr->i2c_handle != nullptr);
01879         break;
01880     default:
01881         *ready = false;
01882         rx_log_error(s_tag, "Channel ready: invalid channel");
01883         return k_rx_err_invalid_arg;
01884     }
01885     return k_rx_ok;
01886 }

```

```

01892
01893 /**
01894  * @brief Get human-readable name for communication channel
01895  *
01896  * @details
01897  * Returns static string name for given channel enum value. Used for logging,
01898  * debugging, and user interface display. Always returns a valid string (never NULL).
01899  *
01900  * **Returned Strings**:
01901  * - k_comm_channel_usb -> "USB"
01902  * - k_comm_channel_spi -> "SPI"
01903  * - Invalid/unknown -> "UNKNOWN"
01904  *
01905  * **Use Cases**:
01906  * - Logging channel activity
01907  * - Debugging frame traffic
01908  * - User interface display
01909  * - Error messages
01910  *
01911  *
01912  *
01913  * @pre None (accepts any input)
01914  * @post Returns pointer to static string (valid for program lifetime)
01915  *
01916  * @note Returned string is static - do not free
01917  * @note Thread-safe (returns read-only static strings)
01918  * @note Never returns NULL - always safe to use in printf
01919  *
01920  * @par Example - Logging:
01921  * @code{.c}
01922  * static void on_frame_received(rx_comm_channel_t channel,
01923  *                               const rx_frame_t* frame,
01924  *                               void* ctx)
01925  * {
01926  *     printf("Frame received on %s: type=%d len=%d\n",
01927  *           rx_comm_manager_channel_name(channel),
01928  *           frame->header.type,
01929  *           frame->header.length);
01930  * }
01931  * @endcode
01932  *
01933  * @par Example - Error Messages:
01934  * @code{.c}
01935  * rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
01936  * if (err != k_rx_ok) {
01937  *     fprintf(stderr, "Failed to send on %s: error %d\n",
01938  *           rx_comm_manager_channel_name(params.channel), err);
01939  * }
01940  * @endcode
01941  *
01942  * @see rx_comm_channel_t Channel enumeration
01943  *
01944  * @since Version 1.0.0
01945  */
01946 const char* rx_comm_manager_channel_name(rx_comm_channel_t channel)
01947 {
01948     switch (channel) {
01949         case k_comm_channel_uart:
01950             return "UART";
01951         case k_comm_channel_spi:
01952             return "SPI";
01953         case k_comm_channel_i2c:
01954             return "I2C";
01955         default:
01956             return "UNKNOWN";
01957     }
01958 }
01959
01960 /*
=====
01961  * Retransmission Pass-Through
01962  *
=====
01963  */
01964
01965 rx_err_t rx_comm_manager_process_retransmits(rx_comm_manager_t* mgr, const uint32_t current_time_ms)
01966 {
01967     if (mgr == nullptr) {
01968         return k_rx_err_invalid_arg;
01969     }
01970
01971     if (!mgr->initialized) {
01972         return k_rx_err_invalid_state;
01973     }
01974
01975     if (mgr->spi_handle == nullptr) {
01976         return k_rx_ok; /* No SPI channel configured */

```

```
01977 }
01978
01979 return rx_spi_comm_process_retransmits(mgr->spi_handle, current_time_ms);
01980 }
01981
01982 rx_err_t rx_comm_manager_set_auto_retransmit(rx_comm_manager_t* mgr,
01983 const bool enabled,
01984 const rx_spi_comm_retransmit_config_t* config)
01985 {
01986 if (mgr == nullptr) {
01987 return k_rx_err_invalid_arg;
01988 }
01989
01990 if (!mgr->initialized) {
01991 return k_rx_err_invalid_state;
01992 }
01993
01994 if (mgr->spi_handle == nullptr) {
01995 return k_rx_err_not_supported;
01996 }
01997
01998 return rx_spi_comm_set_auto_retransmit(mgr->spi_handle, enabled, config);
01999 }
```